



Cairo University
Faculty of Engineering
Department of Computer Engineering

Accelera



A Graduation Project Report Submitted
to
Faculty of Engineering, Cairo University
in Partial Fulfillment of the Requirements for the Degree
of
Bachelor of Science in Computer Engineering.

Presented by

Mohamed Ashraf
Mazen Adel

Nesma Osama
Mohamed Khater

Supervised by

Dr. Salma Abdel Monem

June 30, 2026

All rights reserved. This report may not be reproduced in whole or in part, by photocopying or other means, without the permission of the authors or department.

Abstract

Accelerera is a hybrid Python/C++ machine-learning framework designed to support the complete data-science workflow, from data preparation and model selection to pipeline execution, code acceleration, deployment, and benchmark comparison. The framework combines a high-level Python interface with native execution components to improve usability, organization, and performance in machine-learning experiments.

The system is organized around six main modules: Auto Preprocessing, Accelerera AutoML, Accelerera Pipe, the Code Parallelizer, Deployment, and the Benchmark Platform. The Auto Preprocessing module automates common preparation steps for tabular, text, and image data, including missing-value handling, encoding, scaling, feature selection, text vectorization, and report generation. The Accelerera AutoML module supports classification and regression tasks by searching over candidate models, preprocessing choices, and hyperparameter configurations under time and trial constraints.

Accelerera Pipe represents machine-learning workflows as directed graph nodes for preprocessing, modelling, prediction, metric evaluation, branching, and merging. This graph-based design helps organize complex workflows and supports reuse of shared execution paths. The Code Parallelizer analyzes C/C++ and supported Python code, extracts loop features, predicts parallelization opportunities, applies safety rules, and emits OpenMP-annotated C++ code when parallelization is safe. The Benchmark Platform provides a foundation for comparing datasets, submissions, metrics, and leaderboard results, while the Deployment module supports the future use of trained models as deployable services.

This report presents the motivation, market visibility study, literature survey, system design, implementation methodology, testing, and experimental results of Accelerera. The results show that the implemented modules can reduce repetitive machine-learning development work, improve workflow organization, and provide a basis for future performance optimization and fair benchmark comparison.

Arabic Abstract

مشروع أكسيليرا هو إطار عمل هجين للتعليم الآلي مبني باستخدام بايثون وسي بلس بلس، ويجمع بين واجهة بايثون عالية المستوى ومكونات تنفيذ أصلية. يتكون النظام من ست وحدات، وهي أكسيليرا بايب، ووحدة موازنة الشفرة، والمعالجة المسبقة التلقائية، والتعلم الآلي التلقائي، والنشر، ومنصة القياس المرجعي. تمثل أكسيليرا بايب سير عمل التعلم الآلي في صورة عقد ضمن رسم بياني موجه للمعالجة المسبقة، وبناء النماذج، والتنبؤ، وتقييم المقاييس، والتفرع، والدمج. تحلل وحدة موازنة الشفرة شفرات سي وسي بلس بلس وشفرات بايثون المدعومة، وتستخرج خصائص الحلقات، وتنبأ بفرص التنفيذ المتوازي، وتنتج شفرة سي بلس بلس مزودة بتوجيهات أوبن إم بي. يوثق هذا التقرير التصميم والمنهجية والنتائج التجريبية المتاحة للوحدات المنفذة، مع الحفاظ على هيكل التقرير المطلوب لبقية الوحدات.

Acknowledgment

The authors would like to express sincere gratitude to Dr. Salma Abdelmonem for her supervision, guidance, and support throughout this project. The authors also thanks the Computer Engineering Department and everyone who supported the design, implementation, and evaluation of this work.

Contents

Abstract	2
Arabic Abstract	3
Acknowledgment	4
List of Abbreviations	14
List of Symbols	15
1 Introduction	17
1.1 Motivation and Justification	17
1.2 Project Objectives and Problem Definition	20
1.3 Project Outcomes	21
1.4 Document Organization	23
2 Market Visibility Study	24
2.1 Targeted Customers	24
2.2 Market Survey	26
2.2.1 AutoCleanML	26
2.2.2 scikit-learn	26
2.2.3 AutoML Tools	27
2.2.4 Code Acceleration Tools	27
2.2.5 Deployment and Hosting Platforms	28
2.3 Business Case and Financial Analysis	29
2.3.1 Business Case	29
2.3.2 Financial Analysis	31
3 Literature Survey	33
3.1 Background and Previous Work	33
3.1.1 Data Science and Machine Learning	33
3.1.1.1 Data Exploration and Preprocessing [1]	33
3.1.1.2 Model Evaluation Metrics [2]	34
3.1.2 Natural Language Processing (NLP) [3]	36
3.1.2.1 Term Frequency-Inverse Document Frequency (TF-IDF)	36

3.1.3	Data Augmentation	37
3.1.4	Machine Learning Pipelines	37
3.1.5	Graph-Based Pipeline Execution	37
3.1.6	Code Parallelization	38
3.1.7	OpenMP	38
3.1.8	Deployment	38
3.1.9	Model Deployment	38
3.1.10	Machine Learning Deployment	38
3.1.11	Cloud and Server Deployment	39
3.1.12	Web Services, APIs, and REST APIs	39
3.1.13	HTTP, HTTPS, JSON, URLs, and Ports	39
3.1.14	Docker and Containerization	39
3.1.15	Virtual Machines, AWS, and AWS EC2	39
3.1.16	Heroku and Container Hosting	39
3.1.17	SSH and CLI Tools	40
3.1.18	Environment Variables and Configuration Files	40
3.1.19	Pickle Files and Model Artifacts	40
3.1.20	Version Control, Git, GitHub, and Repositories	40
3.1.21	Backend Services, FastAPI, and Uvicorn	40
3.1.22	Deployment Scripts	40
3.2	Comparative Study of Previous Work	41
3.2.1	Data Preprocessing and Feature Engineering for Data Mining: Techniques, Tools, and Best Practices [4]	41
3.2.2	scikit-learn Pipeline	41
3.2.3	OMPar: Automatic Parallelization with AI-Driven Source-to-Source Compilation	41
3.2.4	Combined Algorithm Selection and Hyperparameter Optimization	42
3.2.5	Search Strategies for AutoML	42
3.2.6	Multi-Fidelity Evaluation	42
3.2.7	Meta-Learning for Warm Starts	43
3.2.8	Ensemble Learning	43
3.2.9	Related AutoML Systems	44
3.2.10	Model Deployment Platforms	44
3.3	Implemented Approach	45
3.3.1	Auto Preprocessing	45
3.3.2	Benchmark Platform	46
3.3.3	accelera_pipe	46
3.3.4	Parallelizer	47
3.3.5	Deployment Module Technologies	48
4	System Design and Architecture	50
4.1	Overview and Assumptions	50

4.1.1	Accelera Pipeline Assumptions	50
4.1.2	Auto Preprocessing Assumptions	50
4.1.3	AutoML Assumptions	50
4.1.4	Benchmark Assumptions	50
4.1.5	Deployment Assumptions	51
4.2	System Architecture	52
4.2.1	Block Diagram	52
4.3	Accelera Pipe Module	60
4.3.1	Functional Description	60
4.3.2	Modular Decomposition	61
4.3.3	Python-to-Native Graph Construction	63
4.3.4	Preprocessing Execution, Copy Policy, and Optional Cache	65
4.3.5	Branching and Path Selection	67
4.3.6	Executed Graph Creation and Pickle State	68
4.3.7	Intermediate Memory Management	69
4.3.8	Parallel Graph Execution	70
4.3.9	Design Constraints	71
4.3.10	Summary	72
4.4	Code Parallelizer Module	73
4.4.1	Functional Description	73
4.4.2	Modular Decomposition	75
4.4.3	Input Dispatch and Source Normalization	77
4.4.4	Deterministic Pragma Generation Without AI	79
4.4.5	Feature Vector and Source Rewriting Details	81
4.4.6	Native Wrapper, Compilation, and Cache Layers	81
4.4.7	Integration with Accelera Pipe	83
4.4.8	Design Constraints	83
4.4.9	Summary	84
4.5	Auto Preprocessing Module	85
4.5.1	Functional Description	86
4.5.1.1	Tabular datasets	87
4.5.1.2	Text datasets	87
4.5.1.3	Image datasets	87
4.5.2	Modular Decomposition	87
4.5.2.1	Tabular Data Preprocessing Functions	87
4.5.2.2	Text Data Preprocessing Functions	92
4.5.2.3	Image Data Preprocessing	94
4.5.2.4	Inference and Reusability:	98
4.5.3	Design Constraints	98
4.5.4	Module Summary	98
4.6	AutoML Module	98
4.6.1	Functional Description	98

4.6.2	Block Diagram	99
4.6.3	Accelera AutoML Module Design	100
4.6.3.1	Method Overview	100
4.6.3.2	Configuration-Space Construction	101
4.6.3.3	Evaluation and Cost Conversion	102
4.6.3.4	Three-Stage Fidelity Schedule	104
4.6.3.5	Surrogate-Guided Candidate Selection	105
4.6.3.6	Warm-Start Selection	108
4.6.3.7	Parallel Execution and Timeout Control	109
4.6.3.8	Final Model and Ensemble Selection	110
4.6.4	Modular Decomposition	111
4.6.5	Design Constraints	112
4.6.6	Module Summary	113
4.7	Deployment Module	113
4.7.1	Module Responsibility	113
4.7.2	Service Architecture	115
4.7.3	Configuration and Artifact Preparation	116
4.7.4	Runtime Validation, Inference, and Tracking	117
4.7.5	Model Version Control	118
4.7.6	Deployment Orchestration	120
4.7.7	Command-Line Interface	122
4.7.8	Technical Value of the Module	124
4.8	Benchmark Platform Module	124
4.8.1	Functional Description	124
4.8.2	Modular Decomposition	125
4.8.3	Design Constraints	127
4.8.4	Module Summary	128
5	System Testing and Verification	129
5.1	Testing Setup	129
5.2	Testing Plan and Strategy	129
5.2.1	Module Testing	130
5.2.1.1	Auto Preprocessing Module	130
5.2.1.2	Accelera Pipe Module	132
5.2.1.3	Code Parallelizer Module	134
5.2.1.4	AutoML Module	135
5.2.1.5	Deployment Module	136
5.2.1.6	Accelera Pipeline Reporting Module	140
5.2.1.7	Benchmark Platform Module	140
5.2.2	Integration Testing	141
5.3	Testing Schedule	141
5.4	Comparative Results to Previous Work	142

5.4.1	Accelerera Pipe Module	142
5.4.2	Code Parallelizer Module	144
5.4.3	Auto Preprocessing	147
5.4.4	AutoML Module	149
6	Conclusions and Future Work	155
6.1	Faced Challenges	155
6.2	Gained Experience	155
6.3	Conclusions	155
6.4	Future Work	156
	Reference	156
	Appendix A Development Platforms and Tools	159
A.1	Software Platform	159
	Appendix B Use Cases	160
B.1	Auto Preprocessing Use Cases	160
B.2	AutoML Use Cases	160
B.3	Pipeline Reporting Use Cases	160
B.4	Benchmark Platform Use Cases	160
	Appendix C User Guide	161
C.1	Using Tabular Auto Preprocessing	161
C.2	Using Text Auto Preprocessing	162
C.3	Using Image Auto Preprocessing	163
C.4	Reading Auto Preprocessing Reports	164
C.5	Using Accelerera AutoML	164
C.6	Using Pipeline Reports	165
C.7	Using the Benchmark Platform	165
	Appendix D Feasibility Study	166
D.1	Purpose and Evaluation Criteria	166
D.2	Technical Feasibility	166
D.2.1	Architecture and Technology Availability	166
D.2.2	Integration and Portability	168
D.3	Operational Feasibility	168
D.4	Performance, Scalability, and Maintenance Feasibility	169
D.5	Overall Feasibility Decision	170

List of Figures

4.1	Overall block diagram of the Accelera project architecture.	52
4.2	Accelera Pipe workflow with custom preprocessing parallelization and pipeline reuse.	61
4.3	Before duplicate first-node sharing	68
4.4	After duplicate first-node sharing	68
4.5	Code Parallelizer module workflow	76
4.6	Simplified flow for parallelizing a Python method or transformer and using it inside an Accelera Pipe graph node.	83
4.7	Auto Preprocessing Workflow	86
4.8	Tabular Preprocessing Workflow	92
4.9	Text Preprocessing Workflow	94
4.10	Image Preprocessing Workflow	97
4.11	Block diagram of the Accelera AutoML module.	100
4.12	Simplified promotion flow in the three-stage fidelity schedule	105
4.13	Surrogate-guided candidate-selection workflow	107
4.14	Warm-start transfer from similar historical datasets	108
4.15	Process-level parallel execution and timeout handling	109
4.16	Deployment module overview from workspace artifacts to serving targets.114	
4.17	Model serving architecture and execution flow.	116
4.18	Deployment VCS registry structure.	119
4.19	VCS commit workflow for deployment artifacts.	119
4.20	VCS deploy and rollback workflow.	120
4.21	Multi-target deployment orchestration flow.	121
5.1	Mean classification accuracy for the evaluated AutoML systems.	150
5.2	Mean classification runtime for the evaluated AutoML systems.	150
5.3	Per-dataset classification accuracy on common datasets.	151
5.4	Per-dataset classification duration on common datasets.	151
5.5	Mean regression R^2 for the evaluated AutoML systems.	152
5.6	Mean regression runtime for the evaluated AutoML systems.	153
5.7	Per-dataset regression R^2 on common datasets.	153
5.8	Per-dataset regression duration on common datasets.	154

List of Tables

1.1	Problems addressed by Accelera.	19
1.2	Summary of project outcomes.	23
2.1	Targeted customers for Accelera.	25
2.2	Market comparison summary.	29
2.3	Possible business value of Accelera.	30
2.4	Expected cost categories.	31
2.5	Simplified business plan stages.	32
3.1	Selected AutoML systems and their primary design emphasis	44
3.2	Deployment-related platforms and Accelera's difference.	45
4.1	Possible paths to the deployment selection switch.	53
4.2	Principal node families in an Accelera Pipe graph.	54
4.3	Parallelizer input dispatch and output behavior.	55
4.4	Parallelizer stages.	57
4.5	Standalone use versus end-to-end use.	59
4.6	Main components of the Accelera Pipe module	62
4.7	Mapping from builder calls to graph node semantics	63
4.8	Concrete payload and output contracts of Accelera Pipe nodes	64
4.9	Supported operations in the Python-to-C++ converter	74
4.10	Main components of the Code Parallelizer module	75
4.11	Parallelizer input dispatch and returned output	78
4.12	Detected Column Types in Tabular Data	89
4.13	EDA graphs generated for tabular preprocessing	89
4.14	Example of conditional hyperparameter activation	102
4.15	Examples of metric-to-cost conversion	103
4.16	Default evaluation fidelity stages	104
4.17	Main quantities used in surrogate-guided candidate selection	107
4.18	AutoML implementation units	112
4.19	Deployment module implementation units.	115
4.20	Deployment orchestrator commands.	123
4.21	Deployment VCS commands.	123
5.1	Datasets Used in Module Testing	131
5.2	Models and Evaluation Metrics Used in Module Testing	132

5.3	Accelerera Pipe module-testing coverage.	133
5.4	Code Parallelizer module-testing coverage.	134
5.5	Deployment module verification objectives.	137
5.6	Verified deployment command flows.	139
5.7	Deployment Module Test Suite Verification Summary	140
5.8	Project-wide testing schedule.	141
5.9	Accelerera Pipe latency scaling in seconds	142
5.10	Accelerera Pipe RSS memory scaling in MB	143
5.11	Selected candidate and accuracy across sample sizes	143
5.12	Largest Accelerera Pipe comparison	144
5.13	OpenMP classifier report	144
5.14	Hard parallelizer evaluation	145
5.15	Linux parallelizer and Accelerera Pipe integration benchmark	145
5.16	Linux direct example-script verification runs	146
5.17	Windows direct example correctness and path timing	146
5.18	Windows compilation and process overhead	146
5.19	Classification comparison using Logistic Regression and Random Forest	148
5.20	Classification comparison using Decision Tree and KNN	148
5.21	Regression comparison using Linear Regression and Random Forest .	148
5.22	Regression comparison using Decision Tree and KNN	149
5.23	Classification benchmark summary.	149
5.24	Updated regression benchmark summary over ten datasets.	152
D.1	Module-level technical feasibility assessment.	167
D.2	Operational workflows and requirements.	168
D.3	Overall project feasibility decision.	170

List of Algorithms

4.1	Branch construction with conservative first-node sharing	65
4.2	Preprocess node execution and data ownership	66
4.3	High-level Accelerera Pipe training flow	67
4.4	Construction of the fitted executed graph	69
4.5	Consumer-count based intermediate release	69
4.6	Parallel graph execution scheduling	71
4.7	Loop classification and pragma insertion	77
4.8	Normalization of a Python callable for compilation	78
4.9	Classifier-free rule-based OpenMP generation	80
4.10	Cached compilation of an optimized Python callable	82
4.11	General tabular preprocessing flow	91
4.12	Text tokenizer flow	93
4.13	Image training preprocessing flow	96
4.14	Preparing deployment artifacts	117
4.15	Validating a request, running inference, and tracking the result	117
4.16	Creating a deployment VCS commit	119
4.17	Building and running the local deployment container	121
4.18	Deploying the model service to AWS EC2	122
4.19	Benchmark creation flow	127
4.20	Submission scoring flow	127
5.1	Deployment test-suite execution command	136
C.1	Calling tabular training preprocessing	161
C.2	Changing tabular preprocessing to regression	161
C.3	Calling tabular testing preprocessing	161
C.4	Calling text training preprocessing	162
C.5	Calling text testing preprocessing	162
C.6	Calling image training preprocessing	163
C.7	Calling image testing preprocessing	163
C.8	Basic AutoML classification workflow	164
C.9	Basic AutoML regression workflow	164
C.10	Calling the pipeline graph report	165

List of Abbreviations

AI	Artificial Intelligence
AMLB	AutoMLBenchmark
API	Application Programming Interface
AST	Abstract Syntax Tree
AutoML	Automated Machine Learning
AWS	Amazon Web Services
CI/CD	Continuous Integration and Continuous Deployment
CLI	Command-Line Interface
CPU	Central Processing Unit
CV	Cross-Validation
Docker	Containerization Platform
EC2	Elastic Compute Cloud
EI	Expected Improvement
Git	Distributed Version Control System
GUI	Graphical User Interface
Heroku	Cloud Application Platform
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IP	Internet Protocol
JSON	JavaScript Object Notation
ML	Machine Learning
OpenMP	Open Multi-Processing
REST	Representational State Transfer
RMSE	Root Mean Squared Error
RSS	Resident Set Size
SMBO	Sequential Model-Based Optimization
SSH	Secure Shell
SVC	Support Vector Classifier
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
VCS	Version Control System
VM	Virtual Machine
VMS	Virtual Memory Size

List of Symbols

X	Input feature matrix
y	Target vector
λ	Hyperparameter configuration
$\mathcal{A}$	Set of candidate algorithms
D	Dataset
f	Fidelity tuple
s	Fidelity stage index
q	Sample fraction
k	Cross-validation fold count
b	Model budget multiplier
μ	Surrogate predicted mean cost
σ	Surrogate predicted cost standard deviation
P	Source program
L	Extracted loop
$F(L)$	Feature representation of loop L

Contacts

Team Members

Name	Email	Phone Number
Mohamed Ashraf	mohamed.abdelaziz03@eng-st.cu.edu.eg	+2 01004538215
Nesma Osama	nesma.ahmed03@eng-st.cu.edu.eg	+2 01067193062
Mazen Adel	mez3n1151@gmail.com	+2 01152794877
Mohamed Khater	mohamed.khater03@eng-st.cu.edu.eg	+2 01068071299

Supervisor

Name	Email	Number
Salma Abdelmoniem	salmaabdelmotaleb@eng.cu.edu.eg	+2 01114302362

Chapter 1: Introduction

Accelerera is a hybrid Python/C++ machine-learning framework designed to support the complete data-science workflow, starting from data preparation and model selection, and extending to graph-based execution, native code acceleration, deployment, and benchmark comparison. The project is not a single isolated algorithm. Instead, it is a collection of connected modules that cooperate to make machine-learning experiments easier to build, easier to repeat, and faster to execute when possible.

The main idea behind Accelerera is simple: Python gives developers a productive environment for experimentation, but real machine-learning workflows often need more structure and better execution control. A typical workflow may include data cleaning, preprocessing, feature engineering, model selection, prediction, metric evaluation, reporting, deployment, and comparison between several alternatives. When these steps are written as separate scripts, repeated work, unclear execution order, and performance bottlenecks become common problems.

Accelerera addresses this by combining six main units: `auto_preprocessing`, `accelera_automl`, `accelera_pipe`, `parallelizer`, the benchmark platform, and the Deployment module.

1.1 Motivation and Justification

Machine-learning development is often presented as a simple process of choosing a model and calling `fit`. In practice, the model is only one stage in a larger workflow. Before training, the data must be cleaned, split, transformed, encoded, scaled, and sometimes reduced. After training, the output must be evaluated, compared, reported, possibly deployed, or submitted to a benchmark system. This creates several practical problems.

- **Repeated preprocessing code:** Similar cleaning, encoding, scaling, and feature-selection steps are often rewritten for every dataset.
- **Manual model comparison:** Users may need to try many models and configurations before finding a good solution.
- **Linear workflow limitations:** Traditional scripts or simple pipelines may duplicate work when several branches share early operations.
- **Python loop bottlenecks:** Custom preprocessing functions written in Python may become slow when they contain large numeric loops.
- **Weak experiment visibility:** Without reports, graphs, and saved artifacts, it becomes harder to understand what preprocessing and evaluation steps were

applied.

- **Deployment gap:** After training, models may still need extra work before they can be used as services or prediction endpoints.
- **Lack of fair comparison:** Pipeline results need a structured place where metrics, submissions, and benchmarks can be compared.
- **Deployment gap:** A trained model is not useful to external users until it is packaged, served through an API, configured for its runtime environment, and exposed on a local or remote host.

These problems justify the need for Accelera. The project provides automation where the workflow is repetitive, graph execution where the workflow contains branches, native acceleration where supported custom code is slow, deployment support where trained models need to be used in practice, and benchmark components where results must be compared. The motivation is not only speed; it is also organization, reproducibility, usability, and practical use.

The `auto_preprocessing` module is motivated by the fact that data preparation usually consumes a large part of the development effort. It handles common preprocessing decisions for tabular datasets, such as dropping duplicate records, detecting column types, imputing missing values, encoding categorical features, applying robust scaling, selecting features, saving preprocessors, and generating reports. It also supports text preprocessing using cleaning, tokenization, stemming, TF-IDF vectorization, and target encoding. For image workflows, it provides preparation paths for classification tasks. This makes the project useful before the model-selection stage even begins.

The `accelera_automl` module is motivated by the difficulty of manual model selection. A user may not know which estimator, preprocessing choice, or hyperparameter setting will perform best. AutoML reduces this burden by evaluating candidate configurations under controlled budgets. This makes the search process more systematic and provides useful outputs such as the best model, score, leaderboard, and run history.

The `accelera_pipe` module is motivated by the need to represent machine-learning workflows as graphs rather than only as linear chains. In branch-heavy experiments, several alternatives may share the same preprocessing steps. A graph representation makes these dependencies explicit. It also allows the system to reason about execution order, branch selection, reuse of fitted paths, and intermediate-data lifetime. This gives the user a high-level Python interface while the backend manages a more structured execution model.

The `parallelizer` module is motivated by the performance gap between Python and native compiled code. Python is easy to write, but loop-heavy Python functions can be slow. The parallelizer attempts to optimize supported custom functions by converting them to C++, extracting loops, computing loop features, classifying safe parallelization

opportunities, applying rule-based safety checks, inserting OpenMP pragmas, and compiling the result as a native Python-callable extension. If the optimization path fails, the original function is preserved, which keeps the workflow correct.

The benchmark platform is motivated by the need for organized comparison. A machine-learning framework becomes more useful when users can compare pipelines on common datasets and metrics. The benchmark module therefore represents a future-facing part of the project, where submissions, metrics, and leaderboard-style results can help users evaluate their solutions fairly.

Finally, the Deployment module is motivated by the final step that many machine-learning projects postpone: making the trained model available outside the notebook or local script. Deployment normally requires packaging artifacts, building an API layer, preparing dependencies, exposing ports, configuring servers, and checking that the service responds correctly. Accelera includes this step so that the workflow can continue from model creation to a reachable prediction service.

Table 1.1: Problems addressed by Accelera.

Problem	Accelera component	Expected benefit
Manual and repetitive data preparation	auto_preprocessing	Automates common preprocessing and produces saved artifacts and reports.
Time-consuming model selection	accelera_automl	Searches candidate models and configurations under controlled budgets.
Repeated work in branch-heavy workflows	accelera_pipe	Represents the workflow as a graph and exposes shared execution structure.
Slow custom Python loops	parallelizer	Converts supported functions to native code and inserts safe OpenMP pragmas.
Model deployment after training	Deployment support	Helps prepare trained models or pipelines for service-based inference.
Difficult result comparison	Benchmark platform	Stores benchmark information, metrics, submissions, and leaderboard views.
Local-only trained models	Deployment module	Packages saved models into validated API services that can run in Docker, Heroku, or AWS EC2 workflows.

1.2 Project Objectives and Problem Definition

The main objective of Accelera is to provide an integrated framework for constructing, preprocessing, executing, evaluating, optimizing, and comparing machine-learning workflows. The project aims to keep the familiar Python programming style while adding automation, graph execution, native acceleration, and benchmark-oriented evaluation.

The main objectives of the project are:

1. Build an automated preprocessing module for tabular, text, and image datasets.
2. Implement AutoML classifiers and regressors that search across supported model configurations using scoring metrics, cross-validation, time budgets, trial budgets, and optional ensemble construction.
3. Provide a graph-based pipeline module that represents workflow steps as executable graph nodes rather than only as a sequential list.
4. Support pipeline operations such as preprocessing, model fitting, prediction, metric evaluation, branching, merging, and executed-graph reuse.
5. Integrate custom preprocessing functions with the parallelizer so that supported loop-heavy functions can be optimized automatically.
6. Implement a conservative code parallelization path that uses C++, OpenMP, loop-feature extraction, classification, rule-based safety checks, and native compilation.
7. Provide reporting tools that make preprocessing outputs, model metrics, and pipeline structure more visible to the user.
8. Build a benchmark platform prototype for datasets, submissions, users, metrics, and leaderboard-based comparison.
9. Provide a Deployment module that packages trained models and generated artifacts into usable services, supports local and remote deployment workflows, manages deployment configuration, and reduces the infrastructure knowledge required to expose a trained model through an API.

The problem addressed by the project can be stated as follows:

Machine-learning workflows in Python are easy to write but become difficult to organize, repeat, compare, and optimize when they include many preprocessing steps, model alternatives, custom functions, evaluation paths, and deployment requirements. Accelera aims to preserve Python usability while adding automated preprocessing, budgeted model search,

graph-based execution, safe loop-level acceleration, and benchmark-oriented evaluation, while also supporting the final step of turning trained artifacts into accessible services.

Workflow complexity increases when preprocessing steps, branches, and candidate models grow, while execution performance may suffer from repeated work and slow Python loops. Accelera addresses both issues by combining high-level workflow automation with lower-level execution optimization.

1.3 Project Outcomes

The project outcomes are the implemented modules and the integration between them. Each module provides a different part of the final system.

Outcome 1: Automated preprocessing

The `auto_preprocessing` module prepares datasets for machine-learning models. For tabular data, it handles duplicates, missing values, column-type detection, categorical encoding, robust scaling, target processing, feature selection, and report generation. For text data, it applies cleaning, tokenization, stemming, TF-IDF vectorization, and label encoding. For image data, it supports classification preparation. The outcome is a set of processed arrays, saved preprocessors, selected features, and reports.

Outcome 2: AutoML model search

The `accelera_automl` module provides estimator-style AutoML classes for classification and regression. It searches candidate configurations, evaluates them with cross-validation, supports warm-starting from dataset metafeatures, and can build voting or stacked ensembles. The outcome is a fitted final model, best score, leaderboard, and run history.

Outcome 3: Graph-based pipeline execution

The `accelera_pipe` module provides a graph-based pipeline interface. It represents preprocessing, model, prediction, metric, branch, and merge operations as graph nodes. The output of a pipeline run includes predictions and an `ExecutedGraph`, which stores the selected fitted path for later use. This outcome gives the project its workflow-execution layer.

Outcome 4: Source-level parallelization

The parallelizer module accelerates supported loop-heavy code. It can process C/C++ code, supported Python code, custom Python functions, and transformer methods. It extracts loops, computes features, classifies parallelization opportunities, applies

safety rules, inserts OpenMP pragmas, and compiles native Python-callable extensions when possible. If any step fails, it falls back to the original Python implementation.

Outcome 5: Pipeline and parallelizer integration

One of the most important outcomes is the connection between `accelera_pipe` and the parallelizer. When a preprocessing node contains a supported custom Python function, the pipeline attempts to optimize it before adding it to the graph. This allows the system to combine graph-level workflow execution with code-level acceleration.

Outcome 6: Reporting and benchmark support

The project includes reports for preprocessing, model metrics, and graph visualization. It also includes a benchmark platform prototype with backend and frontend components for datasets, metrics, users, submissions, and leaderboards. These outcomes improve visibility and comparison.

Outcome 7: Deployment support

The Deployment module extends Accelera beyond experimentation by converting saved machine-learning artifacts into runnable services. It organizes `config.json` files, model pickle files, serving code, schema validation, telemetry logging, Docker packaging, and target-specific commands for local containers, Heroku, and AWS EC2. This outcome is important because it allows Accelera to support the complete journey from building a model to making the model reachable through an endpoint.

Table 1.2: Summary of project outcomes.

Outcome	Description
Automated preprocessing	The project provides reusable preprocessing support for tabular, text, and image data, with generated reports and saved preprocessing artifacts.
Automated model selection	The AutoML module helps search and compare candidate classification and regression models under time and trial constraints.
Graph-based pipeline execution	Accelera Pipe represents machine-learning workflows as executable graphs, supporting preprocessing, modelling, prediction, metrics, branching, and merging.
Code-level acceleration	The Parallelizer attempts to optimize supported loop-heavy Python/C++ code using static analysis, safety checks, OpenMP pragmas, compilation, and caching.
Benchmarking support	The Benchmark Platform provides a foundation for dataset-based evaluation, prediction submission, metric calculation, and leaderboard-style comparison.
Deployment support	The Deployment module packages trained models and supporting artifacts into validated API services that can run locally in Docker or be deployed to supported cloud/server targets.

1.4 Document Organization

The rest of this report is organized into seven main parts: Chapter 2 presents the market visibility study, Chapter 3 discusses the literature survey and related work, Chapter 4 explains the methodology and system design of the main Accelera modules, Chapter 5 presents testing, verification, and experimental results, and Chapter 6 concludes the report by summarizing the contributions, limitations, and possible future improvements. The appendices include supporting material such as documentation, usage instructions, feasibility details, and additional implementation notes.

Chapter 2: Market Visibility Study

This chapter studies the market need for Accelera and compares it with existing tools that solve related problems. The goal is to show who can benefit from the project, what alternatives already exist, and why Accelera provides a different value. The chapter also presents a simplified business case and financial analysis to clarify how the project could be developed into a product or service.

Accelera targets the data-science workflow as a complete process, not as one isolated task. Some existing tools focus on preprocessing, others focus on model selection, others focus on competition platforms or code acceleration, and others focus only on hosting or cloud deployment. Accelera also can be used as a compination of five connected parts (end-2-end) E2E module: `auto_preprocessing`, `accelera_automl`, `accelera_pipe`, parallelizer, the benchmark platform, and the Deployment module.

2.1 Targeted Customers

The targeted customers for Accelera are users and organizations that build, test, compare, and optimize machine-learning workflows. These customers can be divided into several groups, as shown in Table 2.1.

The most important customer segment is the data-science and machine-learning user who wants to move from a raw dataset to an evaluated model with less manual work. For beginners, Accelera provides guidance and automation. For experienced users, it provides a more structured execution model and opportunities for acceleration.

Accelera can also be useful in educational environments. In many machine learning courses, students repeat the same steps: clean the data, encode features, scale numerical values, split the dataset, train models, calculate metrics, and compare results. Accelera can make these steps easier to understand because it provides reports and separates the workflow into clear modules.

For professional users, the value is different. They care more about reproducibility, execution time, reusable pipeline structure, and fair comparison. The `accelera_pipe` and benchmark modules are especially useful here because they organize workflow execution and comparison. The `parallelizer` adds value when the workflow includes supported custom preprocessing functions that are slow in Python.

Customer Value Proposition

The main value proposition of Accelera is that it reduces the gap between experimentation and structured execution. Instead of forcing the user to choose between simple scripts

Table 2.1: Targeted customers for Accelerera.

Customer group	Main need	How Accelerera helps
Students and beginners	Need an easier way to build complete machine-learning workflows without writing every step manually.	Provides automated preprocessing, model search, reports, and a pipeline interface that organizes the workflow.
Data scientists	Need to prepare data, compare models, and repeat experiments quickly.	Reduces repetitive preprocessing, supports AutoML search, and gives reports and leaderboards for comparison.
Machine-learning engineers	Need structured, reusable, and optimized workflows that can be inspected and executed reliably.	Provides graph-based pipeline execution, executed-graph reuse, and integration with custom preprocessing acceleration.
Researchers	Need to compare approaches, measure results, and experiment with workflow and code optimization.	Provides modular components for benchmarking, graph execution, OpenMP parallelization, and performance evaluation.
Small teams and startups	Need a practical toolkit that reduces development time without building an internal ML platform from scratch.	Combines preprocessing, AutoML, pipeline execution, reports, and benchmark support in one project, with deployment support for turning selected models into accessible services.

and large industrial platforms, Accelerera provides a research-friendly framework with practical automation and optimization features.

- **Less manual work:** preprocessing and model search are partially automated.
- **Better organization:** workflows can be represented as graph pipelines instead of scattered scripts.
- **Better visibility:** reports and benchmark views help users inspect results.
- **Possible speedup:** supported loop-heavy preprocessing code can be accelerated through C++ and OpenMP.
- **Fairer comparison:** benchmark components help compare pipelines using common datasets and metrics.

- **Practical delivery:** deployment commands help move saved models from local artifacts to runnable API services.

2.2 Market Survey

Many existing tools provide features related to Accelera, but most of them focus on only one part of the workflow. Some tools focus on preprocessing, some on AutoML, some on competitions, and some on code acceleration. Accelera is different because it tries to connect these ideas into one toolkit.

2.2.1 AutoCleanML

AutoCleanML is a Python tool focused on automating data cleaning and preprocessing tasks for machine-learning datasets. It is useful because data preparation is one of the most repetitive and time-consuming parts of the machine-learning workflow. The tool helps automate common data-cleaning and preprocessing decisions, which can make dataset preparation easier, especially for beginners. It also provides reporting features that explain some of the preprocessing decisions applied during the workflow.

However, AutoCleanML mainly focuses on preprocessing and data cleaning rather than the complete machine-learning workflow. It does not provide a graph-based pipeline execution engine, does not target custom code acceleration or OpenMP parallelization, and does not provide a benchmark platform for comparing full pipelines.

Compared with AutoCleanML, Accelera includes preprocessing but does not stop there. It also includes AutoML search, graph pipeline execution, source-level parallelization, reports, and benchmark support. Therefore, Accelera aims to be closer to a complete workflow framework.

2.2.2 scikit-learn

scikit-learn is one of the strongest tools in the machine-learning market. It provides a wide range of models, preprocessing tools, metrics, and a simple Pipeline abstraction. For this reason, it is the most important competitor for `accelera_pipe`.

Pros:

- Mature and widely used.
- Provides many models, preprocessing utilities, and metrics.
- Has a simple and reliable pipeline interface.
- Integrates well with the Python data-science ecosystem.

Cons:

- The standard pipeline is mainly linear.

- Branch-heavy workflows may require repeated pipelines or external search utilities.
- It does not focus on native graph scheduling for pipeline branches.
- It does not automatically convert supported custom preprocessing functions to C++ and OpenMP.

Accelera uses `scikit-learn` as a strong baseline rather than ignoring it. The difference is that `accelera_pipe` represents the workflow as a graph and integrates with the Parallelizer for supported custom preprocessing acceleration. For this reason, `scikit-learn` is the most important competitor for `accelera_pipe`.

2.2.3 AutoML Tools

AutoML tools such as Auto-sklearn, TPOT, and AutoGluon focus on automating model selection and hyperparameter search. They are useful because they reduce the manual effort needed to test many models and configurations. These tools can often provide strong baseline performance, and some of them also support ensemble construction.

However, most AutoML tools focus mainly on model search rather than complete workflow execution. They do not mainly target graph-based pipeline execution, and they do not automatically accelerate custom preprocessing loops using OpenMP. Some AutoML systems can also be heavy or complex for beginners.

Accelera includes `accelera_automl`, but also connects AutoML with preprocessing, graph execution, reports, and parallelization. This makes the AutoML module one part of a larger framework rather than the only feature.

2.2.4 Code Acceleration Tools

Tools such as Numba, Cython, and manual C++ extensions can speed up Python code. They are relevant because Accelera also tries to accelerate supported custom preprocessing functions. These tools can significantly improve performance for numeric and loop-heavy workloads, and they provide more control for experienced developers who understand optimization and compilation.

However, these tools often require users to understand extra syntax, compilation steps, or type constraints. They also do not automatically decide where acceleration fits inside a machine-learning pipeline. In addition, they do not provide pipeline graphs, AutoML, reports, or benchmark features.

The Accelera Parallelizer is different because it is designed to work with the pipeline module. It attempts to optimize supported custom preprocessing functions and attach the optimized callable back to the pipeline node. This makes code acceleration part of the workflow instead of a separate manual task. “

2.2.5 Deployment and Hosting Platforms

Deployment platforms such as Heroku, AWS EC2, Render, Railway, and Google Cloud Run are relevant to Accelera because they help developers publish software services. Their main value is infrastructure: they provide servers, routing, containers, or managed execution environments. This is important, but it does not by itself solve the machine-learning-specific work of collecting model artifacts, preserving deployment configuration, validating input schemas, and connecting a trained model to an inference API.

Pros:

- Provide public or cloud-hosted environments for running applications.
- Reduce the need to own physical servers.
- Support common deployment units such as containers, web services, or virtual machines.

Cons:

- Usually require the user to prepare the ML serving code and runtime dependencies separately.
- Do not automatically understand Accelera model artifacts, `config.json`, schema validation, or prediction telemetry.
- General-purpose platforms may be difficult for beginners because they involve CLI tools, SSH, ports, environment variables, and deployment logs.

Accelera's Deployment module is different because it is not only a hosting target. It is part of the ML workflow itself. It receives the outputs produced by training or pipeline modules, copies model binaries, writes deployment requirements, builds a Docker service, exposes prediction endpoints, validates requests, records telemetry, and provides commands for local Docker, Heroku, and AWS EC2. This integration increases Accelera's market value because it makes the system closer to an end-to-end ML platform rather than only a model training toolkit.

Table 2.2: Market comparison summary.

Tool or platform	Main focus	Limitation	Accelera difference
scikit-learn	Machine-learning models, preprocessing tools, and linear pipelines.	Pipeline execution is mainly linear and does not focus on graph-based branch execution or custom code parallelization.	Adds graph-based pipeline execution and integration with the Parallelizer.
AutoCleanML	Automatic data cleaning and preprocessing.	Focused mainly on data cleaning and does not cover the whole workflow.	Adds AutoML, graph pipelines, parallelization, reports, and benchmarks.
Auto-sklearn / TPOT / AutoGluon	Automated model selection and machine-learning search.	Focus mainly on model search, not graph execution or source-level code parallelization.	Combines AutoML with preprocessing, graph pipelines, and the Parallelizer.
Numba / Cython	Python performance acceleration.	Require user knowledge of acceleration syntax and do not manage ML workflow structure.	Provides an automatic attempt to optimize supported preprocessing functions inside the pipeline.
Heroku	Cloud application hosting and container deployment.	Provides hosting infrastructure but does not manage ML artifacts, schema validation, or model version snapshots as part of the training workflow.	Connects model packaging, serving code, validation, telemetry, and Heroku container commands inside one ML-oriented deployment module.
AWS EC2	General-purpose virtual machines for cloud hosting.	Powerful but requires manual server access, SSH, Docker setup, port exposure, and operational knowledge.	Automates EC2-oriented file synchronization, remote Docker build/run commands, and health checks for Accelera model services.

2.3 Business Case and Financial Analysis

Accelera can be developed as an open-source framework with optional hosted services around benchmarking, reporting, team collaboration, managed execution, and deployment assistance. The open-source part helps adoption because students, researchers, and developers can use the framework freely. The business value can come from hosted benchmark features, team dashboards, managed experiment tracking, deployment templates, and support services.

2.3.1 Business Case

The main business case is based on reducing time spent in repetitive data-science work. Users and teams spend time preparing data, comparing models, rewriting scripts, debugging pipeline steps, and measuring results. Accelera can reduce this time by providing one framework that handles multiple stages of the workflow. The Deployment module strengthens this business case because organizations do not only need trained models; they need trained models to become usable services.

Automating deployment-related steps can reduce manual DevOps effort, reduce time-to-market, and lower the technical barrier for users who understand ML but are less experienced with Docker, SSH, cloud machines, or service exposure.

Table 2.3: Possible business value of Accelera.

Business value	Reason	Possible customer
Faster experimentation	Automated preprocessing and AutoML reduce repeated manual trials.	Students, data scientists, small teams.
Better organization	Graph pipelines and executed graphs make workflows easier to inspect and reuse.	ML engineers and research teams.
Performance improvement	Parallelizer can speed up supported loop-heavy preprocessing code.	Teams with custom preprocessing bottlenecks.
Better comparison	Benchmark platform supports submissions, metrics, and leaderboards.	Universities, communities, and organizations.
Shorter deployment path	Deployment support helps package model artifacts, build a serving API, and run the service on supported targets.	Small teams, prototypes, and applied ML projects.
Lower entry barrier	Reports and automation make ML workflows easier for beginners.	Education and training programs.

The first possible product direction is a hosted benchmark platform. Users can submit pipelines, compare results on public datasets, and view leaderboards. A free tier can support public datasets and basic submissions, while paid tiers can support private benchmarks, team spaces, larger storage, and advanced reports.

The second possible product direction is a managed workflow dashboard. This dashboard can visualize preprocessing results, model leaderboards, pipeline graphs, and runtime comparisons. Teams can use it to organize experiments and share results without relying only on notebooks.

The third possible product direction is professional support and integration. Organizations that want to use Accelera internally may need help adapting the framework to their datasets, infrastructure, or benchmark requirements. This creates a possible support and consulting model.

The Deployment module also creates a practical commercial path for a managed deployment tier. A hosted product could provide prepared deployment templates, private model-serving environments, monitored endpoints, and simplified rollback

support. Even when the current implementation remains lightweight and developer-controlled, its presence makes Accelera easier to present as a complete platform because it connects experimentation with operational use.

2.3.2 Financial Analysis

The financial analysis can be divided into expected costs and possible revenue streams. Since Accelera is mainly a software project, the capital expenditure is limited compared with hardware-based products. Most costs come from development, hosting, cloud resources, storage, and maintenance.

Table 2.4: Expected cost categories.

Cost category	Description	Notes
Development cost	Time spent implementing, testing, documenting, and maintaining the framework.	Main cost during early stages.
Cloud hosting	Servers for benchmark backend, database, frontend, and possible job execution.	Can start small and scale with users.
Storage	Datasets, submissions, reports, and model artifacts.	Increases with benchmark usage.
Compute resources	Running benchmark submissions or managed experiments.	Can be limited by quotas or paid tiers.
Maintenance and support	Bug fixes, user support, documentation, and security updates.	Needed for long-term usage.

Possible revenue streams include:

- **Free open-source adoption:** increases user base and community trust.
- **Hosted benchmark subscriptions:** paid private benchmarks, team workspaces, and larger storage limits.
- **Managed experiment dashboards:** paid reporting and workflow monitoring for teams.
- **Enterprise support:** integration, customization, and technical support for organizations.
- **Education packages:** university or training-center packages for teaching machine-learning workflows.

A simple financial plan can start with low-cost hosting and manual support. In the first stage, the focus should be adoption and validation. The open-source framework can attract users, while the benchmark platform can collect early feedback. In the second stage, private benchmark spaces and team dashboards can be introduced. In the third stage, enterprise support and managed compute can be added if the user base grows.

Table 2.5: Simplified business plan stages.

Stage	Focus	Expected result
Stage 1	Release open-source framework, documentation, examples, and local benchmark prototype.	Build trust, collect users, and validate the technical direction.
Stage 2	Launch hosted benchmark features, public leaderboards, and private team workspaces.	Create first revenue stream through hosted services.
Stage 3	Add managed experiment dashboards, larger storage, and paid support.	Serve teams and organizations with more advanced workflow needs.
Stage 4	Provide enterprise integration and managed compute for benchmark execution.	Scale the platform into a commercial ML workflow service.

The expected break-even point depends on hosting cost, development effort, and the number of paying users. Since the software can begin as open source, the initial financial risk is lower than products that require manufacturing or special hardware. The main challenge is user adoption. If Accelera can attract students, researchers, and small teams through the free framework, the benchmark and reporting services can become the commercial layer that supports future development.

Chapter 3: Literature Survey

In this chapter, we give an overview of the topics, algorithms, and techniques needed to understand the project as a whole. We discuss the technical subjects and background knowledge, and then summarize the research papers related to our project.

3.1 Background and Previous Work

This section introduces the main background needed to understand the Accelera project as a whole. It explains the important concepts, techniques, and tools that were used in different parts of the system. These background topics help clarify how the project supports data preparation, pipeline execution, report generation, model evaluation, and benchmarking. The goal of this section is to give the reader enough context before moving to the methodology and implementation chapters.

3.1.1 Data Science and Machine Learning

Data science is the field that uses data, statistics, programming, and domain knowledge to understand a problem and extract useful insights from it. It is usually used when we have raw data and we want to turn it into useful information or predictions.

Machine learning is one of the main parts of data science. Instead of writing fixed rules for every case, we train a model on examples, and the model learns patterns from these examples. After training, the model can use what it learned to make predictions on new data.

In supervised machine learning, the training data contains input features and a target label. The model learns the relationship between the features and the target. If the target is a class, such as spam or not spam, the problem is called classification. If the target is a continuous number, such as price or temperature, the problem is called regression.

3.1.1.1 Data Exploration and Preprocessing [1]

Exploratory Data Analysis, or EDA, is the step where we inspect the dataset before building the model. We summarize columns, draw visualizations, check missing values, and try to understand relationships between variables. This step helps us notice problems early instead of discovering them after training the model.

Data preprocessing is the stage of transforming raw data into a clean and usable format for machine learning models. In real datasets, the data may contain missing values, duplicated rows, different feature scales, and categorical columns that cannot be used directly by many models. Because of this, preprocessing is usually considered an essential part of a machine learning workflow.

Missing value imputation is used when some values in a dataset are empty. Instead of removing all rows that contain missing values, the missing values can be replaced using a suitable value, such as the mean, median, mode, or another estimated value. Robust scaling is a scaling technique that uses the median and the interquartile range instead of the mean and standard deviation. It is useful when the data contains outliers, because the median and interquartile range are less affected by extreme values:

$$x' = \frac{x - \text{median}(X)}{\text{IQR}(X)}$$

where x' is the scaled value, x is the original value, $\text{median}(X)$ is the median of the feature values, and $\text{IQR}(X)$ is the interquartile range.

Categorical encoding is the process of converting text categories into numeric values. Many traditional machine learning models and preprocessing pipelines expect numerical input. Two examples are binary encoding and frequency encoding. Binary encoding first gives each category a number, then converts this number into binary form and stores the binary digits in separate columns. Frequency encoding replaces each category with how often it appears in the dataset:

$$f(c) = \frac{\text{count}(c)}{N}$$

where $f(c)$ is the frequency of category c , $\text{count}(c)$ is the number of rows containing this category, and N is the total number of rows.

3.1.1.2 Model Evaluation Metrics [2]

After training a model, we need metrics to check whether the model is performing well or not. The selected metric depends on the type of problem. Classification models are evaluated using metrics that compare predicted classes with actual classes, while regression models are evaluated using metrics that measure the difference between predicted numerical values and real values.

Classification metrics: Classification metrics are used when the output is a class or category, such as spam or not spam, positive or negative, or one image class from many classes. The confusion matrix is the base idea behind many classification metrics. It compares the actual classes with the predicted classes. For binary classification, it contains four values: true positives, true negatives, false positives, and false negatives.

- **True Positive (TP):** The model predicts positive, and the actual class is positive.
- **True Negative (TN):** The model predicts negative, and the actual class is negative.
- **False Positive (FP):** The model predicts positive, but the actual class is negative.

- **False Negative (FN):** The model predicts negative, but the actual class is positive.

Accuracy measures how many predictions are correct out of all predictions:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

where TP is true positives, TN is true negatives, FP is false positives, and FN is false negatives. Accuracy is simple and useful when the classes are balanced, but it may be misleading when one class is much larger than the other.

Precision measures how many predicted positive samples are actually positive:

$$Precision = \frac{TP}{TP + FP}$$

Precision is useful when false positives should be reduced.

Recall measures how many actual positive samples were correctly detected:

$$Recall = \frac{TP}{TP + FN}$$

Recall is useful when false negatives should be reduced.

The F1-score combines precision and recall:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

It is useful when both precision and recall are important, especially with imbalanced datasets.

Regression metrics: Regression metrics are used when the output is a numerical value, such as price, score, or temperature.

Mean Absolute Error measures the average absolute difference between the real values and predicted values:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

where y_i is the actual value, $\hat{y}_i$ is the predicted value, and n is the number of samples. MAE is easy to understand because it gives the average error in the same unit as the target value.

Mean Squared Error gives more penalty to large errors:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

This means large mistakes affect the score more than small mistakes.

Root Mean Squared Error is the square root of MSE:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

RMSE is easier to read than MSE because it returns the error to the same unit as the target value.

R-squared score measures how well the regression model explains the variation in the actual values:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

where y_i is the actual value, $\hat{y}_i$ is the predicted value, and $\bar{y}$ is the mean of the actual values. A value close to 1 means the model explains the data well, while a value close to 0 means the model does not explain the data well.

3.1.2 Natural Language Processing (NLP) [3]

Natural Language Processing is the field that helps computers understand and work with human language. It is used in many applications such as text classification, sentiment analysis, spam detection, and chatbots. Text data cannot be used directly by most machine learning models, so it needs to be cleaned and converted into numbers first.

In text preprocessing, the text may be converted to lowercase, cleaned from unnecessary symbols, split into words, and simplified using stemming. After that, methods such as TF-IDF are used to transform the text into numerical features.

3.1.2.1 Term Frequency-Inverse Document Frequency (TF-IDF)

Term Frequency-Inverse Document Frequency is one common technique used to convert text into numerical features. It gives each word a weight based on how important it is in one document and across the full dataset. Term Frequency measures how often a word appears in a document. Inverse Document Frequency gives lower weight to words that appear in many documents, because these words are usually less useful for classification:

$$TFIDF(t, d) = TF(t, d) \times IDF(t)$$

where $TFIDF(t, d)$ is the weight of term t in document d , $TF(t, d)$ is the frequency of the term in the document, and $IDF(t)$ measures how rare the term is across all documents.

3.1.3 Data Augmentation

Data augmentation is a technique used to increase the diversity of a dataset without actually collecting new data. It works by applying transformations to existing samples to create new modified versions. For example, in image classification, augmentation may include flipping, rotating, cropping, or changing brightness. This can help the model generalize better because it sees more variations during training.

3.1.4 Machine Learning Pipelines

A machine learning pipeline is a sequence of steps used to prepare data, train a model, make predictions, and evaluate results. Pipelines are useful because they organize the full workflow in one structure instead of writing each step separately.

A typical pipeline may include preprocessing steps such as scaling, encoding, or feature selection, followed by a machine learning model. This makes experiments easier to repeat and reduces mistakes, because the same preprocessing steps are applied during training and testing.

One of the most common tools for building machine learning pipelines is `scikit-learn`. It provides a `Pipeline` object that connects transformers and estimators together. In this project, `scikit-learn` is considered the main competitor for the `accelera_pipe` module because both systems aim to help users build machine learning workflows in Python.

3.1.5 Graph-Based Pipeline Execution

Traditional machine learning pipelines usually execute steps in a linear order. This is suitable for simple workflows, but it can become inefficient when several branches share the same preprocessing steps or when multiple models are compared.

Graph-based execution represents the workflow as nodes and edges. Each node represents an operation, such as preprocessing, model training, prediction, or metric evaluation. Each edge represents the dependency between operations. This representation can help avoid repeated work, because shared steps can be executed once and reused by different branches.

The `accelera_pipe` module follows this idea by representing the pipeline internally as an execution graph. This allows the system to support branching, merging, parallel execution of independent paths, and better management of intermediate data.

3.1.6 Code Parallelization

Code parallelization is the process of running parts of a program at the same time to reduce execution time. In many programs, the most expensive part is a loop that repeats the same operation over a large amount of data. If the loop iterations are independent, they may be executed in parallel.

Manual parallelization is difficult because the developer must check that there are no unsafe dependencies between iterations. If a loop writes to shared variables or depends on results from previous iterations, parallel execution may produce wrong results. Because of this, automatic parallelization tools try to analyze code and decide whether a loop can be safely parallelized.

3.1.7 OpenMP

OpenMP is a programming interface used to add parallel execution to C and C++ programs. It allows developers to use compiler directives, called pragmas, to tell the compiler that some parts of the code can run in parallel.

For example, the directive `#pragma omp parallel for` can be used before a loop to distribute loop iterations across multiple threads. OpenMP is useful because it does not require rewriting the whole program. However, the pragma must be inserted only when the loop is safe to parallelize.

3.1.8 Deployment

Deployment is the process of moving software from a development environment to an environment where users or other systems can access it. In machine-learning projects, deployment is important because the trained model must be connected to software that receives input, runs inference, and returns predictions.

3.1.9 Model Deployment

Model deployment means packaging a trained model together with the code and dependencies needed to use it. A deployed model is usually exposed through a service endpoint so that other applications can send data and receive predictions. This is different from only saving a model file, because a saved file still requires a runtime service before it becomes usable by external users.

3.1.10 Machine Learning Deployment

Machine-learning deployment adds extra concerns beyond ordinary software deployment. The system must preserve preprocessing steps, load model binaries, validate input schemas, handle prediction errors, and sometimes track requests for monitoring. These concerns explain why Accelerera treats deployment as a module connected to the ML workflow rather than only as a cloud-hosting task.

3.1.11 Cloud and Server Deployment

Cloud deployment runs a service on infrastructure provided by a cloud provider. Server deployment runs the service on a machine that may be local, virtual, or cloud-hosted. In both cases, the deployed service must receive network traffic on a configured port and must be started with the correct dependencies and environment variables.

3.1.12 Web Services, APIs, and REST APIs

A web service is a program that exposes functionality over the network. An API defines how other programs interact with that service. A REST API commonly uses HTTP methods and URLs to exchange structured data. In the Deployment module, prediction requests are sent to API endpoints such as `/predict`, while status information is exposed through endpoints such as `/health`.

3.1.13 HTTP, HTTPS, JSON, URLs, and Ports

HTTP is the common protocol used by web clients and servers. HTTPS is the secure version of HTTP. JSON is a text format frequently used to send structured data between clients and APIs. A URL identifies where a service can be reached, and a port identifies the network entry point used by the service on a machine. These concepts are needed to understand deployed prediction endpoints.

3.1.14 Docker and Containerization

Docker is a containerization tool that packages an application with its runtime environment. A container helps the same service run consistently across machines because dependencies, files, and startup commands are described explicitly. The Deployment module uses Docker to package the FastAPI serving application, model files, configuration, and required Python libraries into one runnable service image.

3.1.15 Virtual Machines, AWS, and AWS EC2

A virtual machine is a software-defined computer that runs on physical cloud hardware. AWS is a major cloud provider, and EC2 is its virtual-machine service. Deploying to EC2 gives the user flexible control over the server, but it also requires practical infrastructure steps such as SSH access, file synchronization, Docker installation, port exposure, and health checking.

3.1.16 Heroku and Container Hosting

Heroku is a platform for deploying web applications with less direct server management. It can run containerized applications through its container registry. In Accelera, Heroku is treated as a deployment target where the prepared Docker service can be pushed and released through Heroku CLI commands.

3.1.17 SSH and CLI Tools

SSH is a secure protocol for connecting to remote machines and running commands. A CLI is a text-based interface used to execute commands from a terminal. The Deployment module uses CLI workflows for local Docker, Heroku, and EC2. For EC2, SSH is used to run remote commands, build Docker images, start containers, and read logs.

3.1.18 Environment Variables and Configuration Files

Environment variables are runtime values provided outside the source code, such as the port on which a server should listen. Configuration files store structured settings used by the program. Accelerera uses `config.json` to locate model artifacts, define input schema rules, and configure tracking behavior.

3.1.19 Pickle Files and Model Artifacts

Pickle files with the `.pkl` extension are commonly used in Python to serialize trained objects such as Scikit-learn models or preprocessors. In the Deployment module, these files are treated as model artifacts. They are copied into the deployment workspace, loaded by the model service, and used during inference.

3.1.20 Version Control, Git, GitHub, and Repositories

Version control records changes over time and helps users return to previous states. Git is a widely used version-control system, GitHub hosts Git repositories, and a repository stores a project's files and history. Accelerera also implements a lightweight model VCS for deployment artifacts. This VCS is not a replacement for Git; it snapshots model files and deployment configuration so that earlier model versions can be restored.

3.1.21 Backend Services, FastAPI, and Uvicorn

A backend service runs server-side logic and responds to client requests. FastAPI is a Python framework for building APIs, and Uvicorn is an ASGI server used to run FastAPI applications. The Deployment module uses FastAPI to define prediction and health endpoints, while Uvicorn starts the application inside the Docker container.

3.1.22 Deployment Scripts

Deployment scripts automate repeated infrastructure commands. Instead of asking the user to copy files, build containers, start services, and check endpoints manually, a script can execute these steps in sequence. Accelerera's `deployment.py` script provides commands for preparing files, building Docker images, running locally, deploying to Heroku, and deploying to EC2.

3.2 Comparative Study of Previous Work

3.2.1 Data Preprocessing and Feature Engineering for Data Mining: Techniques, Tools, and Best Practices [4]

Koukaras and Tjortjis discuss the role of data preprocessing and feature engineering in machine learning and data mining projects. They explain that preprocessing is not only a preparation step before model training, but also an important part of the complete machine learning pipeline. The paper also presents a general preprocessing framework that includes data cleaning, data transformation, feature construction, feature selection, and dimensionality reduction. For each stage, it explains different techniques and discusses the advantages and disadvantages of each one.

3.2.2 `scikit-learn` Pipeline

`scikit-learn` is one of the most widely used machine learning libraries in Python [9]. It provides many preprocessing tools, machine learning models, evaluation metrics, and utilities for building complete machine learning workflows.

The `Pipeline` object in `scikit-learn` allows users to connect several processing steps together. Each step is usually a transformer, and the final step is usually an estimator. This makes the workflow cleaner and easier to reuse.

In this project, `scikit-learn` is the main competitor for `accelera_pipe`. Both systems provide a way to build machine learning workflows. However, `scikit-learn` pipelines are mainly linear, while `accelera_pipe` uses a graph-based backend. This allows `accelera_pipe` to support branching, shared intermediate results, and parallel execution of independent parts of the workflow.

3.2.3 OMPAr: Automatic Parallelization with AI-Driven Source-to-Source Compilation

OMPAr is a related work that focuses on automatic parallelization using AI-driven source-to-source compilation [10]. The main idea of OMPAr is to analyze C and C++ code, detect loops that can be parallelized, and generate OpenMP pragmas automatically.

OMPAr is important for this project because it inspired the general direction of the Parallelizer module. Both approaches use a staged process that includes loop analysis, loop classification, and pragma insertion. This makes the problem easier to handle because the system does not treat parallelization as one single step.

However, there is an important difference between OMPAr and our Parallelizer. OMPAr uses an AI-based pragma generation approach. In our project, the pragma generator was replaced with a rule-based generator. This decision was made because

the available training data was limited, and some of the available examples were synthetic data. Depending completely on generated or biased data could lead to unsafe pragma generation.

Therefore, our Parallelizer uses AI mainly to support loop classification, while the final pragma insertion is controlled by deterministic safety rules. This makes the system more conservative, but also safer and easier to explain.

3.2.4 Combined Algorithm Selection and Hyperparameter Optimization

Automated machine learning treats model-family selection, preprocessing choices, and hyperparameter tuning as one structured optimization problem. Instead of manually testing a small set of estimators, an AutoML system defines a search space and evaluates configurations under a consistent validation protocol.

Given a set of algorithms $\mathcal{A}$ and a hyperparameter domain Λ_a for every algorithm a , combined algorithm selection and hyperparameter optimization seeks

$$(a^*, \lambda^*) = \arg \min_{a \in \mathcal{A}, \lambda \in \Lambda_a} \widehat{L}(a, \lambda; D), \quad (3.1)$$

where $\widehat{L}$ is an estimated validation loss on dataset D . Auto-WEKA formalized this combined view by searching over both learning algorithms and their hyperparameters [5]. The domain is conditional: parameters belonging to a random forest are inactive when the selected model is logistic regression. This conditional structure motivates the use of the ConfigSpace representation in Accelera AutoML.

3.2.5 Search Strategies for AutoML

Grid search evaluates a fixed Cartesian product and scales poorly as the number of dimensions increases. Random search allocates trials more effectively when only a subset of hyperparameters strongly affects performance [13]. Sequential model-based optimization improves on uninformed sampling by fitting a surrogate to observed configuration costs. SMAC uses a random-forest model for algorithm configuration problems and selects candidates through an acquisition function [12].

Accelera AutoML follows this general model-based pattern. It first evaluates initial and warm-start configurations, represents each configuration as a numeric vector, augments that vector with fidelity variables, and fits a random-forest regressor. Predictions from individual trees provide an empirical mean and standard deviation. Expected improvement then ranks a randomly sampled candidate pool. The implementation is inspired by SMAC principles but is a project-specific optimizer rather than a dependency on the external SMAC package.

3.2.6 Multi-Fidelity Evaluation

Full cross-validation on an entire dataset is expensive, particularly for large ensembles or boosting models. Multi-fidelity optimization first evaluates candidates using cheaper

approximations. Fidelity may be varied by the number of training instances, the number of validation folds, training iterations, or model capacity. Poor candidates can be discarded early, while promising ones receive more resources.

In this work, fidelity is represented by a tuple

$$f = (s, q, k, b), \quad (3.2)$$

where s is the stage index, q is the sample fraction, k is the number of cross-validation folds, and b is a multiplicative model budget. The fidelity vector is included in the surrogate input so that observations from different stages remain distinguishable.

3.2.7 Meta-Learning for Warm Starts

Meta-learning attempts to transfer knowledge across machine-learning tasks. An AutoML system can characterize a new dataset using metafeatures, compare it with previously evaluated datasets, and schedule successful historical configurations early. Auto-sklearn combines Bayesian optimization, meta-learning, and ensemble construction [6]. The intended benefit is not to guarantee that a historical configuration remains optimal, but to reduce the cold-start period before useful observations are available.

Accelera AutoML computes task-dependent statistical descriptors. Shared descriptors include the number of instances and features, numeric/categorical ratios, missing-value ratios, skewness, kurtosis, and categorical symbol statistics. Classification additionally records class entropy, class-count statistics, and class probabilities. Regression records target mean, standard deviation, skewness, and kurtosis. Stored datasets are ranked by distance in this metafeature space, and candidate configurations from the closest datasets are converted back into active ConfigSpace objects.

3.2.8 Ensemble Learning

Ensembles combine several fitted models to reduce the risk of selecting one unstable estimator. Voting uses the predictions or probabilities of multiple base estimators. Stacked generalization instead trains a meta-model on out-of-fold predictions from base models [14]. Out-of-fold generation is essential: fitting the meta-model on predictions made on each base model's own training observations would produce leakage and an overly optimistic result.

The Accelera implementation provides both strategies. Stacking uses bagged base estimators, creates out-of-fold prediction blocks, performs forward selection, and fits a logistic-regression meta-model for classification or a ridge-regression meta-model for regression. The original feature matrix may optionally be appended to the prediction blocks.

3.2.9 Related AutoML Systems

Table 3.1: Selected AutoML systems and their primary design emphasis

System	Design emphasis
Auto-WEKA	Combined algorithm selection and hyperparameter optimization over WEKA components [5].
Auto-sklearn	Bayesian optimization, dataset metafeatures, and post-hoc ensembles for Scikit-learn workflows [6].
TPOT	Evolutionary search over tree-structured machine-learning pipelines [7].
FLAML	Cost-aware, lightweight AutoML designed to find strong configurations under limited resources [8].
AutoGluon Tabular	Multi-layer stacking and ensembling with a high-level tabular prediction interface [11].
Accelera AutoML	Chosen when AutoML needs to run inside the Accelera workflow: conditional model spaces, staged fidelity, surrogate-guided sampling, packaged warm starts, voting or stacking, and integration with preprocessing, graph pipelines, and the Parallelizer.

3.2.10 Model Deployment Platforms

Deployment platforms such as Heroku, AWS EC2, AWS Elastic Beanstalk, Render, Railway, Google Cloud Run, and Hugging Face Spaces provide ways to host applications or model demos. Their role is important because they give developers infrastructure for running web services. However, these platforms mainly provide the hosting layer. The user usually remains responsible for building the ML serving application, preparing dependency files, loading model artifacts, validating inputs, logging predictions, and maintaining model versions.

Accelera's Deployment module is different in scope. It is not a general cloud provider; it is an ML-oriented deployment layer inside the Accelera workflow. It takes the generated configuration and model pickle files, prepares a FastAPI serving container, exposes prediction endpoints, runs schema validation, records telemetry, and provides commands for local Docker, Heroku, and AWS EC2 deployment. Therefore, Accelera complements infrastructure platforms by preparing the ML service itself and connecting it to the artifacts created by the rest of the framework.

Table 3.2: Deployment-related platforms and Accelera's difference.

Platform	Main focus	Accelera difference
Heroku	Application hosting with container deployment support.	Accelera prepares the ML serving container and provides Heroku-specific commands for pushing and releasing it.
AWS EC2	General-purpose virtual machines with full server control.	Accelera automates file synchronization, remote Docker build/run steps, and health checks for model services.
Render, Railway, or Cloud Run	Managed hosting for web services or containers.	Accelera focuses on creating the ML API, validation, telemetry, and deployment package before such hosting is considered.
Hugging Face Spaces	Hosted ML demos and interactive applications.	Accelera emphasizes integration with local model artifacts and repeatable deployment workflows inside the project.

3.3 Implemented Approach

3.3.1 Auto Preprocessing

In this module, we decided to implement an automated preprocessing approach that helps users prepare datasets before model training without repeating the same manual cleaning steps each time. The approach was first designed for tabular datasets, then extended to support text and image data. Its goal is to inspect the input, choose suitable preprocessing steps, apply them, save reusable artifacts, and generate reports that help the user understand what happened to the data.

For tabular datasets, the chosen approach follows a classical automated preprocessing pipeline. The main preprocessing decisions are:

- **Missing values:** Median imputation is used for numerical features because it is simple and more robust to outliers than mean imputation. For categorical features, most-frequent imputation is used because it replaces missing values with valid existing categories.
- **Numerical scaling:** `RobustScaler` is used for numerical features because tabular datasets may contain outliers or skewed values. It depends on the median and interquartile range, which makes it more stable than standard scaling when extreme values exist.

- **Categorical encoding:** The encoding method is selected according to the feature type and cardinality. Binary columns are encoded using ordinal encoding, low-cardinality categorical columns are encoded using binary encoding, and high-cardinality categorical columns are encoded using frequency encoding. This helps reduce the number of generated features and avoids high dimensionality.
- **Feature selection:** Mutual information is used because it is a filter-based method that can work with both classification and regression tasks. It helps keep the most informative features and remove weak features before model training.

This approach was chosen because it gives a good balance between automation, simplicity, and speed. More advanced techniques, such as K-nearest neighbour imputation, autoencoders, and wrapper feature selection, can be useful in specific cases, but they need more computation and may make the system harder to use for beginners.

One important addition in this project is that the module is not only used for data preparation. It also generates exploratory data analysis reports so the user can learn about missing values, feature distributions, correlations, and the preprocessing actions applied to the dataset. The same idea was extended to text and image inputs, where the module applies suitable preparation steps for each data type instead of forcing all datasets into one tabular workflow.

3.3.2 Benchmark Platform

In this module, we decided to implement a benchmark platform that supports learning through repeated experimentation. The goal is not only to rank models, but also to let users try a task, submit predictions, see their score, inspect repository links from other submissions, learn from stronger techniques, improve their own solution, and submit again.

The implemented approach separates benchmark creation from submission evaluation. A benchmark creator defines the problem, dataset links, test set, hidden target file, target column, problem type, and evaluation metric. After that, participants submit prediction files and repository links. The system validates the files, calculates the score automatically using the selected metric, stores the submission, and displays the result on a leaderboard.

This approach was chosen because it keeps the learning process fair and repeatable. All participants are evaluated using the same hidden ground truth file and the same metric configuration, while the leaderboard and repository links give users a practical way to compare approaches and improve their work.

3.3.3 accelera_pipe

In this module, we decided to implement a graph-based machine learning pipeline approach. The goal of this module is to make machine learning workflows easier to

build while also improving execution when the workflow contains branches or repeated shared steps.

The general framework of the approach is as follows:

1. The user builds a pipeline using high-level Python methods.
2. The pipeline stores each operation as a node in an internal graph.
3. Preprocessing, model training, prediction, metric evaluation, branching, and merging are represented as graph operations.
4. The graph executor determines the correct order of execution.
5. Independent branches can be executed in parallel when possible.
6. Shared intermediate results can be reused instead of recomputed.
7. Intermediate data can be released when it is no longer needed.
8. The executed graph can be stored and reused for inspection or later execution.

This approach was chosen because it keeps the pipeline idea familiar to users who already know `scikit-learn`, but changes the internal execution model. Instead of treating the workflow as only a sequence of steps, `accelera_pipe` treats it as a graph. This helps the system handle more complex workflows that include multiple models, shared preprocessing, and branch comparisons.

3.3.4 Parallelizer

In this module, we decided to implement a conservative automatic parallelization approach. The goal is to speed up supported loop-heavy code by converting it to C++, analyzing its loops, inserting safe OpenMP pragmas, and compiling it into a native Python-callable extension.

The general framework of the approach is as follows:

1. Take the input source code, file, function, or transformer method.
2. Convert supported Python code into C++ when needed.
3. Extract loops using a compiler-based analysis stage.
4. Compute loop features such as memory access, dependency indicators, reduction patterns, branches, and function calls.
5. Classify whether the loop may be parallelized.
6. Apply rule-based safety checks before inserting any pragma.
7. Insert `#pragma omp parallel for` or a reduction pragma only when the loop is considered safe.

8. Compile the transformed C++ code into a native extension.
9. Return the optimized callable to be used from Python.

The chosen approach follows the same general idea as AI-assisted parallelization systems, but with a safer final pragma generation stage. The pragma generator is rule-based instead of fully AI-generated because the available data was limited and partly synthetic. This reduces the risk of generating unsafe OpenMP directives.

The Parallelizer is also connected to `accelera_pipe`. When a preprocessing node contains a supported custom Python function, the pipeline can attempt to optimize this function using the Parallelizer. This allows the project to combine graph-level workflow execution with code-level acceleration.

3.3.5 Deployment Module Technologies

The Deployment module uses a set of practical software-engineering technologies to move trained artifacts toward a runnable service. This section introduces the tools used by the module; the detailed module architecture is described later in Section 4.7.

Git-style versioning inspired the internal model VCS. The module does not replace Git for source code, but it applies the same idea of named snapshots to deployment artifacts. The VCS stores model files and `config.json` inside commit directories, records metadata in an index file, and allows a previous model snapshot to be restored before deployment.

FastAPI and Uvicorn form the serving layer. FastAPI defines HTTP endpoints for single prediction, CSV prediction, health checks, tracking summaries, and the interactive GUI. Uvicorn runs the application inside the container and listens on the configured service port.

Docker is used as the portability layer. The deployment script writes a `Dockerfile`, copies serving files, installs runtime dependencies, adds the saved model artifacts, exposes the service port, and starts the Uvicorn server. This keeps the runtime environment more repeatable than launching the service directly from an arbitrary local Python environment.

Great Expectations supports the runtime validation layer. The configuration file defines expected feature names, data types, required fields, allowed values, and numeric bounds. Incoming requests are checked and coerced before they reach the model service, which reduces the risk of invalid payloads causing hidden prediction errors.

SSH and `rsync` are used for AWS EC2 deployment. The script connects to the remote machine, creates the target directory, synchronizes the `Dockerfile`, configuration, serving source, and model files, builds the container on the remote host, starts it, and checks the health endpoint. Heroku deployment uses Heroku CLI commands to authenticate, push the container image, release it, and open the deployed application.

Configuration files and generated artifacts are the connection point between training and deployment. The `config.json` file tells the service where to find model pickle files, how to validate input data, and whether telemetry should be enabled. The deployment scripts then convert these local artifacts into a concrete service package.

Chapter 4: System Design and Architecture

4.1 Overview and Assumptions

This section introduces the assumptions made for each module in Accelerera and explains why these assumptions were needed during the implementation. The system includes modules for preprocessing, AutoML, graph-based pipeline execution, code parallelization, deployment, and benchmarking. In this architecture, the AutoML module provides automated supervised model selection for tabular classification and regression under configurable search limits.

4.1.1 Accelerera Pipeline Assumptions

The system combines Python-facing APIs with native execution components. The design assumes that machine-learning workflows can be represented as a graph of operations and that only a restricted, analyzable subset of custom source code should be translated to parallel C++.

4.1.2 Auto Preprocessing Assumptions

The Auto Preprocessing module was designed based on the following assumptions:

- **Data assumption:** The input data should be valid, meaningful, and one of the supported types, so the module can select the correct preprocessing workflow.
- **Configuration assumption:** The user should provide the needed configuration values, so the module can apply the correct preprocessing steps.

4.1.3 AutoML Assumptions

The Accelerera AutoML module was designed based on the following assumptions:

- **Task assumption:** The input task is assumed to be supervised tabular classification or regression, so the module can validate the feature matrix and target vector using Scikit-learn-compatible utilities.
- **User configuration assumption:** The user is assumed to provide the required search settings, such as metric, cross-validation folds, time budget, trial limit, and whether meta-learning or ensembles should be enabled.

4.1.4 Benchmark Assumptions

The Benchmark Platform was designed based on the following assumptions:

- **Benchmark data assumption:** The benchmark creator is assumed to provide the benchmark files correctly, including the training data, test data, and separate hidden target file. The creator is also responsible for ensuring that the file separation does not cause data leakage, so the platform can evaluate submissions fairly.
- **Learning and submission assumption:** Users are assumed to submit prediction files in the required format and, when possible, provide a repository link for their solution. This allows other users to compare techniques, learn from stronger submissions, improve their own models, and submit again.
- **Metric assumption:** The admin or benchmark creator is assumed to define the correct metric information for the benchmark, so the scoring script can calculate results consistently for all submissions.

4.1.5 Deployment Assumptions

The Deployment module assumes that the selected model or pipeline has already produced serializable model artifacts and a deployment configuration file. The configuration must describe the model files relative to the deployment workspace, the expected input schema when validation is enabled, and the tracking settings used by the serving application.

For local execution and cloud/server execution, the module assumes that Docker is available on the target machine or can be installed during the EC2 workflow when the corresponding option is selected. For remote EC2 deployment, the user must provide a reachable host, a valid SSH user, and any required private key. The public port used by the service must also be reachable from the client machine for the final health check and GUI endpoint to work.

4.2 System Architecture

4.2.1 Block Diagram

The block diagram in Figure 4.1 gives the big picture of Accelera. It shows the system as a group of connected modules rather than one single fixed pipeline. This is intentional: a user can use each module alone, or can connect several modules together to build an end-to-end machine-learning workflow.

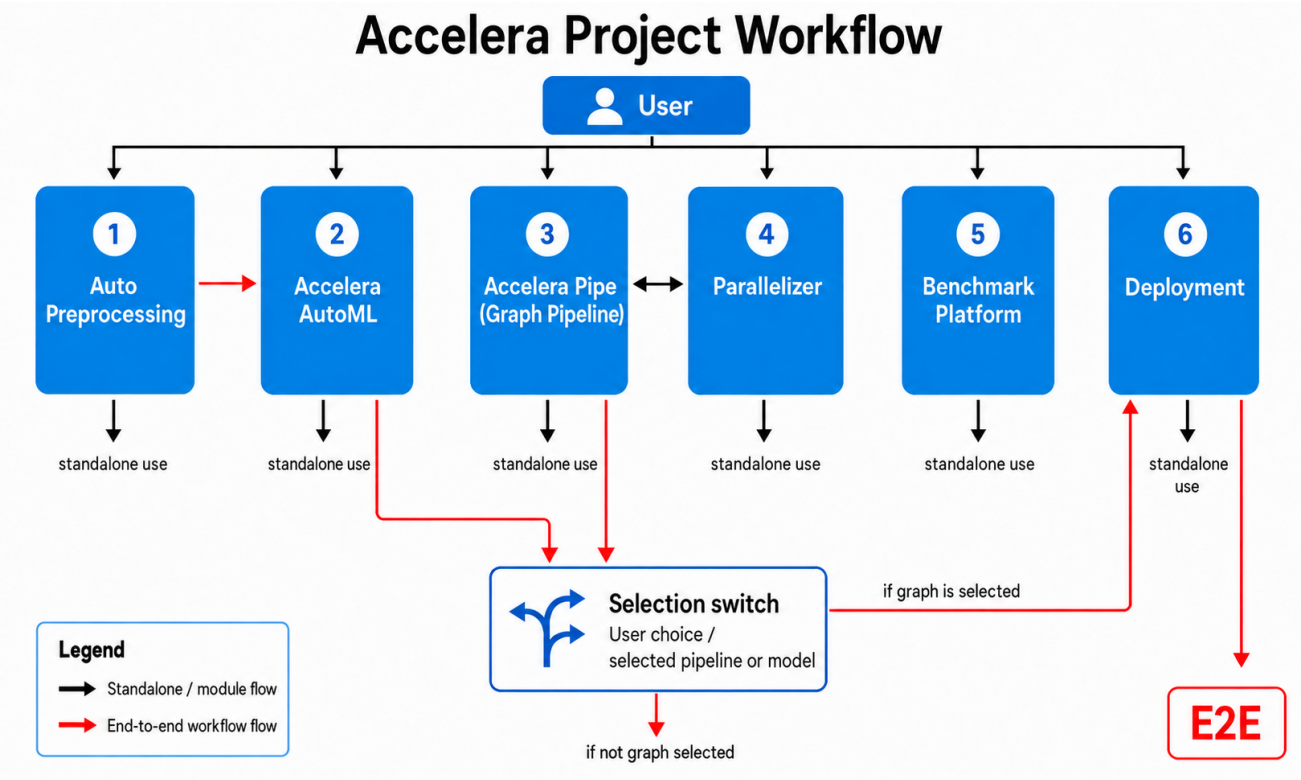


Figure 4.1: Overall block diagram of the Accelera project architecture.

At the highest level, the user can interact directly with the six main parts of the project: Auto Preprocessing, Accelera AutoML, Accelera Pipe, Parallelizer, Benchmark Platform, and Deployment. The blue arrows in the diagram represent standalone or module-level flows, while the red arrows represent the end-to-end workflow path.

The architecture follows three broad stages: data preparation, model or graph construction, and artifact consumption. Auto Preprocessing prepares reusable features and preprocessing artifacts; AutoML and Accelera Pipe provide alternative model-building paths; and Deployment or the Benchmark Platform consumes the resulting model, executed graph, endpoint, or prediction file. The Parallelizer is a supporting optimization module that can operate independently on source code or accelerate supported custom preprocessing inside Accelera Pipe.

The red route represents the complete data-to-service lifecycle, while the blue routes show standalone use. Modules exchange explicit artifacts rather than internal state, allowing users to stop after any stage.

The Deployment module turns a saved model or pipeline into a validated API service by packaging model artifacts, configuration, serving code, and deployment commands. Its outputs may include a model URL, service endpoint, health endpoint, telemetry summary, and optional inference UI.

Auto Preprocessing flow

The Auto Preprocessing block starts from a dataset and tries to prepare it for machine learning usage. The module first profiles the data and detects its type. Then it applies preprocessing steps depending on the type of input.

- **Tabular data:** missing value imputation, categorical encoding, robust scaling, and feature selection.
- **Text data:** cleaning, tokenization, stemming, and TF-IDF feature extraction.
- **Image data:** image preparation for classification tasks.

The output can be used in two ways. In the standalone path, the user can directly use the processed data, reports, and saved preprocessors. In the end-to-end path, the processed data can be passed to the AutoML block for model search.

AutoML and selection flow

The AutoML block receives a dataset, extracts statistics and metafeatures, builds candidate models and preprocessing choices, and runs a search process. Its goal is to return the best model according to the selected metric.

The important point is that AutoML is not forced to be the only way to build a model. The user may choose an AutoML result, or may choose a graph-pipeline result from Accelera Pipe. This is why the diagram contains a selection switch.

Table 4.1: Possible paths to the deployment selection switch.

Path	Meaning
AutoML path	The user lets <code>accelera_automl</code> search for the best model, then chooses the selected model for deployment.
Graph pipeline path	The user builds a workflow using <code>accelera_pipe</code> , executes the graph, and selects the executed graph or best pipeline path for deployment.
Standalone path	The user may stop after any module and use its output directly without continuing to deployment.

Accelera Pipe module

Accelera Pipe is the graph-execution module of the system. Its role is to let the user define a machine-learning workflow as a graph instead of a fixed linear sequence. Each graph node represents one operation in the workflow, such as data input, preprocessing, model training, prediction, metric evaluation, merging, or branching. The edges between nodes describe data dependencies, meaning that a node can execute only after its required source nodes have produced their outputs.

This design allows Accelera Pipe to support workflows that are more flexible than a traditional sequential pipeline. For example, the graph may contain multiple model branches, shared preprocessing nodes, or metric nodes used to select the best path. The module is also responsible for managing execution order, selected-path extraction, persistence, and intermediate-memory lifetime. After training, the selected fitted path can be stored in an executed graph so it can be reused later for inference without keeping unnecessary temporary training data.

Accelera Pipe therefore works at the workflow level. It does not only run a list of steps; it manages how the steps are connected, when they are executed, which branches are selected, and which intermediate results can be released safely.

Formally, a pipeline is represented as a directed acyclic graph

$$G = (V, E), \quad E \subseteq V \times V, \quad (4.1)$$

where each vertex $v \in V$ is an executable node and each edge $(u, v) \in E$ indicates that v consumes the output of u . A node becomes ready only after all its predecessors have completed:

$$\text{ready}(v) \iff \forall u \in \text{Pred}(v), \text{state}(u) = \text{finished}. \quad (4.2)$$

Ready nodes without mutual dependencies may be scheduled concurrently by the C++ backend. This is the basis of parallel branch execution; it is different from OpenMP loop parallelization, which operates inside one custom function.

Table 4.2: Principal node families in an Accelera Pipe graph.

Node family	Architectural responsibility
Preprocess	Fits or applies a transformer and forwards transformed features.
Model	Fits a supervised or unsupervised estimator using upstream data.
Predict	Applies a fitted model method such as <code>predict</code> or <code>predict_proba</code> .
Metric	Evaluates predictions and supplies values used for path selection.
Branch/Merge	Expands alternatives or combines compatible branch outputs.

The graph also applies consumer-count-based lifetime management. If $c_t(u)$ is the number of unfinished downstream consumers of node u after execution step t , then completing consumer v updates the count by

$$c_{t+1}(u) = c_t(u) - 1, \quad u \in \text{Pred}(v). \quad (4.3)$$

Temporary output from u becomes releasable when $c_{t+1}(u) = 0$. Consequently, a transformed array shared by several model branches remains alive until the last model consumes it, then can be cleared without affecting correctness. Duplicate first preprocessing nodes may also be shared, allowing equivalent branches to reference one transformed result rather than repeat the same work.

After metric evaluation, the configured selection strategy determines the path retained in the `ExecutedGraph`. This inference graph keeps fitted preprocessors and models but does not need transient training arrays. It can be serialized as one pickle artifact and later called with new input data.

Code Parallelizer module

The Code Parallelizer is the code-level optimization module of the system. Its main role is to analyze supported loop-heavy code and attempt to accelerate it using C++ and OpenMP. The module can be used as a standalone tool or as an internal optimization step when Accelera Pipe receives a custom preprocessing function.

The Parallelizer accepts several kinds of input:

- C/C++ file or source code.
- Python file or Python source code.
- Python function.
- Custom transformer method.

Table 4.3: Parallelizer input dispatch and output behavior.

Input	Normalization	Result
C/C++ file	Read native source directly	Parallelized source file.
Python file	Convert supported Python AST to C++	Parallelized source file.
C/C++ or Python string	Detect syntax and convert Python when required	Parallelized C++ source string.
Python function	Capture source and helpers, flatten nested functions, then convert	Cached native Python-callable or original function on failure.
Custom transformer	Extract its custom <code>transform()</code> method	Optimized method attached to the same instance when compilation succeeds.

After receiving the input, the module first normalizes it into a form that can be analyzed. If the input is supported Python code, it is converted into C++ so that

both Python-originated and native C/C++ code can follow the same analysis path. The module then extracts loops using a Clang AST based analysis stage. After loop extraction, it computes features such as memory access patterns, dependencies, reductions, branches, nested loops, function calls, and vectorization potential.

These features are passed through a classifier and combined with deterministic rule-based safety checks. This is important because the classifier alone is not trusted to change program semantics. If the loop is considered safe, the module inserts an OpenMP pragma such as `#pragma omp parallel for`. If the loop matches a reduction pattern, the module can generate an OpenMP reduction clause. If the loop is unsafe or unsupported, it is left unchanged.

For each extracted loop L , feature extraction produces a fixed-dimensional vector $\mathbf{x}(L) \in \mathbb{R}^{40}$. The classifier proposes a class from

$$\hat{y}(L) \in \{\text{parallel_for}, \text{reduction}, \text{none}\}. \quad (4.4)$$

The proposal is then resolved with deterministic evidence. In simplified form, the implemented decision can be written as

$$D(L) = \begin{cases} \hat{y}(L), & \hat{y}(L) \neq \text{none}, \\ \text{reduction}, & L \text{ contains a supported scalar reduction,} \\ \text{parallel_for}, & L \text{ has independent array writes,} \\ \text{parallel_for}, & L \text{ is a supported consecutive nest,} \\ \text{none}, & \text{otherwise.} \end{cases} \quad (4.5)$$

The safety analysis rejects evidence such as loop-carried dependencies, indirect writes, and early exits before an independent-write rule is accepted. For supported reductions, the generated directive has the form

$$\text{\#pragma omp parallel for reduction}(\odot : r), \quad (4.6)$$

where r is the reduction variable and $\odot$ is the recognized operator.

The final pragma generation is intentionally rule-based. This makes the behavior safer and easier to inspect, especially when the available data for fully AI-generated pragma generation is limited or synthetic.

Interaction between Accelera Pipe and the Parallelizer

The two modules interact at custom preprocessing nodes. If an Accelera Pipe graph contains a preprocessing node with a supported custom method, the graph can send that method to the Parallelizer before storing or executing it. The Parallelizer then attempts to analyze, rewrite, compile, and cache an optimized version of the method. If successful, the optimized callable is attached back to the same graph node. If

Table 4.4: Parallelizer stages.

Stage	Description
Input normalization	Accepts source code, files, functions, or transformer methods and prepares them for analysis.
Python-to-C++ conversion	Converts supported Python code into C++ so it can enter the same loop-analysis path as native C/C++ code.
Loop extraction	Uses compiler-style analysis to locate loops and record loop ranges.
Feature extraction	Computes indicators such as reductions, dependencies, array access, branches, function calls, and vectorization potential.
Safety decision	Combines classifier output with deterministic rule-based checks to avoid unsafe parallelization.
Pragma insertion	Adds OpenMP pragmas only when the loop is considered safe.
Compilation and caching	Builds optimized native code when needed and reuses cached results when possible.

optimization fails, the original method is kept.

1. The user defines a graph workflow in `accelera_pipe`.
2. The graph detects a supported custom preprocessing method.
3. The method is sent to the Parallelizer.
4. The Parallelizer analyzes, rewrites, compiles, and caches the optimized implementation when possible.
5. The optimized callable is returned and reattached to the graph node.
6. The graph continues execution using either the optimized method or the original method if optimization fails.

This interaction connects workflow-level optimization with code-level optimization. Accelera Pipe controls the structure and execution of the machine learning graph, while the Parallelizer attempts to speed up supported custom code inside that graph. As a result, the system can optimize both the way the workflow is scheduled and the way expensive preprocessing code is executed.

The replacement contract preserves behavior at the pipeline boundary. Let f be the original Python preprocessing callable and let $\mathcal{C}(f)$ denote source capture, conversion, OpenMP rewriting, compilation, and module loading. The callable attached to the preprocessing node is

$$f_{\text{node}} = \begin{cases} \mathcal{C}(f), & \text{if every optimization stage succeeds,} \\ f, & \text{if source conversion, analysis, or compilation fails.} \end{cases} \quad (4.7)$$

This fallback contract gives the integration three useful properties:

- **Transparent use:** the graph calls the node through the same Python interface regardless of which implementation is attached.
- **Failure isolation:** unsupported syntax or a compiler failure disables the optimization for that callable without invalidating the graph.
- **Persistent reuse:** source-backed functions retain serializable Python source, while successful native modules can be recovered from cache.

Benchmark Platform flow

The Benchmark Platform is used for learning, evaluation, and ranking. It does not take a full user pipeline as input. Instead, it works with benchmark data links, user prediction submissions, and optional repository links that show the technique used by the participant.

A benchmark owner creates a benchmark by providing the required CSV links:

- training CSV file link,
- testing CSV file link,
- ground-truth target CSV file link.

A participant submits:

- a predictions CSV file containing the prediction column,
- and optionally a GitHub repository URL for the project.

The platform compares the submitted prediction column with the true labels, computes the score, and updates the ranking. Users can then inspect the leaderboard, open repository links from other submissions, learn which techniques performed better, improve their own model, and submit another prediction file. This makes the benchmark module focused on fair evaluation and continuous learning, not on directly executing the user's pipeline.

Deployment flow

The Deployment block receives saved model artifacts and the active deployment configuration. It prepares the deployment service by collecting the model files, copying the serving source code, writing runtime requirements, generating a Docker image definition, and exposing a FastAPI application. Inside the serving container, requests pass through schema validation before reaching the model service. After inference, telemetry is written so the user can inspect request counts, statuses, latencies, and prediction activity.

The block also separates the service package from the target environment. The same prepared service can be built locally in Docker, pushed to Heroku through container commands, or synchronized to an AWS EC2 virtual machine over SSH and run as a remote Docker container. This design makes Deployment the final architectural bridge between the outputs of AutoML or graph pipelines and real-world usage through a reachable HTTP endpoint.

The final deployment block produces a deployed model URL or service. This output has two possible meanings:

- **Standalone use:** the user deploys a saved model or pipeline directly and uses the endpoint.
- **End-to-end use:** the user selects a model or pipeline through the selection switch, deploys it, and receives the final E2E service output.

Overall architecture meaning

The main idea of the block diagram is that Accelera is modular but connected. Each module gives value by itself, but the full system becomes stronger when the modules are combined.

Table 4.5: Standalone use versus end-to-end use.

Mode	Description	Example
Standalone use	The user uses one module and stops at its output.	Use Auto Preprocessing only to generate processed data and reports.
End-to-end use	The user connects several modules together to move from data to deployed result.	Preprocess data, run AutoML or graph pipeline, select output, deploy service.
Integrated acceleration	A module internally uses another module to improve execution.	Accelera Pipe sends supported custom preprocessing methods to the Parallelizer.

Therefore, the block diagram communicates three important ideas. First, the system supports direct access to each module. Second, the modules can be connected

into a complete machine-learning workflow. Third, the interaction between Accelera Pipe and the Parallelizer allows the project to combine graph-based execution with source-level acceleration.

4.3 Accelera Pipe Module

4.3.1 Functional Description

The Accelera Pipe module can be described formally as a directed acyclic graph:

$$G = (V, E)$$

where V is the set of computational nodes and E is the set of directed dependency edges between them. Each node $v_i \in V$ represents one pipeline operation, such as preprocessing, model fitting, prediction, merging, or metric evaluation. Each edge $(v_i, v_j) \in E$ means that node v_j depends on the output of node v_i . To guarantee valid execution, the graph must satisfy the acyclic condition:

$$\nexists (v_1, v_2, \dots, v_k) \text{ such that } v_1 = v_k$$

This condition ensures that the graph contains no circular dependencies. Therefore, the backend can execute the pipeline using a valid topological ordering. If $\tau(G)$ denotes a topological order of the graph, then every dependency must appear before the node that consumes it:

$$u \rightarrow v \Rightarrow \tau(u) < \tau(v)$$

During execution, each node applies its stored operation to the outputs received from its parent nodes. For a node v_i , the output can be represented as:

$$o_i = f_i(o_{p_1}, o_{p_2}, \dots, o_{p_m})$$

where f_i is the operation stored in the node, and $o_{p_1}, o_{p_2}, \dots, o_{p_m}$ are the outputs of its parent nodes. This formulation allows both sequential and branched workflows to be handled by the same execution model.

For branch-based execution, the pipeline may produce several candidate paths. If $P = \{p_1, p_2, \dots, p_n\}$ is the set of candidate paths and $M(p_i)$ is the metric value of path p_i , the selected path can be written as:

$$p^* = \arg \max_{p_i \in P} M(p_i)$$

for maximization metrics such as accuracy, or:

$$p^* = \arg \min_{p_i \in P} M(p_i)$$

for minimization metrics such as loss or error. The selected path p^* is stored inside the ExecutedGraph, which keeps only the fitted transformations and models needed for later inference.

At inference time, the executed graph applies only the selected fitted path to new input data. If X_{new} is the new input, the inference output can be represented as:

$$\hat{y} = G^*(X_{\text{new}})$$

where G^* is the fitted executed graph selected during training. This separation reduces unnecessary training-time objects and keeps inference focused on the final reusable path.

4.3.2 Modular Decomposition

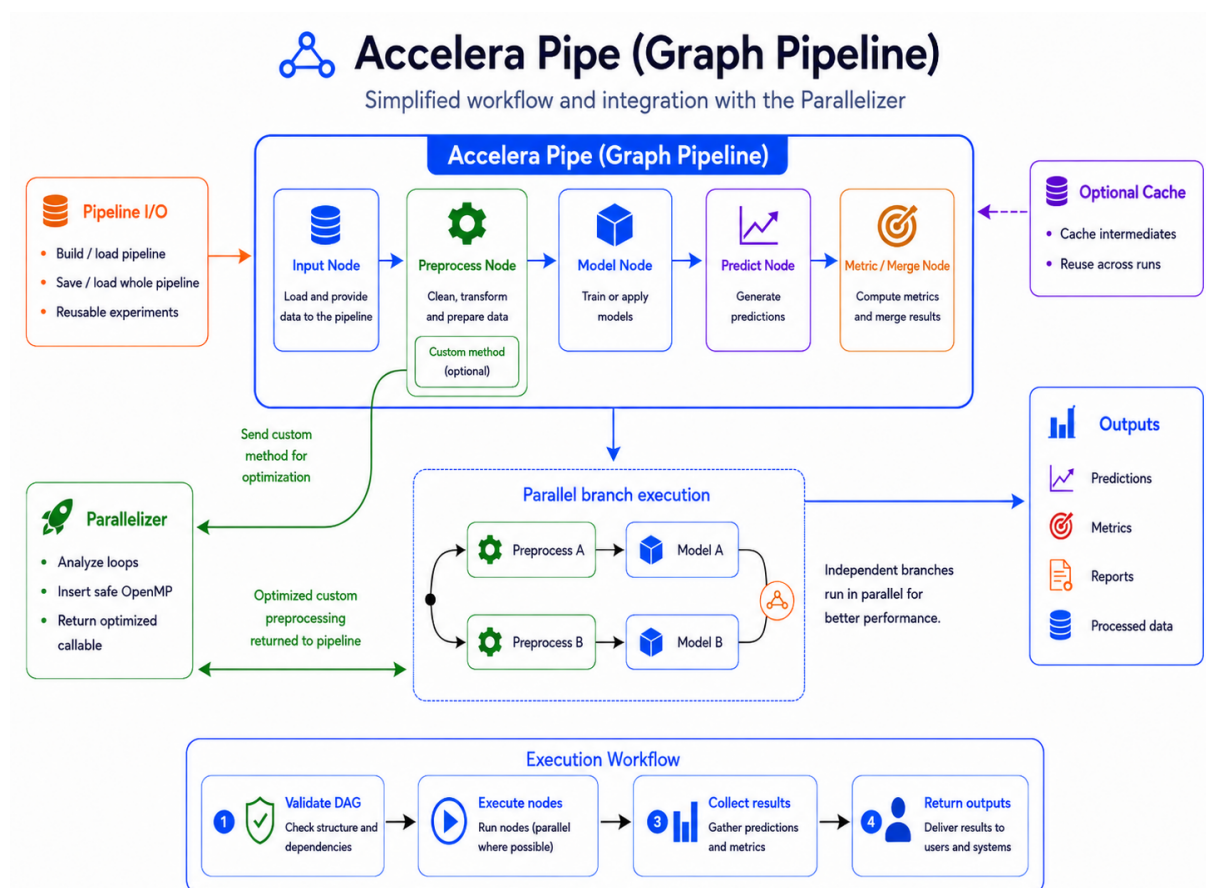


Figure 4.2: Accelera Pipe workflow with custom preprocessing parallelization and pipeline reuse.

The Accelera Pipe module is decomposed into two main layers: a Python API layer and a native C++ graph execution layer. As shown in Figure 4.2, the Python layer provides the user-facing builder interface used to define preprocessing, model, prediction, metric, branch, and merge nodes. It also handles metric wrapping, reports,

serialization, save/load support, and integration with the Code Parallelizer.

The native C++ layer manages the internal graph representation and execution behavior. It stores nodes and edges, validates graph connections, computes the topological execution order, schedules independent branches for parallel execution, selects the final fitted path, and manages intermediate memory. When a preprocessing node contains a supported custom method, Accelera Pipe can send it to the Parallelizer, receive an optimized callable, and attach it back to the graph before execution.

Table 4.6: Main components of the Accelera Pipe module

Component	Responsibility
Pipeline Builder Interface	Provides the user-facing fluent API used to define preprocessing stages, models, prediction steps, metrics, merge operations, and branches. It translates each user call into a typed computational node instead of immediately executing the operation.
Pipeline State and Persistence Manager	Owns the native graph object, maps high-level node categories to graph node types, controls graph execution settings, and provides save/load behavior for reusable pipeline objects.
Executed Graph Runtime	Represents the fitted execution path selected after training. It allows the same trained transformations and models to be reused for inference without keeping unnecessary validation data or temporary training branches.
Metric and Display Wrappers	Adapt supervised metrics, unsupervised metrics, arrays, figures, dictionaries, and reports into graph-compatible outputs that can be stored, selected, displayed, or used in branch-selection strategies.
Native Graph Engine	Stores the pipeline as a directed acyclic graph, validates node connections, computes execution order, schedules sequential or parallel execution, manages branch selection, and controls intermediate-memory lifetime.
Node Abstraction Layer	Defines the common behavior of input, preprocessing, model, prediction, merge, and metric nodes, including source-node relationships, stored data, GPU usage flags, and selected-path markers.

At the Python API level, each builder method creates or configures one graph operation. A preprocessing node stores a function or transformer together with execution parameters such as `execute_fit` and `cache`. Before a preprocessing function is stored, the module attempts to parallelize it through `parallelizer.parallelize`.

Model nodes store estimators, prediction nodes store test data and the prediction method name, metric nodes store metric wrappers, and merge nodes store the merge strategy.

Table 4.7: Mapping from builder calls to graph node semantics

Builder method	Node type	Main role
<code>preprocess(name, func)</code>	PREPROCESS	Transform input data or selected columns.
<code>model(name, model)</code>	MODEL	Fit or apply a machine-learning estimator.
<code>predict(name, test_data)</code>	PREDICT	Run a model output function such as <code>predict</code> .
<code>metric(name, metric)</code>	METRIC	Evaluate predictions using a supervised or unsupervised metric.
<code>merge(name, strategy)</code>	MERGE	Combine outputs from multiple prediction branches.
<code>branch(name, ...)</code>	Multiple nodes	Create alternative candidate paths in the DAG.

4.3.3 Python-to-Native Graph Construction

The implementation divides graph construction between the Python builder in `accelera/src/accelera_pipe/core/pipeline.py` and the C++ graph in `src/core/graph.cpp`. The Python methods do not execute machine learning operations immediately. Instead, each method packages the operation and its options into a Python object and passes that object, a node type, and a name to the PyBind11 graph binding. Table ?? shows the concrete payload and runtime output used by each node.

This separation keeps the user-facing code simple while allowing the backend to control the actual execution. For example, when the user adds a preprocessing step or a model, the Python side only describes what should happen. The C++ backend later decides when this node is ready to run based on its dependencies. This is useful because the graph may contain branches, shared nodes, prediction nodes, and metric nodes that cannot be executed safely in the same order they were written by the user.

The C++ graph also stores runtime information that is needed during execution, such as node sources, cached outputs, selected branches, and fitted objects. This means that the pipeline is not just a list of Python function calls. It is converted into an executable graph structure that can be scheduled, evaluated, saved, and reused. As a result, the Python layer focuses on usability, while the native layer handles dependency tracking, memory management, and graph execution.

Table 4.8: Concrete payload and output contracts of Accelera Pipe nodes

Node	Stored Python payload	Runtime output
Input	Training or inference values supplied to <code>Graph.execute</code> .	Provides <code>x</code> and optional <code>y</code> to downstream nodes.
Preprocess	<code>func</code> , <code>execute_fit</code> , and optional <code>cache</code> , a column selection is transformed <code>x</code> and unchanged or first wrapped in a Scikit-learn <code>ColumnTransformer</code> .	A shared dictionary containing transformed <code>y</code> .
Model	Estimator, <code>execute_fit</code> , and optional <code>cache</code> .	A deep-copied fitted estimator stored as the node data.
Predict	Validation <code>test_data</code> , output method name, and optional positive-class index.	Predictions, probabilities, or one selected probability column.
Metric	A supervised or unsupervised metric wrapper containing the execution method and optional target or feature data.	A structured metric result used for reporting and path selection.
Merge	A strategy name, the current implementation supports hard or majority voting.	A sample-wise modal class after converting branch outputs to NumPy arrays.

The native graph validates every proposed edge before adding it. A preprocessing or model node can follow an input or preprocessing node, prediction must follow a model, merge receives prediction leaves, and metric follows prediction or merge. A normal node added after several leaves is copied once per valid leaf, whereas a merge node records all valid leaves as its source vector. Any graph mutation clears the compiled flag so that the topological order is regenerated before the next execution.

The Python `branch` method accepts either one node per branch or an ordered list of nodes. It flattens the corresponding payloads, types, and names before calling `Graph.split`. The C++ method reconstructs each chain using branch offsets. Duplicate sharing is local to one original leaf: a fresh `first_branch_nodes` map is created for every leaf, so nodes from unrelated split points are never merged. This keeps branch optimization safe, because only branches that truly start from the same source can share their first repeated operation. It also prevents accidental connections between separate parts of the graph. As a result, the graph can reduce duplicated work inside one split while still preserving the exact structure intended by the user.

Algorithm 4.1: Branch construction with conservative first-node sharing

Input: current graph leaves L and branch chains B

Output: expanded DAG with duplicate first operations shared

```

for each leaf  $l$  in  $L$  do
  first_nodes = empty map
  for each branch chain  $b$  in  $B$  do
    source =  $l$ 
    for each operation  $o$  at position  $j$  in  $b$  do
      type = node type recorded by the Python builder
      reject  $o$  when type cannot legally follow source
      signature = integer(type) + ":" + string(payload( $o$ ))

      if  $j == 0$  and signature exists in first_nodes then
        source = first_nodes[signature]
        continue without creating another node
      end if

       $n$  = create typed node from payload( $o$ )
      if  $j == 0$  then first_nodes[signature] =  $n$  end if
       $n$ .create_new_data = ( $j == 0$  and type is PREPROCESS)
       $n$ .copy_input = ( $j == 0$  and preprocessing may mutate input)
      connect source to  $n$ 
      source =  $n$ 
    end for
  end for
end for
mark the graph as requiring recompilation

```

The signature deliberately excludes the user-visible node name. It combines the node type with the string representation of the payload, so differently named instances representing the same first operation can share one node. Only the first position of a branch chain is eligible. Thus, branches $[p_1, p_2, p_3]$ and $[p_4, p_2, p_3]$ keep separate copies of the later p_2 and p_3 , because their incoming data may differ. A one-node branch is naturally covered because its only node is also its first node.

4.3.4 Preprocessing Execution, Copy Policy, and Optional Cache

The preprocessing implementation distinguishes creation of a new graph data container from copying the arrays inside that container. The `should_create_new_data` flag creates a new $\{X, y\}$ dictionary at the start of an independent preprocessing branch. Otherwise, the node updates and reuses its source dictionary. The separate

`should_copy_input` flag protects mutable arrays before a custom operation is called.

The copy decision is conservative for unknown callables. Custom functions are assumed capable of in-place mutation and therefore receive copies of `X` and `y`. A Scikit-learn transformer normally receives the original input because its standard transform contract returns transformed data. If its shallow parameters expose `copy=False`, or parameter inspection fails, copying is enabled. This avoids unconditional branch copies while still protecting the graph from known or uncertain in-place behavior.

Algorithm 4.2: Preprocess node execution and data ownership

Input: source node `S` and preprocessing payload `F`

Output: transformed data object attached to node `N`

```

read (X, y) from the input node or S.data
if N.copy_input then
    X = X.copy()
    y = None if y is None else y.copy()
end if

if F.func provides fit then
    remember the original unfitted function in the training graph
    transformer = deepcopy(F.func)
    if cache is enabled and matching model/data files exist then
        load transformer and transformed X with joblib
    else
        if graph is a training graph then fit transformer on (X, y) end if
        X = transformer.transform(X)
        store the fitted transformer in F.func
        if cache is enabled then persist transformer and X end if
    end if
else
    X = F.func(X)
end if

if N.create_new_data then
    N.data = {"X": X, "y": y}
else
    update the shared source dictionary and point N.data to it
end if

```

Caching is disabled by default because hashing and storing large arrays can cost more time and memory than recomputation. When enabled for preprocessing,

the cache key is a Joblib hash of the transformer and X , the fitted transformer and transformed data are stored separately. Model caching similarly hashes the estimator, X , and y , then stores the fitted model. Both paths use the project cache directory and are intended for repeated runs with equivalent inputs, not as a replacement for executed-graph persistence.

During prediction, a `PredictNode` walks backward from its fitted model, collects all preprocessing payloads, reverses them into training order, and applies each fitted transformer or custom function to validation data. An executed graph instead obtains new data from its input node. The configured output method is then invoked on the fitted estimator. For `predict_proba`, a `nonnegative positive_class` selects one probability column after bounds validation.

4.3.5 Branching and Path Selection

The native backend uses a graph abstraction rather than a linear pipeline. The `Graph` class stores all nodes, tracks whether the graph has already been compiled, supports cloning, and exposes methods for adding nodes, splitting branches, executing, clearing, saving, loading, and enabling parallel execution. The graph computes a topological execution order before running. This guarantees that each node is executed only after its source nodes have produced their outputs. If the graph contains branches, the execution result is followed by path selection using one of the supported strategies: maximum metric, minimum metric, custom strategy, or all paths.

Algorithm 4.3: High-level Accelerera Pipe training flow

Input: training data X , optional labels y , graph G , selection strategy S

Output: predictions or metric results R , executed graph E

- 1: Add X and y to the input node of G
 - 2: Compile G if its topological order is not already available
 - 3: Execute graph nodes sequentially or in parallel depending on G settings
 - 4: Collect outputs from final leaf nodes
 - 5: If G contains branches, select the path requested by S
 - 6: Clone the selected fitted path into a compact executed graph E
 - 7: Remove validation-only data and temporary training data
 - 8: Return R and E
-

Branching is handled by the native `split` operation. Each branch is converted into a chain of graph nodes starting from the current leaf. If the current leaf node is L , then a branch B_i can be represented as:

$$B_i = (L, n_{i1}, n_{i2}, \dots, n_{ik})$$

where $n_{i1}, n_{i2}, \dots, n_{ik}$ are the operations added to that branch. A branch may contain

a single node or a list of nodes, so the same mechanism supports simple model comparison as well as full alternative sub-pipelines.

A useful optimization in branch construction is duplicate first-node sharing. If several branches begin with the same first node, the graph creates that first node once and connects the later branch consumers to its output. The sharing condition is:

$$n_{11} = n_{21} = \dots = n_{m1}$$

where n_{i1} is the first node of branch B_i . This is safe for the first node because all such branches receive the same original input. The optimization is deliberately conservative: later repeated nodes are not merged automatically because their inputs may already differ due to earlier branch operations.

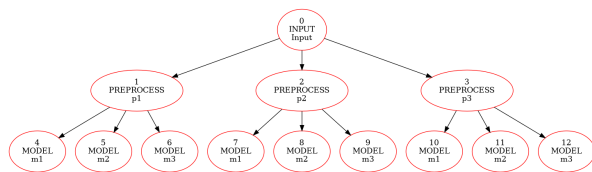


Figure 4.3: Before duplicate first-node sharing

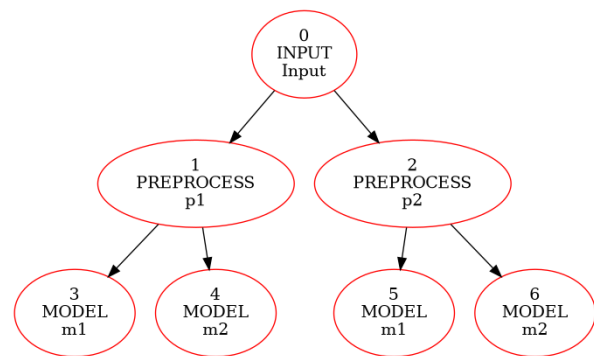


Figure 4.4: After duplicate first-node sharing

4.3.6 Executed Graph Creation and Pickle State

Path selection occurs after graph execution because metric values are required to choose a branch. Numeric `max` and `min` strategies inspect the result field of each metric node. A custom strategy receives all metric dictionaries with their node names and returns the selected metric-node name. Selection marks the chosen metric and repeatedly follows its source pointer to the input, while `all` marks every node.

This separation allows the original pipeline to be trained again while the executed graph owns the fitted models and fitted preprocessing payloads required for inference. Saving either object uses Python `pickle`. `PyBind11` links `pickle` to `Graph.getState` and `Graph.setState`: every node stores its type, name, Python payload, flags, selected-path state, data, and source-node indices. Loading first recreates typed nodes, then performs a second pass that reconnects indices to node pointers. Compilation state is invalidated after loading so the topological order is safely rebuilt. This makes saved pipelines portable across program runs as long as the same Python classes and dependencies are available. It also keeps inference simpler because the executed graph already contains the selected fitted path instead of all candidate branches.

 Algorithm 4.4: Construction of the fitted executed graph

Input: completed training graph G and selection strategy s

Output: compact fitted graph G_{star}

```

evaluate  $s$  and mark selected_in_path on the retained ancestry
create a new input node for  $G_{\text{star}}$ 
for each selected non-input node  $n$  in  $G$  do
    clone  $n$  and record old-to-new mapping
    retain fitted data only for MODEL nodes
end for
reconnect every cloned source through the old-to-new mapping
preserve selected topological order when available
remove validation test_data from cloned PREDICT nodes
remove  $y_{\text{true}}$  and  $X$  from cloned METRIC wrappers
mark  $G_{\text{star}}$  as executed
clear arrays and temporary runtime data from training graph  $G$ 
restore original unfitted preprocessors in  $G$ 
return  $G_{\text{star}}$ 
  
```

4.3.7 Intermediate Memory Management

Machine-learning pipelines can produce large intermediate arrays, especially when several preprocessing branches are evaluated. Accelera Pipe therefore tracks remaining consumers for each node output. Once the last downstream consumer finishes, the corresponding intermediate data can be released. Model and metric nodes are treated more carefully because they may hold fitted estimators or evaluation state, while input and preprocessing data can often be cleared after their consumers are done. This policy reduces memory pressure without changing the logical result of the graph.

 Algorithm 4.5: Consumer-count based intermediate release

Input: executed node N , remaining consumer table C

Output: graph with unused temporary data released

```

1: for each source node  $S$  that feeds  $N$  do
2:      $C[S] = C[S] - 1$ 
3:     if  $C[S] == 0$  and  $S$  is releasable then
4:         clear the temporary data stored by  $S$ 
5:     end if
6: end for
  
```

The consumer-count mechanism is used after each node finishes executing. Every

source node keeps a counter representing how many downstream nodes still need its output. For a source node S , the remaining-consumer count can be written as:

$$C(S) = |\{N \mid S \rightarrow N \text{ and } N \text{ has not executed yet}\}|$$

When a downstream consumer completes, the counter is decreased:

$$C(S) \leftarrow C(S) - 1$$

If the counter reaches zero and the source node is safe to clear, the temporary output is released. This allows the graph to reduce memory pressure during branch-heavy training runs without changing the final predictions or selected path.

4.3.8 Parallel Graph Execution

The graph can run independent nodes in parallel, but a node can start only after all of its parents have finished. For any node N , the readiness condition is:

$$\text{ready}(N) \iff \forall P \in \text{Parents}(N), \text{finished}(P) = \text{true}$$

The backend uses CPU thread semaphores, a separate GPU semaphore, and dependency checks over the topological order. This prevents a downstream node from reading incomplete data and also serializes GPU work when necessary. CPU nodes are allowed to execute concurrently as long as worker slots are available, while GPU nodes are handled more conservatively using a single execution slot. This makes the scheduler safe for mixed CPU/GPU graphs without changing the dependency semantics.

The scheduler repeatedly checks which nodes are ready and assigns them to available execution resources. If several branches become ready at the same time, they do not need to wait for each other unless there is a dependency between them. This is useful in branch-heavy workflows because preprocessing, model training, prediction, or metric nodes from different branches may run at the same time. At the same time, synchronization keeps the final execution result equivalent to a valid sequential topological run. In other words, parallelism is used only to improve execution time, not to change the logical order of the pipeline. The number of CPU threads is based on hardware concurrency but can be overridden using the `ACCELERA_NUM_THREADS` environment variable.

 Algorithm 4.6: Parallel graph execution scheduling

Input: topological execution order T

Output: completed graph execution or reported error

```

1:  Initialize finished and started state for all nodes
2:  Build remaining-consumer counts for intermediate outputs
3:  Create limited CPU worker slots and one GPU execution slot
4:  while unfinished nodes still exist do
5:      for each node  $N$  in  $T$  do
6:          if  $N$  has not started and all parent nodes are finished then
7:              Acquire a CPU slot or the GPU slot depending on  $N$ 
8:              Mark  $N$  as started
9:              Launch  $N$  asynchronously
10:             After  $N$  finishes:
11:                 Mark  $N$  as finished
12:                 Release unused source outputs
13:                 Release the CPU or GPU slot
14:             end if
15:         end for
16:     end while
17:     Wait for all launched tasks
18:     Rethrow any stored node execution error
  
```

Algorithm 4.6 describes the runtime scheduling idea implemented by the native graph backend. The graph is already arranged in topological order, but the scheduler does not simply execute nodes one by one. Instead, it repeatedly searches for nodes whose parents have finished and launches them asynchronously when a CPU or GPU execution slot is available. CPU nodes share a limited pool of worker slots, while GPU nodes use a separate single-slot semaphore to avoid unsafe concurrent GPU execution. After a node completes, the backend marks it as finished, releases any temporary source outputs that are no longer needed, and signals the scheduler so that newly ready child nodes can start. Any exception raised inside a node is stored and rethrown after all launched tasks are joined, which gives a clear error message containing the failed node name and whether it was executed as a CPU or GPU node.

4.3.9 Design Constraints

The Accelera Pipe module has several design constraints that guide how the graph is constructed, executed, and reused. The first constraint is that the internal workflow must always remain a valid directed acyclic graph. Since execution depends on topological ordering, cyclic dependencies are not allowed. For any edge from node u

to node v , the execution order must satisfy:

$$u \rightarrow v \Rightarrow \tau(u) < \tau(v)$$

where τ represents the topological order of the graph. This guarantees that every node is executed only after its required source nodes have already produced their outputs.

The second constraint is node-type validity. Not every node can be connected to any other node. For example, a prediction node must follow a model node, a metric node must follow a prediction or merge node, and a merge node should combine compatible prediction outputs. These restrictions preserve the semantic meaning of the pipeline and prevent invalid workflows from reaching the execution stage.

The third constraint is safe memory management. Since branch-heavy workflows can produce many temporary arrays, the backend must release intermediate data only when it is no longer needed. For a source node S , the remaining-consumer count is:

$$C(S) = |\{N \mid S \rightarrow N \text{ and } N \text{ has not executed yet}\}|$$

The output of S can be cleared only when $C(S) = 0$ and the node is safe to release. This reduces memory pressure without changing the final selected path or prediction results.

The fourth constraint is parallel execution safety. Independent nodes may run in parallel, but a node can start only after all of its parent nodes have completed:

$$\text{ready}(N) \iff \forall P \in \text{Parents}(N), \text{finished}(P) = \text{true}$$

This prevents downstream nodes from reading incomplete data. The backend also uses limited CPU worker slots and a separate GPU slot to avoid unsafe resource sharing during execution.

Finally, the executed graph must not retain training-only data. After training, the selected path is cloned into an `ExecutedGraph`, validation inputs are removed, metric targets are cleared, and temporary arrays are released. This keeps the saved inference graph compact and focused only on fitted transformations and models.

4.3.10 Summary

The Accelera Pipe module also supports persistence. The Python `PipelineBase` class saves and loads pipeline objects using pickle, while the C++ graph exposes its state as a Python dictionary containing node information, selected-path flags, source indices, cached data, graph compilation state, and parallel execution configuration. During loading, the graph rebuilds the nodes and reconnects them using the stored source indices, making the native graph serializable.

The module can also export a graph-like representation of the pipeline structure. The graph utility code writes an XML-style network description containing layers for

input, preprocessing, model, prediction, merge, and metric nodes, together with their edges. This representation is useful for debugging, visualization, and understanding the executable graph generated from the high-level pipeline.

In summary, Accelera Pipe is a hybrid Python/C++ pipeline execution system. Python provides the user-facing construction interface, while C++ provides graph semantics, execution scheduling, branch selection, memory-lifetime control, and parallel execution support, keeping the module both efficient and easy to use.

4.4 Code Parallelizer Module

4.4.1 Functional Description

The Code Parallelizer module is responsible for transforming supported loop-heavy source code into optimized C++ code annotated with OpenMP pragmas. It is designed to reduce the execution time of custom preprocessing functions and standalone code snippets by identifying loops that can be safely executed in parallel. The module accepts different input forms: a file path, a source string, a custom Python function, or a custom transformer instance. Regardless of the input form, the scientific goal is the same: normalize the input into a C++ loop-analysis path, detect parallelization opportunities, and return either parallelized source code or a compiled Python-callable native function.

This process can be viewed as a transformation from an original program P into an optimized program P' :

$$P' = \mathcal{T}(P)$$

where $\mathcal{T}$ represents the parallelization pipeline, including source normalization, loop extraction, loop classification, safety validation, and OpenMP pragma insertion. If no safe optimization is found, the transformation keeps the program behavior unchanged by returning the original code or callable.

For Python input, the module first lowers a restricted Python subset into C++. This conversion is intentionally limited to predictable language constructs such as arithmetic expressions, assignments, `for range(...)` loops, conditionals, function definitions, array-style indexing, and simple calls. The resulting C++ code is then analyzed by the same loop extraction path used for native C/C++ input. This design avoids building two separate parallelization pipelines and ensures that both Python and C++ inputs are judged using the same loop features and safety checks.

After normalization, the module extracts candidate loops and checks whether each loop can be parallelized safely. For a loop L_i , the final decision can be represented as:

$$d_i \in \{\text{sequential}, \text{parallel_for}, \text{reduction}\}$$

where `sequential` means that the loop is left unchanged, `parallel_for` means that a normal OpenMP parallel loop pragma can be inserted, and `reduction` means that the

loop can be parallelized using an OpenMP reduction clause. This decision is based on both learned classification and rule-based safety checks, so the module does not rely only on a model prediction when program correctness may be affected.

Table 4.9: Supported operations in the Python-to-C++ converter

Category	Supported forms and notes
Constants and names	None, booleans, integers, floats, strings, and variables.
Arithmetic	Addition, subtraction, multiplication, division, modulo, and power using <code>std::pow</code> .
Unary and boolean	Unary plus, unary minus, logical not, logical and, and logical or.
Comparisons	Equality, inequality, less-than, less-than-or-equal, greater-than, and greater-than-or-equal, chained comparisons rejected.
Calls and printing	Ordinary function calls and <code>print</code> , keyword arguments are generally unsupported.
Access operations	Attribute access and indexing are supported, slicing is rejected.
Assignment	Single-target assignment and indexed assignment are supported, multi-target assignment is rejected.
Augmented assignment	Simple-name <code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code> , and <code>%=</code> .
Control flow	<code>if/else</code> blocks and <code>return</code> statements.
Loops	<code>for i in range(...)</code> loops with one, two, or three range arguments.
Functions	Function definitions are emitted as C++ template-style functions, decorators are rejected.

After normalization, the module uses Clang AST bindings to extract loop information, including the loop source code and line range. The Python layer then computes loop features, sends them to a classifier endpoint, and applies rule-based validation. If the loop is considered safe, an OpenMP directive such as `#pragma omp parallel for` is inserted. For valid reduction-style loops, a reduction clause can also be generated.

The module follows a fail-safe design. If conversion, analysis, classification, compilation, or native extension loading fails, the system keeps the original Python function or leaves the code unchanged. This ensures that parallelization remains an optimization without altering the program semantics.

4.4.2 Modular Decomposition

The Code Parallelizer is built from several stages that prepare the input, analyze loops, check safety, and generate optimized OpenMP/native code.

Table 4.10: Main components of the Code Parallelizer module

Component	Responsibility
Parallelization Orchestrator	Receives source strings, file paths, custom functions, and transformer instances. It coordinates conversion, loop extraction, feature analysis, classification, pragma insertion, caching, compilation, and fallback behavior.
Input Normalization Layer	Converts accepted input forms into a common source-analysis representation. Files are read, source strings are analyzed directly, functions are captured from source, and transformer instances are inspected through their transformation method.
Python-to-C++ Converter	Translates a restricted subset of Python into C++ so Python preprocessing logic can enter the same static loop-analysis pipeline as native C/C++ code.
Loop Extraction Engine	Uses a compiler-style front end to locate loops, record source ranges, and expose structured loop information for feature extraction and rewriting.
Feature Extraction and Embedding Generator	Computes structural, dependency, memory-access, control-flow, arithmetic, and reduction features, then maps them into a fixed-size numeric representation.
Parallelization Classifier and Safety Resolver	Combines classifier predictions with deterministic safety rules to decide whether a loop remains sequential, receives a parallel-for pragma, or receives a reduction pragma.
OpenMP Rewriter	Inserts the selected OpenMP directive while avoiding unsafe duplicate insertion, especially when an outer loop already contains selected inner loops.
Native Compilation and Loading Layer	Wraps optimized C++ into a Python-callable extension, adds OpenMP flags only when needed, builds the native module, loads it at runtime, and returns the optimized callable.
Caching Layer	Stores processed source and compiled native functions using source-based hashes to avoid repeated conversion and compilation.

Figure 4.5 summarizes the overall data flow. In general, supported inputs are

normalized into a common C++ loop-analysis path, while callable Python inputs additionally proceed to PyBind11 extension generation.

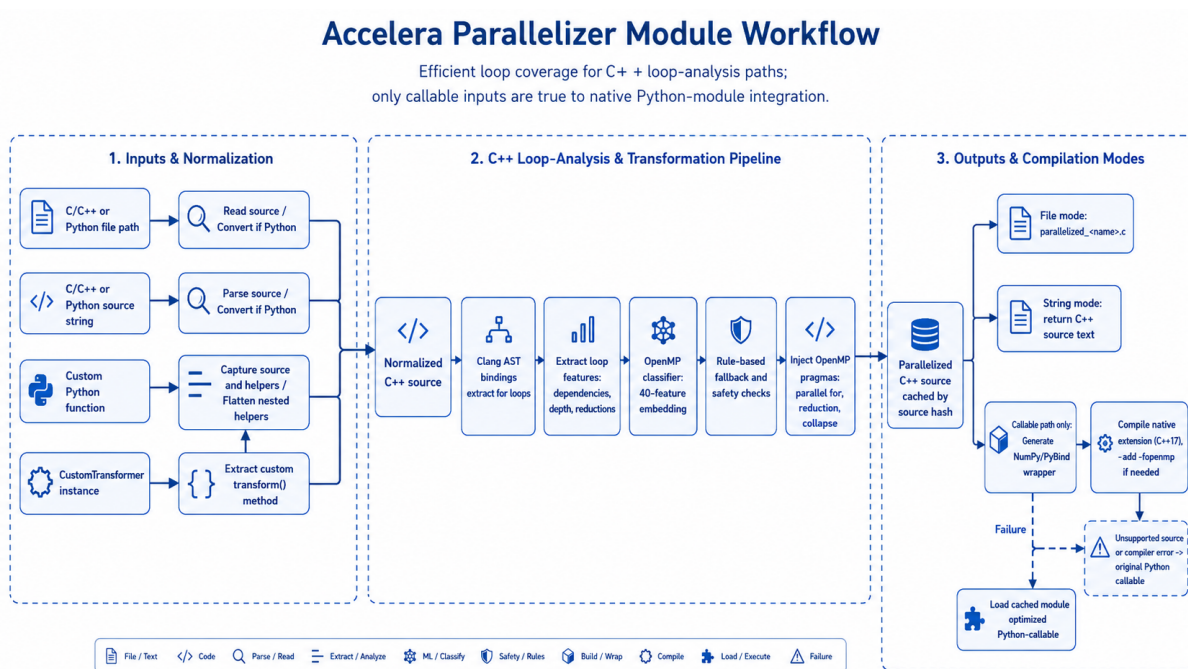


Figure 4.5: Code Parallelizer module workflow

The feature extraction stage analyzes both code structure and loop behavior, including reductions, array accesses, dependencies, indirect memory access, early exits, nesting depth, branches, function calls, arithmetic operations, and vectorization potential.

These indicators are encoded into a 40-dimensional vector for the classifier, then refined by deterministic safety rules that can still detect safe cases such as reductions, independent array writes, and nested loop patterns. This two-step design is important because the classifier gives a learned prediction, but the final decision must still protect program correctness. For example, a loop may look parallel from its general structure, but a hidden dependency or unsafe write can make it incorrect to parallelize.

The feature vector therefore acts as a compact summary of the loop, while the rule-based stage acts as a final safety filter. If the rules detect an unsafe pattern, the loop is kept sequential even if the classifier suggests parallelization. On the other hand, if the loop matches a known safe pattern, such as a valid reduction or independent indexed writes, the system can generate the appropriate OpenMP pragma. This makes the decision process more reliable and easier to inspect during debugging.

Algorithm 4.7: Loop classification and pragma insertion

Input: source program P

Output: source program P' with safe OpenMP pragmas

```

1:  if P is Python source then
2:      convert the supported Python subset into C++
3:  end if
4:  Extract for-loops using the Clang AST frontend
5:  for each extracted loop L do
6:      compute structural, memory, dependency, and reduction features
7:      send the 40-feature vector to the classifier service
8:      combine the predicted class with rule-based safety checks
9:      if the resolved class is parallel_for then
10:         insert #pragma omp parallel for
11:      else if the resolved class is reduction then
12:         insert #pragma omp parallel for reduction(...)
13:      else
14:         leave the loop unchanged
15:      end if
16: end for
17: Cache and return the transformed source code

```

The compilation path is used when the input is a Python callable. The parallelizer extracts or receives the function source, converts it to C++, parallelizes the C++ loop body, wraps the result in a PyBind11 module, compiles it as a shared native extension, loads the extension, and returns the optimized function. Caching is used at both the transformed-source level and the compiled method level. The cache key includes the compilation cache version, the function name, the compiler optimization level, the extension suffix, and the source code. This prevents recompiling the same function repeatedly.

4.4.3 Input Dispatch and Source Normalization

The public `parallelize` method is a dispatcher rather than one large conversion routine. Its behavior depends on the runtime type of the input, as shown in Table 4.11. This separation keeps file output, in-memory source transformation, and callable compilation as distinct operations while reusing the same loop-selection functions.

Table 4.11: Parallelizer input dispatch and returned output

Input form	Implementation path	Output
Existing file path	Read the file, convert it when the suffix is .py, extract, select, and rewrite loops.	A <code>parallelized_<stem>.c</code> file in the requested output directory or the repository root.
Source string	Try Python AST parsing, successful Python source is converted to C++, while other source continues as C/C++.	Parallelized C++ text returned in memory.
Custom function	Python Capture restorable source, bundle helpers, transform and compile the source, then attach the runtime native callable.	A pickle-compatible <code>SourceBackedFunction</code> , original Python execution is retained on failure.
Custom transformer	Optimize only a <code>transform</code> method defined directly by the custom class.	The same transformer instance with the method replaced when compilation succeeds.
Other Python object	No supported dispatch rule is applied.	The original object is returned unchanged.

For a file, extracted loops are serialized to a JSON record containing the loop type, start line, end line, and source text. The JSON filename is based on an MD5 hash of the normalized source. In-memory code avoids this intermediate file and converts the native `LoopInfo` objects directly to dictionaries. Both paths then call the same selection and line-rewriting routines.

Custom functions require additional source handling because a compiled runtime callable cannot normally be pickled as portable Python source. `SourceBackedFunction` stores the function name and normalized source, and recreates the Python function with `exec` during unpickling. Referenced top-level helper functions are discovered recursively through the function's global namespace and bundled before the outer function. Nested helper functions are removed from the outer body, ordered by their call dependencies, and emitted as top-level functions before conversion. The implementation rejects closures, captured outer variables, duplicate helper names, and recursive nested helpers, because those cases cannot be represented by the current stateless C++ path. Simple lambdas can be preserved as source for pipeline serialization, but the current Python-to-C++ subset does not compile lambda expressions, so callable optimization falls back to Python for that case.

Algorithm 4.8: Normalization of a Python callable for compilation

Input: custom Python function `f`

Output: restorable source `S` and compilation source `C`

```

reject f when it captures closure variables
obtain and dedent inspect.getsource(f)
parse the source into a Python AST
recursively collect referenced top-level helper functions
bundle helpers before f and save the bundle as S

```

```

find nested function definitions inside f
for each nested helper h do
    reject captured outer variables
    record calls from h to other nested helpers
end for
reject duplicate names or recursive helper dependencies
topologically order and hoist nested helpers
remove their original nested definitions from f
emit top-level helpers followed by f as C

```

4.4.4 Deterministic Pragma Generation Without AI

The classifier service is optional for the currently recognized rule patterns. For every extracted `for` loop, `_select_parallel_loops` first calls the resolver with the class `none`. The resolver can upgrade this class locally without any network request or learned model. Therefore, the system can generate pragmas when the classifier is offline, times out, returns an error, or is deliberately bypassed, provided that a loop matches one of the implemented deterministic patterns.

The rule-only path currently recognizes three families:

1. **Scalar reduction update.** The extractor recognizes scalar `v += expression`, `v = v + expression`, `v *= expression`, and `v = v * expression`. A candidate is rejected as a scalar reduction when the same name is indexed as an array. The pragma emitter supports OpenMP `reduction(+ : v)` and `reduction(* : v)` when `v` is not declared inside the loop. Maximum and minimum calls are recorded as classifier features, but the current emitter does not yet construct OpenMP `max` or `min` clauses.
2. **Independent array write.** A loop is considered an independent write candidate when at least one indexed assignment is found and the local heuristics find no previous-iteration access of the form `A[i-1]`, no indirect indexing such as `A[B[i]]`, and no early `break`, `continue`, or `return`. This rule produces a plain `parallel for` directive.

3. **Consecutive nested loops.** The parser counts directly consecutive for headers, allowing whitespace and braces between them. A depth greater than one selects the outer loop and produces `collapse(depth)`. When both outer and inner ranges appear in the selected-loop list, the rewriting stage skips the inner range so that it does not create nested duplicate pragmas.

 Algorithm 4.9: Classifier-free rule-based OpenMP generation

Input: normalized C++ source `P` and extracted for-loops `L`

Output: source `P_rule` containing deterministic OpenMP pragmas

```

selected = empty list
for each loop l in L do
  features = extract_features(l.code)

  if l contains a supported scalar reduction update then
    class = reduction
  else if l writes an array and has no detected carried dependency,
    indirect access, or early exit then
    class = parallel_for
  else if consecutive_for_depth(l.code) > 1 then
    class = parallel_for
  else
    class = none
  end if

  if class is not none then append (l, class) to selected end if
end for

for each selected loop ordered by its source range do
  skip it when its range is contained by a selected outer loop
  clauses = empty list
  if consecutive depth > 1 then add collapse(depth) end if
  if external scalar += reduction then add reduction(+ : variable) end if
  if external scalar *= reduction then add reduction(* : variable) end if
  replace the original source range with pragma, clauses, and loop text
  update the line-offset shift caused by inserted pragma lines
end for
return rewritten source

```

No classifier endpoint is contacted for a loop selected by these rules. If the rules return `none`, the implementation attempts to send the 40-dimensional feature vector to the remote classifier. A classifier failure is caught and converted back to `none`, the

resolver is then called again, so unresolved loops remain unchanged. If the classifier returns a non- `none` class, the current resolver accepts that class and the pragma emitter constructs the directive. Consequently, the deterministic path is a useful offline fallback and pre-classifier shortcut, but the current implementation should not be described as a complete dependence proof for all classifier-selected loops. Extending the rejection rules for aliasing, side-effecting calls, and general loop-carried dependencies remains future work.

4.4.5 Feature Vector and Source Rewriting Details

Before classification, comments are removed and structural features are computed from the loop text. The 40-dimensional vector contains Boolean flags for reductions, carried dependencies, control flow, constant bounds, nesting, RAW/WAR indicators, indirect access, early exit, computable trip count, and vectorizability. Other positions encode bounded counts for array reads and writes, nesting depth, branches, calls, arithmetic operations, memory operations, read/write variables, and reduction variables. Array names are mapped into fixed five-dimensional hashed summaries so the request size remains constant across loops.

The classifier is called with an HTTP POST request whose JSON body contains the vector under `embedding`. The configured timeout bounds network delay. The expected response provides one of `none`, `parallel_for`, or `reduction`. Rule-selected loops bypass this request entirely, which also makes common reductions and indexed writes available when no Internet connection is present.

Rewriting uses the source ranges reported by Clang rather than searching for the loop text globally. For each selected loop, the one-based start and end lines are converted to Python slice indices. The generated pragma-plus-loop segment replaces that slice. A running `shift` records how many lines earlier insertions added, ensuring that the ranges of later loops still address the correct code. A selected loop is skipped when its complete range lies inside another selected range, preventing duplicate inner directives.

4.4.6 Native Wrapper, Compilation, and Cache Layers

Callable optimization adds a second transformation after pragma insertion. `to_numpy_code_cpp` removes the generated `main`, strips template declarations, maps temporary template types to `double`, and rewrites the selected one-argument function to accept a `py::array_t<double>`. The wrapper creates a mutable unchecked two- dimensional view, translates `len(X)` and `len(X[i])` to array shape queries, and converts two-dimensional indexing to view calls. This means the current direct native-callable path is intentionally restricted to one named input parameter representing a two-dimensional double array.

 Algorithm 4.10: Cached compilation of an optimized Python callable

Input: Python function `f` and restorable source `S`

Output: compiled callable `f_native` or original `f`

```

C = build compilation source from S and helper dependencies
method_key = hash(cache_version, name, optimization, extension, C)
if method metadata and shared module exist then
    dynamically load and return the cached exported function
end if

C_cpp = convert supported Python AST to C++
P = apply rule/classifier loop selection and insert OpenMP pragmas
module_key = hash(cache_version, name, optimization, extension, P)
wrap P as a PyBind11 NumPy extension named from module_key

if the shared module does not exist then
    compile with C++17, shared-library, and release-definition flags
    add -fopenmp only when P contains an OpenMP pragma
    add Python and PyBind11 include paths
    add PIC and pipe flags on non-Windows systems
    use sccache or ccache when enabled and available
end if

dynamically import the extension and register it in sys.modules
write method metadata that points to the compiled module
return the exported function
on conversion, compilation, or loading failure, return original f
  
```

There are two persistent cache levels in addition to Python's in-process module table. The processed-code cache hashes the cache-format version and input source and stores rewritten C++. The Python-method cache additionally includes the function name, compiler optimization flag, and platform extension suffix, then stores the generated module name in JSON. The compiled-module name hashes the final C++ text with the same compatibility inputs. Including `COMPILE_CACHE_VERSION` lets developers invalidate old artifacts after changing conversion or wrapper semantics.

The compiler defaults to `c++` on Unix-like systems and searches for Clang on Windows, the `CXX` environment variable can override it. `ACCELERA_CPP_OPT_LEVEL` selects the optimization flag, and `ACCELERA_USE_COMPILER_CACHE=0` disables `sccache` or `ccache`. OpenMP is linked only when the transformed source actually contains `#pragma omp`, avoiding an unnecessary OpenMP requirement for unchanged serial callables.

4.4.7 Integration with Accelera Pipe

The Code Parallelizer integrates with Accelera Pipe through custom preprocessing nodes. If `Pipeline.preprocess()` receives a custom Python function, the pipeline attempts to optimize it with the Parallelizer before storing it inside the graph. If it receives a custom transformer object, the transformer instance can be optimized through its supported preprocessing method. In both cases, the optimization is best-effort: failed conversion, unsupported syntax, compilation failure, or module loading failure falls back to the original Python callable.

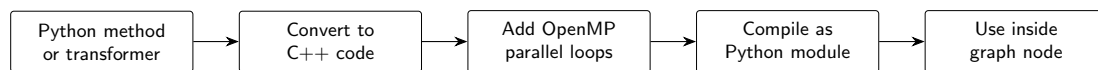


Figure 4.6: Simplified flow for parallelizing a Python method or transformer and using it inside an Accelera Pipe graph node.

Figure 4.6 summarizes the integration at a higher level. The preprocessing callable is converted to C++ when it belongs to the supported subset, the loop analysis stage adds OpenMP parallelism, and the compiled module is attached back to the graph node. The graph therefore keeps the same node structure while replacing eligible Python loops with native execution.

Overall, the Code Parallelizer acts as an optimization layer around user-defined numeric loops. It does not attempt to parallelize all Python programs. Instead, it targets the subset where static analysis, feature extraction, classifier prediction, rule validation, and native compilation can cooperate safely.

4.4.8 Design Constraints

The most important design constraint of the Code Parallelizer is correctness. OpenMP pragmas can improve performance, but an incorrect pragma can produce wrong numerical results. Therefore, classifier output is not applied blindly. The module applies additional safety rules to avoid unsafe transformations when there are loop-carried dependencies, indirect accesses, early exits, side-effect function calls, or invalid reduction variables. Inner loops are also skipped when an outer selected loop already covers their range, preventing nested duplicate pragma insertion.

The second constraint is the restricted Python subset. Full Python semantics are too dynamic to translate safely into C++ using a lightweight source converter. The converter therefore rejects unsupported syntax such as decorators, slicing, chained comparisons, unsupported operators, non-range loop iterators, multi-target assignment, complex dynamic dispatch, and unsupported expression forms. This restriction is not a weakness in the design, it is a correctness boundary. The system optimizes code only when it can produce predictable C++.

The third constraint is compiler and platform compatibility. Native callable optimization depends on a working C++ compiler, Python include paths, PyBind11

headers, NumPy-compatible array wrapping, and platform-specific shared extension suffixes. The compiler path uses C++17 and adds `-fopenmp` only when the parallelized code actually contains OpenMP pragmas. On non-Windows platforms it also uses position-independent code flags suitable for shared libraries. If compilation fails, the original Python callable is returned.

The fourth constraint is input and output compatibility. The current compiled callable path is oriented toward functions with one named input parameter and NumPy-like two-dimensional numeric data. The wrapper rewrites array access to PyBind11 unchecked NumPy access and exposes the compiled function back to Python. This makes the generated extension useful for numeric preprocessing loops, but it also means that arbitrary Python functions with complex object structures are outside the supported optimization target.

The fifth constraint is reproducibility and caching. Because native compilation can be expensive, the module caches processed C++ code and compiled functions using source hashes. The cache must include enough information to avoid stale or incorrect reuse when compiler options, cache version, function name, source code, or extension suffix changes. This design improves repeated execution time while keeping cache invalidation explicit.

4.4.9 Summary

Overall, the Code Parallelizer adds a code-level acceleration layer to Accelerera. It accepts supported source code, functions, and transformer methods, analyzes loops, inserts OpenMP pragmas only when the decision is safe enough, and can compile eligible Python callables into native extensions. If any step fails, the original Python behavior is preserved.

This design makes the module useful both as a standalone tool and as an internal optimization backend for Accelerera Pipe preprocessing nodes. It does not try to parallelize every Python program. Instead, it focuses on numeric loop-heavy code where conversion, analysis, rewriting, compilation, and fallback can work reliably together.

4.5 Auto Preprocessing Module

The Auto Preprocessing module automates the data understanding, exploratory data analysis, and preprocessing stages in the data science workflow. It helps reduce the repeated manual work needed to prepare datasets before modeling.

The module also helps the user understand the dataset before and after preprocessing by generating a report. This report shows important information about the input data and the preprocessing steps that were applied. For tabular data, the report shows information such as the number of rows and columns, data types, missing values, duplicate records, numerical and categorical features, basic statistics and EDA . For text data, it shows information such as the number of rows, target classes, class distribution, and number of missing. For image data, it shows information such as the number of images, image classes, class distribution, image sizes, and corrupted images. After preprocessing, the report shows the steps applied to the data, such as imputation, encoding, scaling, feature selection, text cleaning, TF-IDF feature extraction, image resizing, normalization, and image preparation for classification.

Another important design point is separating the training phase from the inference phase. In the training phase, the preprocessing objects are fitted on the training data and saved as artifacts. In the inference phase, these saved artifacts are loaded and applied to new data. This ensures that new data is processed in the same way as the training data before it is passed to the trained model for prediction.

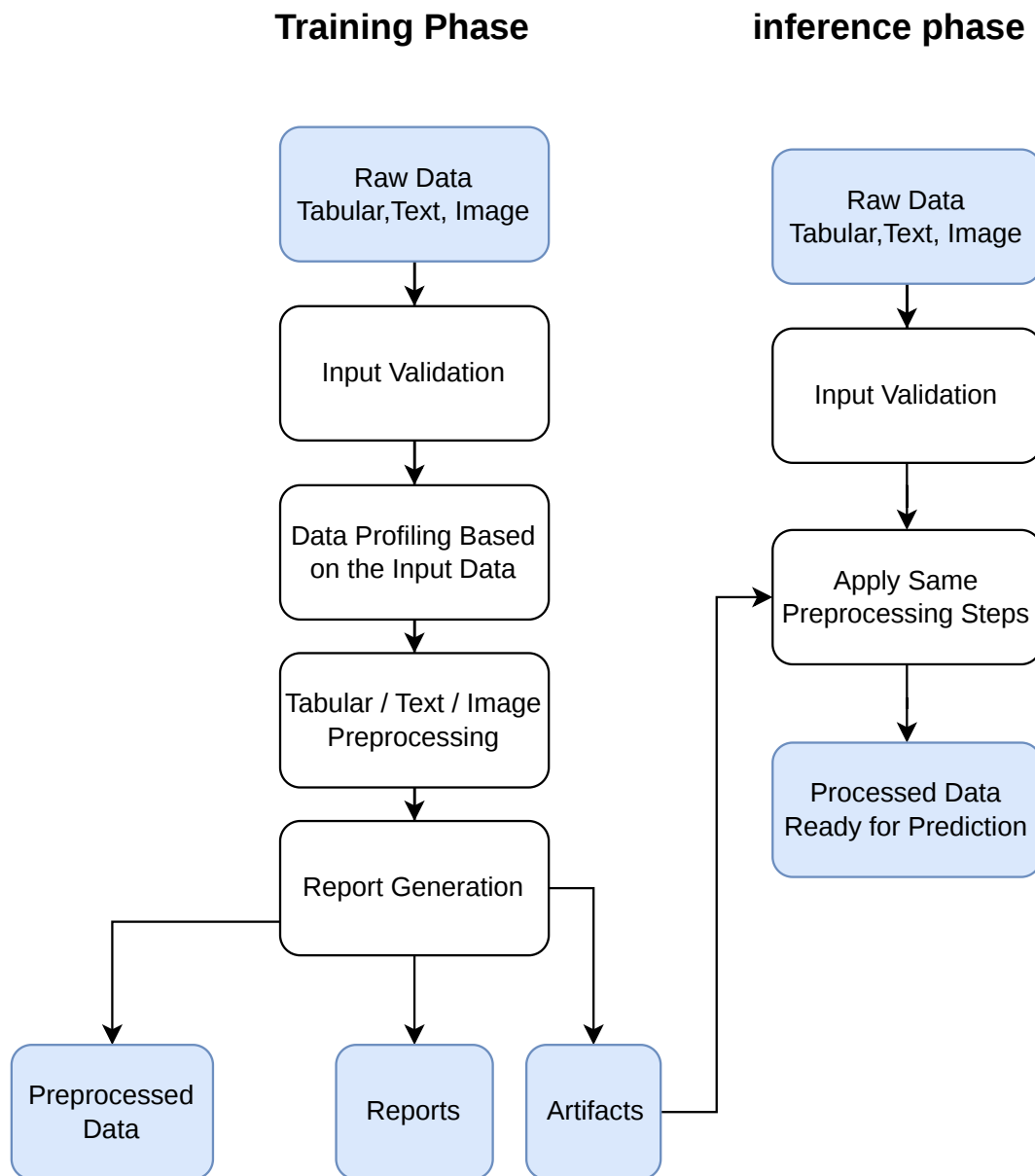


Figure 4.7: Auto Preprocessing Workflow

4.5.1 Functional Description

This module receives a dataset with its configuration, applies the suitable preprocessing workflow based on the data type and task, and returns processed data ready for training or inference. The module supports tabular, text, and image datasets. The module also generates a report when this option is enabled. The report summarizes information about the dataset and the preprocessing steps applied.

4.5.1.1 Tabular datasets

For tabular datasets it supports both classification and regression tasks. It analyzes the dataset, removes duplicate records, splits the data into training and validation sets, detects column types, handles missing values, encodes categorical features, scales numerical features, applies target preprocessing, performs feature selection, and saves the preprocessing objects needed later for test data.

4.5.1.2 Text datasets

It supports classification tasks. It cleans the text, tokenizes it, applies text preprocessing steps, and converts the text into a numerical representation suitable for machine learning models.

4.5.1.3 Image datasets

It supports image classification by reading images from class folders, validating image files, resizing images, applying augmentation, normalizing pixel values, and preparing the data loaders needed for training.

4.5.2 Modular Decomposition

The Auto Preprocessing module is divided based on the type of input data. Each data type has its own preprocessing workflow because every type has a different structure and needs different techniques:

- **Tabular data preprocessing:** handles structured datasets made of rows and columns.
- **Text data preprocessing:** handles text columns and converts cleaned text into numerical features.
- **Image data preprocessing:** handles image classification folders and prepares image loaders for training.

4.5.2.1 Tabular Data Preprocessing Functions

- **__init__():** This is the constructor that receives the dataset and the user settings. It validates these settings before starting the preprocessing process.
- **handel_bool_types():** This function handles columns that have bool data type. It finds these columns and converts them into integer values.
- **data_overview():** This function gives a general overview of the dataset before preprocessing.
- **drop_target_nulls():** This function drops rows have null value in the target.

- **drop_duplicates():** This function removes repeated rows from the dataset.
- **split_data():** This function splits the dataset into training and validation sets. The split is based on the validation size and random state selected by the user. The output of this function is `X_train`, `X_val`, `y_train`, and `y_val`.
- **get_data_info():** This function studies each column in the training data. It collects important information such as data type, number of unique values, missing values, and missing percentage. For numerical columns, it also calculates the mean, and median.
- **is_drop_column():** This function decides whether a column should be removed or kept. A column is removed if it is included in the user selected drop list, if it has only one constant value, or if its missing value percentage is higher than the selected threshold.
- **drop_col():** This function removes the selected columns from both the training and validation datasets. It also saves the dropped columns and their reasons in a file. This helps the system apply the same dropping step later on testing or new data.
- **detect_column_types():** This function detects the type of each feature. This step is important because each feature type needs a different preprocessing method. The features are divided into binary columns, ordinal columns, numerical columns, continuous columns, low cardinality categorical columns, high cardinality categorical columns, and unsupported columns. The function also stores the preprocessing steps for each column to be shown later in the report.
- **make_graphs():** This function creates graphs when the report option is enabled. The type of graph depends on the feature type and the problem type.

Table 4.12: Detected Column Types in Tabular Data

Feature Type	Description	Examples
Binary	Columns containing exactly two unique values.	yes/no, 1/0
Ordinal	Integer columns with a small number of unique values less than or equal to <code>max_ordinal_unique</code> .	Ratings or levels such as 1, 2, 3, 4
Numerical	Integer columns with a number of unique values greater than <code>max_ordinal_unique</code> .	Age, number of transactions
Continuous	Floating point columns.	Height = 175.5, price = 19.99, temperature = 36.6
Low Cardinality Categorical	Object or string columns with a number of unique values less than or equal to <code>cardinality_threshold</code> .	Product categories, country group
High Cardinality Categorical	Object or string columns with a number of unique values greater than <code>cardinality_threshold</code> .	Product names, cities
Other	Columns with unsupported or non standard data types. These columns are excluded from the preprocessing pipeline.	Dates, complex objects, unsupported formats

Table 4.13: EDA graphs generated for tabular preprocessing

Column type	Problem type	Generated graphs
Binary and categorical	Classification	Null vs non null pie chart, category count plot, and category count split by target class.
Binary and categorical	Regression	Null vs non null pie chart, category count plot, and target box plot for each category.
Ordinal	Classification	Null vs non null pie chart, ordered count plot, and ordered count plot split by target class.
Ordinal	Regression	Null vs non null pie chart, ordered count plot, and target box plot for each ordinal value.
Numerical continuous and	Classification	Null vs non null pie chart, histogram with KDE, and feature box plot grouped by class.
Numerical continuous and	Regression	Null vs non null pie chart, histogram with KDE, and scatter plot between feature and target.
Target column	Classification	Null vs non null pie chart and target class count plot.
Target column	Regression	Null vs non null pie chart, target histogram with KDE, and target box plot.
Numeric columns	Both	Correlation heatmap.

- **features_preprocessing():** This function applies the preprocessing steps to the input features. It creates different pipelines based on the detected feature types:
 - Numerical and continuous features are filled using the median and scaled using robust scaling.
 - Binary features are filled using the most frequent value and encoded using ordinal encoding.
 - Low cardinality categorical features are filled using the most frequent value and encoded using binary encoding.
 - High cardinality categorical features are filled using the most frequent value and encoded using frequency encoding.
 - Ordinal features are filled using the most frequent value and scaled using robust scaling.

These pipelines are combined using `ColumnTransformer`. The transformer is fitted on the training data and then applied to the validation data. The fitted preprocessor and the generated feature names are saved for later use.

- **target_preprocessing():** This function prepares the target column based on the problem type. For classification label encoding is used to convert class labels into numbers. For regression robust scaling is applied. The target preprocessor is saved so the same transformation can be used later.
- **features_importance():** This function selects the most useful features using mutual information with the target. Only features with scores greater than or equal to the selected threshold are kept. The selected features are saved in a file to use it in the inference step.
- **common_preprocessing():** This is the main function that controls the full tabular preprocessing workflow. It calls the previous functions in order. It starts by handling boolean columns, generating the data overview, drop rows have null target, removing duplicates, and splitting the data. Then, it extracts column information, drops unsuitable columns, detects column types, generates graphs, preprocesses the features, preprocesses the target, applies feature selection, and generates the report if enabled. It returns the processed training and validation data.

 Algorithm 4.11: General tabular preprocessing flow

Input: DataFrame D, target column y, problem type, other configurations

Output: X_train, X_val, y_train, y_val

- 1: Validate configuration
 - 2: Store original dataset columns
 - 3: Build data overview and report information
 - 4: Remove rows that have null target values
 - 5: Remove duplicate rows
 - 6: Split into training and validation sets
 - 7: Drop user-selected, high-missing, constant, and unsupported columns
 - 8: Detect column types from training data
 - 9: Generate feature and target graphs when report is enabled
 - 10: Fit feature preprocessing objects on X_train
 - 11: Transform X_train and X_val
 - 12: Fit target preprocessing object on y_train
 - 13: Transform y_train and y_val
 - 14: Select useful features using mutual information
 - 15: Save fitted objects and metadata
 - 16: Generate report if enabled
-

Figure 4.8 shows the full tabular preprocessing workflow. It starts from the input dataset, applies data cleaning and feature preprocessing steps, then saves the processed training and validation data with the preprocessing objects needed later for testing.

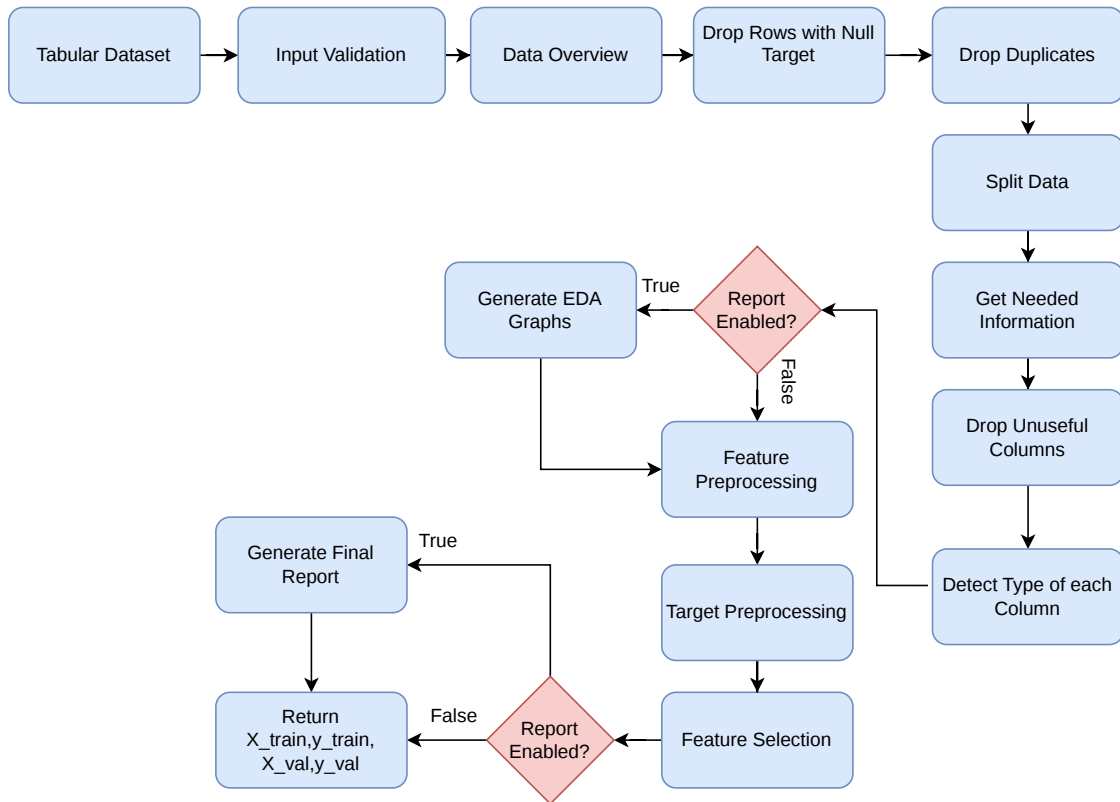


Figure 4.8: Tabular Preprocessing Workflow

4.5.2.2 Text Data Preprocessing Functions

- **__init__():** This is the constructor that receives the dataset and the user settings. It validates these settings before starting the preprocessing process. Also removes any columns that are not the text column or the target column. This keeps only the columns needed for text classification. It also downloads the required NLTK resources, such as `punkt` and `stopwords`.
- **make_graphs():** This function creates graphs if the report option is enabled. It creates a graph for the text column. It also creates a graph for the target column. These graphs help the user understand the text data and the target class distribution.
- **custom_text_tokenizer():** This function is used to clean the text and split it into words. It converts the text to lowercase, removes extra spaces and unnecessary symbols, and handles some contractions such as `can't` and `won't`. It also separates numbers from letters. Then, the text is tokenized into words. Stopwords are removed, but words like `not` and `no` are kept because they affect the meaning. After that, stemming is used to reduce words to their basic form.
- **features_preprocessing():** This function applies the preprocessing steps to the text column. It fills missing text values in the training and validation data with

empty strings. Then it uses the custom tokenizer . It also stores examples of the text before and after cleaning in the report. Then `TfidfVectorizer` is used to convert the cleaned text into numerical features. The vectorizer uses the user settings, such as the maximum number of features, n-gram range, maximum document frequency, and minimum document frequency. The vectorizer is fitted on the training data and applied to the validation data. The fitted vectorizer is saved so the same text transformation can be reused later on testing or new data.

- **target_preprocessing():** This function prepares the target column for classification this function applies label encoding to the target values. The fitted label encoder is saved. Also, the text column and target column information are saved . This helps the system apply the same target transformation later on testing or new data.
- **common_preprocessing():** This is the main function that controls the full text preprocessing workflow. It calls the other functions in the correct order. It creates the data overview. Then, it removes rows that have missing target values and removes duplicate rows. Then, it splits the dataset into training and validation sets. If the report option is enabled, graphs are created. Then, feature preprocessing is applied to clean and vectorize the text. Target preprocessing is applied to encode the class labels. Generates the report if enabled. It returns the processed training and validation data.

Algorithm 4.12: Text tokenizer flow

Input: raw text sentence

Output: cleaned token list

- 1: Convert text to lowercase and remove extra spaces
 - 2: Normalize apostrophe characters
 - 3: Expand common negative contractions
 - 4: Add spaces between letters and digits
 - 5: Remove characters except letters, digits, and apostrophes
 - 6: Split text into words
 - 7: Remove stopwords except "not" and "no"
 - 8: Remove very short tokens
 - 9: Apply Porter stemming
 - 10: Return cleaned tokens
-

Figure 4.9 summarizes the text preprocessing workflow. It shows how the text data is cleaned, tokenized, and converted into TF-IDF features before the final processed data is returned.

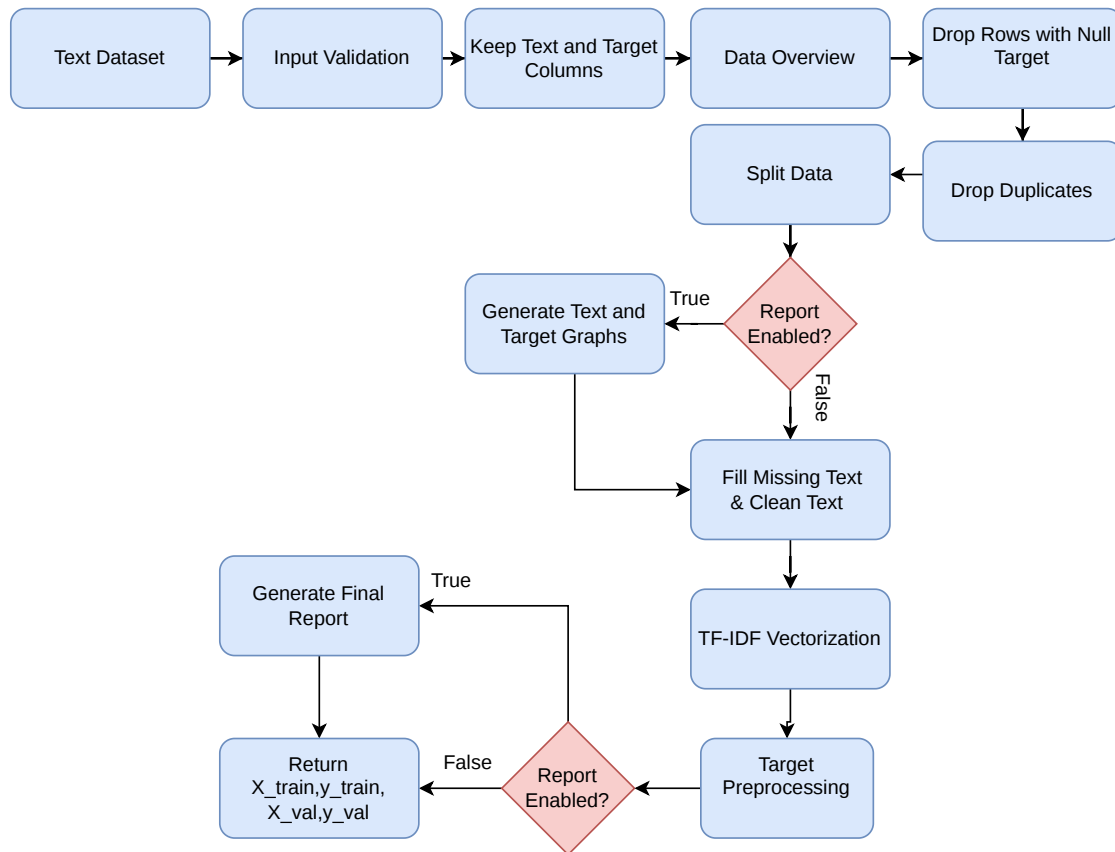


Figure 4.9: Text Preprocessing Workflow

4.5.2.3 Image Data Preprocessing

It is used for image classification datasets. In this module, the images are organized inside folders, where each folder represents one class. It is divided into a set of functions, and each function has a specific role in the image preprocessing workflow. The main functions are described as follows:

- **__init__():** This is the constructor that receives the dataset and the user settings. It validates these settings before starting the preprocessing process. It also prepares empty lists to store invalid images and their labels. It saves the image size information in a file so it can be reused later.
- **get_classes_mapping():** This function creates the class mappings. It gives each class name a numerical label. It creates two mappings: one from class name to label, and another from label to class name. These mappings are saved in files. This helps the system use the same labels during training, validation, testing, and prediction.
- **data_preparing():** This function reads the images from the class folders. It goes through each class folder and checks each image path. If the image is valid and

can be opened, its path is added to the list of valid image paths, and its class label is added to the labels list. If the image is invalid or corrupted, it is added to the invalid images list with its label. If there are no valid images, the function raises an error because the preprocessing process cannot continue without valid image data.

- **data_overview():** This function collects general information about the image dataset. For the training folder, it stores the class names, number of valid images, random sample images, number of invalid images, invalid image paths, and the class mapping. If a validation folder is provided, it also stores the same type of information for the validation folder. This information is used later in the final preprocessing report.

- **make_graphs_label_summary():** This function creates graphs that show the label distribution. It creates a label summary graph for the training folder. If a validation folder is provided, it also creates a label summary graph for the validation folder.

If the training data is split into training and validation sets, the function also creates graphs for the labels after splitting. These graphs help the user understand if the classes are balanced or not.

- **make_garphs_sample():** This function creates sample image graphs. It shows random samples from the training folder. If a validation folder is provided, it also shows random samples from the validation folder.

If the training data is split into training and validation sets, it also shows sample images after splitting. This helps the user visually check the image dataset.

- **make_graphs_loader():** This function creates sample images after using the DataLoader. It takes a small number of images from the training DataLoader and displays them.

If a validation DataLoader exists, it also displays sample images from it. This step helps confirm that the images are loaded correctly after resizing, normalization, and augmentation.

- **make_graphs():** This function controls all graph generation steps. It prepares the graph section in the report, then calls the label summary graphs, sample image graphs, and DataLoader sample graphs.

- **get_loaders():** This function creates the training and validation datasets and DataLoaders. First, it creates a `ClassificationImageDataset` for the training images using the image paths, labels, image size, and augmentation settings.

Then, it creates the training DataLoader with the selected batch size and uses shuffling. Shuffling is used for training because it helps the model see the data in a different order.

If validation data exists, the function creates a validation dataset and validation DataLoader. Augmentation is not applied to the validation data, and shuffling is not used because validation should be more stable.

- **common_preprocessing():** This is the main function that controls the full image preprocessing workflow. It starts by creating the class mappings. Then, it prepares the training images by collecting valid image paths and labels and saving invalid images.

If a validation folder is provided, it also prepares the validation images. After that, it creates the data overview, splits the data if needed, creates the DataLoaders, generates graphs, and creates the final image preprocessing report.

At the end, the function returns the training DataLoader and validation DataLoader. These outputs are ready to be used for training an image classification model.

Algorithm 4.13: Image training preprocessing flow

Input: training image folder, optional validation folder

Output: training and validation DataLoaders

- 1: Read class folders from the training directory
 - 2: Create class-to-label and label-to-class mappings
 - 3: Save mappings and image size information
 - 4: Validate image files and skip invalid images
 - 5: Build the data overview for the report
 - 6: Split training images if no validation folder is provided
 - 7: Create training dataset with resize and optional augmentation
 - 8: Create validation dataset without augmentation
 - 9: Build DataLoaders
 - 10: Generate label, sample, and loader graphs
 - 11: Generate the image preprocessing report
-

Figure 4.10 explains the image preprocessing workflow. It shows how image folders are read, invalid images are detected, class labels are created, images are resized and normalized, and final DataLoaders are prepared.

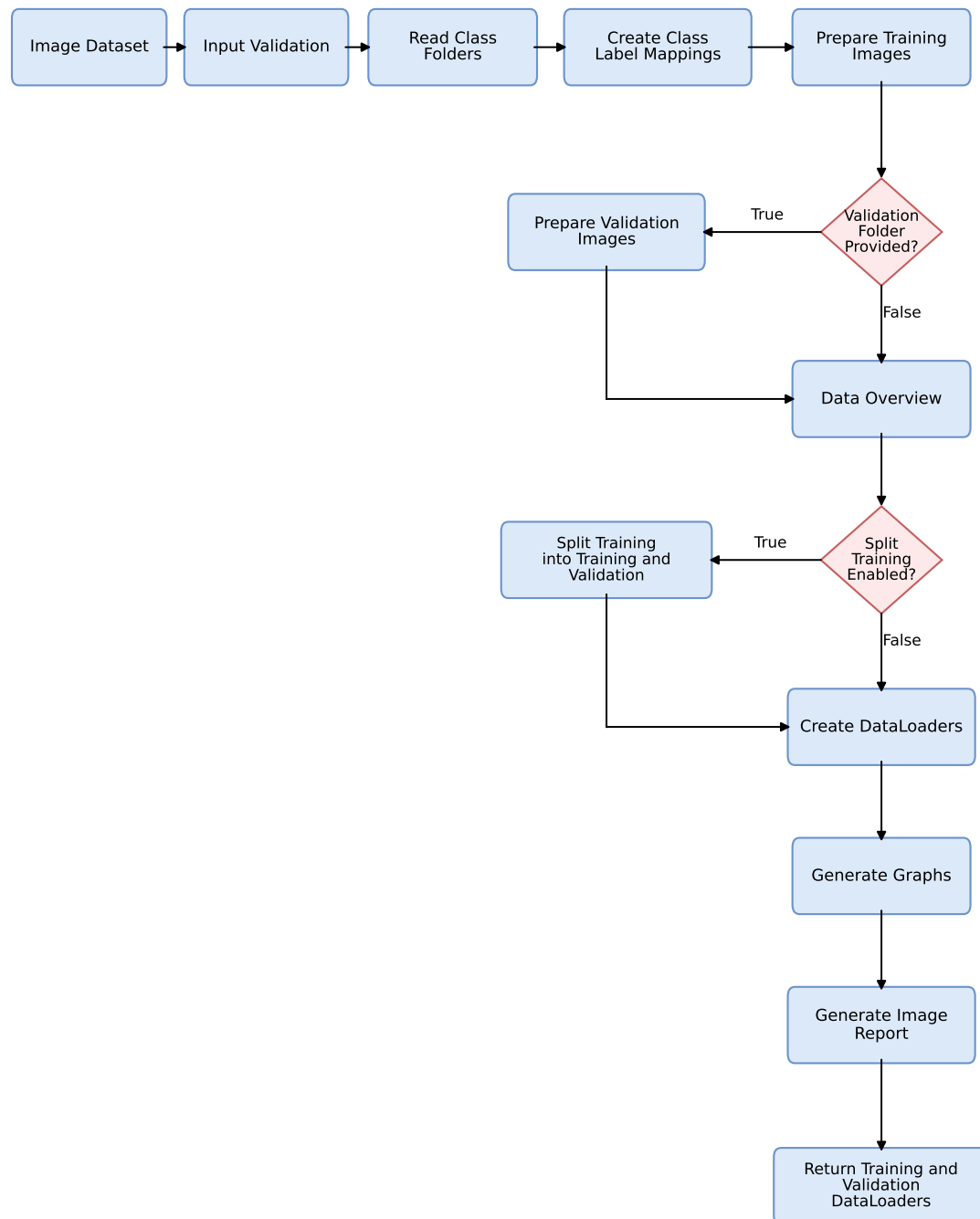


Figure 4.10: Image Preprocessing Workflow

4.5.2.4 Inference and Reusability:

For all supported data types, the saved preprocessing objects are reused during testing. This is important because the testing data must be processed in the same way as the training data.

- For tabular data, the system loads the saved preprocessors, selected features, dropped columns, and target preprocessor.
- For text data, the system loads the saved TF-IDF vectorizer and label encoder.
- For image data, the system loads the saved class mappings and image size.

This makes the preprocessing module reusable and consistent for training, validation, testing, and future prediction.

4.5.3 Design Constraints

The Auto Preprocessing module was designed under the following constraints:

- **Supported data types constraint:** The module is limited to the supported data types only, which are tabular, text, and image data. Other data types are not handled by this module.
- **Input structure constraint:** Each supported data type must follow the expected structure. Tabular and text data should be provided as structured datasets with a target column, while image data should be organized in folders based on class names.
- **Automatic decision constraint:** The module depends on rule-based preprocessing decisions. Therefore, the preprocessing choices are limited to the implemented rules and supported configuration options.

4.5.4 Module Summary

The Auto Preprocessing module provides one reusable preprocessing interface for tabular, text, and image data. It prepares data for training, generates reports when enabled, saves fitted preprocessing artifacts, and reuses the same artifacts during testing and prediction.

4.6 AutoML Module

4.6.1 Functional Description

The AutoML module automates supervised tabular model selection for classification and regression. It receives the feature matrix, target vector, and user configuration,

then searches over valid preprocessing choices, model families, and hyperparameters under explicit time and trial limits. The module exposes two Scikit-learn-style estimators, `AutoMLClassifier` and `AutoMLRegressor`, so the user can call familiar methods such as `fit`, `predict`, `score`, and `return_leaderboard`.

The workflow starts by validating the input data and resolving the task type. The engine builds a conditional configuration space where only the hyperparameters belonging to the selected model are active. Each candidate is evaluated using cross-validation and the selected metric. The evaluator converts metrics into a cost value so the optimizer can compare all candidates using one minimization objective. Failed trials and timeout cases are recorded in the run history instead of stopping the full search.

To reduce unnecessary computation, the optimizer uses a staged fidelity schedule. Early trials can run on smaller sample fractions, fewer folds, and reduced model budgets. Promising candidates are promoted to higher-fidelity stages, while weak candidates are discarded earlier. After enough observations are collected, a surrogate-guided search step ranks new candidate configurations using expected improvement. Meta-learning warm starts can also suggest initial configurations from similar historical datasets.

At the end of the search, the strongest full-fidelity configuration is rebuilt and fitted on the available training data. If ensemble construction is enabled, the module can also build voting or stacked ensembles from successful candidate models. The final model is selected by comparing the best individual model with the ensemble result, and the module returns the fitted model, best score, leaderboard, and complete run history.

4.6.2 Block Diagram

At a high level, Accelera AutoML follows a pipeline architecture. The user interacts with a public estimator, the estimator validates the input data, the AutoML engine builds a task-specific search space, and the optimizer repeatedly selects candidate configurations for evaluation. The evaluator scores candidates using cross-validation and returns structured results to the optimizer. Meta-learning can provide warm-start configurations, and the ensemble stage can combine strong candidates before final model selection.

Figure 4.11 shows the main blocks of the Accelera AutoML module and the flow between user input, search, evaluation, and final output.

Simplified AutoML Workflow

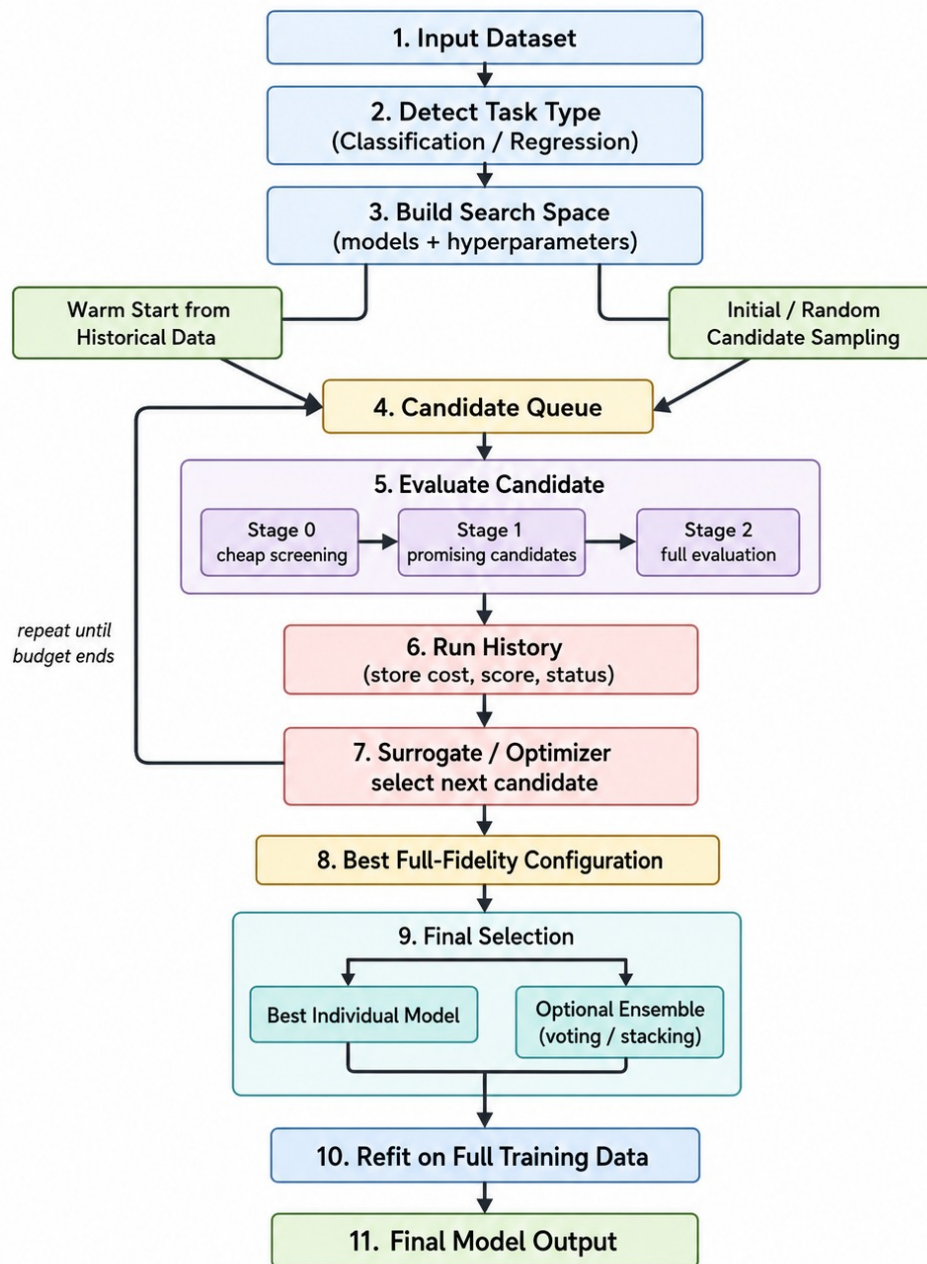


Figure 4.11: Block diagram of the Accelera AutoML module.

4.6.3 Accelera AutoML Module Design

4.6.3.1 Method Overview

The complete method is organized as the following sequence:

1. validate the feature matrix and target;
2. resolve compatible or user-allowed model families;

3. construct the conditional configuration space;
4. compute dataset metafeatures and retrieve warm-start configurations;
5. evaluate initial configurations at low fidelity;
6. fit a surrogate after sufficient finite observations exist;
7. select new candidates by expected improvement;
8. promote strong candidates through higher fidelity stages;
9. fit the best full-fidelity individual configuration;
10. optionally construct and evaluate an ensemble.
11. return whichever fitted candidate has the stronger validation score.

4.6.3.2 Configuration-Space Construction

The configuration space defines the set of machine-learning pipelines that the AutoML engine is allowed to explore. In this work, classification and regression use independent component registries because the available estimators and their valid hyperparameters differ between task types. Each component in a registry specifies four main elements: a model name, a hyperparameter factory, conditional rules, and an estimator builder. The global configuration contains a categorical variable called `model_name`. This variable represents the learning algorithm selected for a trial. After a model is selected, only the hyperparameters belonging to that model are activated. Hyperparameters belonging to other models remain inactive and are ignored during evaluation. This conditional structure prevents the optimizer from treating irrelevant parameters as active decisions. For example, the number of neighbors is meaningful for a k-nearest-neighbors model, but it is not meaningful for a random forest or a support-vector machine.

Formally, let the set of candidate models be

$$\mathcal{M} = \{m_1, m_2, \dots, m_k\}.$$

Each model m_i has its own hyperparameter space Λ_i . The full conditional configuration space can be written as

$$\Lambda = \bigcup_{i=1}^k (\{\text{model_name} = m_i\} \times \Lambda_i).$$

This formulation means that each valid configuration consists of two parts: the selected model and the hyperparameters that are valid for that selected model. Therefore, the

optimizer searches only within the subspace associated with the chosen estimator, instead of searching over all possible hyperparameters at the same time.

Table 4.14: Example of conditional hyperparameter activation

Selected model	Active hyperparameters	Inactive hyperparameters
Random forest	number of trees, maximum depth, split criterion	number of neighbors, kernel type
K-nearest neighbors	number of neighbors, distance metric, weighting strategy	number of trees, maximum depth
Support-vector machine	kernel type, regularization parameter, gamma	number of neighbors, tree depth

Before constructing the final search space, the engine resolves the allowed model set. The user may provide this set explicitly when only specific algorithms should be considered. If no explicit set is provided, the engine starts from the default registry for the task. The engine can then remove incompatible models based on dataset properties. For example, some algorithms assume non-negative feature values and can be excluded when the training data contains negative values.

Restricting the search space has two main benefits. First, it reduces invalid trials because the optimizer is less likely to sample configurations that cannot be evaluated successfully. Second, it allows domain knowledge to be included without changing the optimizer itself. The optimizer only receives the final valid configuration space, while model compatibility and filtering rules are handled by the AutoML engine. This separation makes the system easier to extend, because new models can be added by registering their hyperparameters, conditions, and estimator builder without rewriting the search procedure.

4.6.3.3 Evaluation and Cost Conversion

After a configuration is sampled, the evaluator is responsible for estimating its predictive performance. For a configuration λ , the engine first builds the corresponding estimator and determines the preprocessing strategies that are allowed for the selected model. The evaluator then tests each model-preprocessor pair using cross-validation. Among all successful preprocessing choices, the pair with the best validation performance is retained as the evaluation result for λ .

Let $m(\lambda)$ denote the mean validation metric obtained by cross-validation. Since different metrics have different optimization directions, the evaluator converts the metric value into a cost value $c(\lambda)$. The optimizer always minimizes this cost, so smaller values indicate better configurations.

For metrics that are naturally minimized, such as mean squared error or mean absolute error, the cost can be used directly:

$$c(\lambda) = m(\lambda).$$

For metrics that are naturally maximized, such as accuracy, F_1 -score, or R^2 , the evaluator converts the score into a minimization objective. A common conversion is

$$c(\lambda) = 1 - m(\lambda),$$

when the metric is bounded between 0 and 1. For metrics that are not strictly bounded in this range, the evaluator may instead use a negated form:

$$c(\lambda) = -m(\lambda).$$

This conversion gives the optimizer a consistent objective regardless of the original metric. In all cases, minimizing $c(\lambda)$ corresponds to improving the validation performance of the configuration.

Table 4.15: Examples of metric-to-cost conversion

Metric	Original direction	Example cost conversion
Accuracy	Maximize	$c(\lambda) = 1 - m(\lambda)$
F_1 -score	Maximize	$c(\lambda) = 1 - m(\lambda)$
R^2 score	Maximize	$c(\lambda) = -m(\lambda)$
Mean squared error	Minimize	$c(\lambda) = m(\lambda)$
Mean absolute error	Minimize	$c(\lambda) = m(\lambda)$

If an evaluation fails, the evaluator does not allow the error to terminate the complete AutoML search. Instead, the failed configuration receives a predefined failure cost and the error message is stored in the run history. This allows the optimizer to continue exploring other configurations while still recording which trials were invalid or unsuccessful.

The validation procedure depends on the learning task. For classification, stratified cross-validation is used so that each fold preserves the class distribution as much as possible. Stratified subsampling is also used when only a fraction of the training data is evaluated at a lower fidelity. This is important because random subsampling may accidentally remove minority classes from small subsets.

For regression, ordinary K-fold cross-validation is used because there are no discrete class labels to preserve. Random subsampling is also sufficient for regression tasks. In classification, the requested number of folds may be adjusted when the smallest class does not contain enough samples to support the requested fold count. For example, if the smallest class contains only three samples, then using five stratified folds is not

valid. In this case, the classifier reduces the fold count to a feasible value. This prevents invalid cross-validation splits and improves the robustness of the evaluation process.

4.6.3.4 Three-Stage Fidelity Schedule

To reduce the cost of the AutoML search, the engine evaluates configurations using a three-stage fidelity schedule. Instead of evaluating every configuration using the full dataset, the full number of cross-validation folds, and the full model capacity, the system begins with a cheaper approximation. Only configurations that perform well at cheaper stages are promoted to more expensive stages.

The default fidelity schedule is shown in Table 4.16.

Table 4.16: Default evaluation fidelity stages

Stage	Data	Folds	Budget	Purpose
0	20%	2	25%	Inexpensive screening of many configurations.
1	50%	3	60%	Intermediate confirmation of promising configurations.
2	100%	Configured value	100%	Full-fidelity ranking and final comparison.

Let $s \in \{0, 1, 2\}$ denote the fidelity stage. Each stage is described by three values: the sample fraction ρ_s , the number of cross-validation folds k_s , and the model budget b_s . Therefore, the trial specification at stage s can be written as

$$\tau_s = (\rho_s, k_s, b_s).$$

The sample fraction controls how much of the training data is used during evaluation. The number of folds controls the reliability of the validation estimate. The model budget controls supported model-capacity parameters, such as the number of estimators in ensemble models or the number of training iterations in iterative models. Early stages are therefore cheaper because they use less data, fewer folds, and smaller model budgets.

The evaluator scales supported capacity parameters using the model budget. If θ is a capacity parameter and $\theta_{\max}$ is the value requested by the sampled configuration, the stage-specific value can be written as

$$\theta_s = \max(\theta_{\min}, \lfloor b_s \theta_{\max} \rfloor),$$

where $\theta_{\min}$ is the minimum valid value for that parameter. This keeps the reduced model valid while still preserving the relative capacity requested by the configuration.

A newly sampled configuration is first evaluated at stage zero. After a successful evaluation, the optimizer stores the observed cost in a per-configuration state. Once enough observations are available at a given stage, the optimizer ranks configurations according to their costs. Since the optimizer minimizes cost, smaller values are better. Only configurations inside the configured top promotion quantile are advanced to the next stage.

The promotion rule can be written as

$$\lambda \text{ is promoted from stage } s \iff c_s(\lambda) \leq Q_q(\{c_s(\lambda') : \lambda' \in \mathcal{C}_s\}),$$

where $c_s(\lambda)$ is the cost of configuration λ at stage s , $\mathcal{C}_s$ is the set of configurations evaluated at stage s , and Q_q is the cost threshold for the configured promotion quantile.

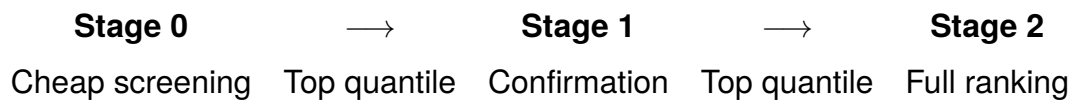


Figure 4.12: Simplified promotion flow in the three-stage fidelity schedule

The schedule spends the largest computational cost on configurations that were already competitive under cheaper approximations. This avoids applying full-fidelity evaluation to every sampled configuration and makes the search more efficient.

4.6.3.5 Surrogate-Guided Candidate Selection

The search space used by the AutoML engine contains categorical, continuous, integer, and conditional hyperparameters. Since the surrogate model requires numerical input, every sampled ConfigSpace configuration is first converted into a fixed-length numerical vector. Active hyperparameters are encoded using their numerical or encoded categorical values. Missing or inactive dimensions are replaced by finite sentinel values so that all configurations have the same vector length.

Let x_λ denote the vector representation of configuration λ . Since the same configuration can be evaluated at different fidelity levels, the vector is extended with the fidelity information. The augmented input vector is defined as

$$x' = [x_\lambda, s, q, k, b],$$

where s is the fidelity stage, q is the sample fraction, k is the number of cross-validation folds, and b is the model budget. Including these values allows the surrogate model to distinguish between cheap low-fidelity evaluations and expensive full-fidelity evaluations of the same configuration.

During the initial exploration phase, configurations are sampled without relying on the surrogate model. This provides the optimizer with a set of observed configuration–cost pairs. After enough successful evaluations have been collected, the optimizer fits a random-forest regressor using only finite observed costs. Failed evaluations or

non-finite results are excluded from the surrogate training data because they do not provide a reliable numerical target.

The random forest acts as an approximation of the unknown cost function. If tree t predicts the cost $\hat{c}_t(x)$ for candidate vector x , then the surrogate mean prediction is estimated as

$$\mu(x) = \frac{1}{T} \sum_{t=1}^T \hat{c}_t(x),$$

where T is the number of trees in the forest. The uncertainty estimate is computed from the disagreement between the trees:

$$\sigma(x) = \sqrt{\frac{1}{T} \sum_{t=1}^T (\hat{c}_t(x) - \mu(x))^2}.$$

A small value of $\sigma(x)$ means that the trees give similar predictions, while a larger value means that the trees disagree more strongly. Therefore, $\mu(x)$ estimates the expected cost of a candidate, and $\sigma(x)$ estimates the uncertainty of that prediction.

The optimizer then uses expected improvement to rank candidate configurations. Because the objective is to minimize cost, improvement is measured relative to the best observed cost c^+ . For a candidate x , the predicted improvement is

$$I(x) = c^+ - \mu(x) - \xi,$$

where ξ is a small exploration parameter. A larger value of ξ encourages the optimizer to prefer candidates that are more likely to produce a clear improvement rather than only a small predicted gain.

The standardized improvement is

$$z(x) = \frac{I(x)}{\sigma(x)}.$$

The expected improvement is then computed as

$$\text{EI}(x) = I(x)\Phi(z) + \sigma(x)\phi(z),$$

where Φ is the standard normal cumulative distribution function and ϕ is the standard normal probability density function. The first term rewards candidates with a low predicted cost, while the second term rewards candidates with higher predictive uncertainty. Therefore, expected improvement balances exploitation and exploration.

Table 4.17: Main quantities used in surrogate-guided candidate selection

Symbol	Meaning
x_λ	Numerical vector representation of configuration λ .
x'	Augmented vector containing configuration and fidelity information.
$\mu(x)$	Mean cost predicted by the random-forest surrogate.
$\sigma(x)$	Prediction uncertainty estimated from disagreement between trees.
c^+	Best observed cost so far.
ξ	Exploration parameter used in expected improvement.
$EI(x)$	Expected improvement score used to rank candidate configurations.

In each surrogate-guided step, the optimizer samples a pool of candidate configurations from the conditional search space. The candidates are converted to numerical vectors, duplicate vectors are removed, and the remaining unseen candidates are scored using expected improvement. The candidate with the highest expected improvement is scheduled for evaluation.

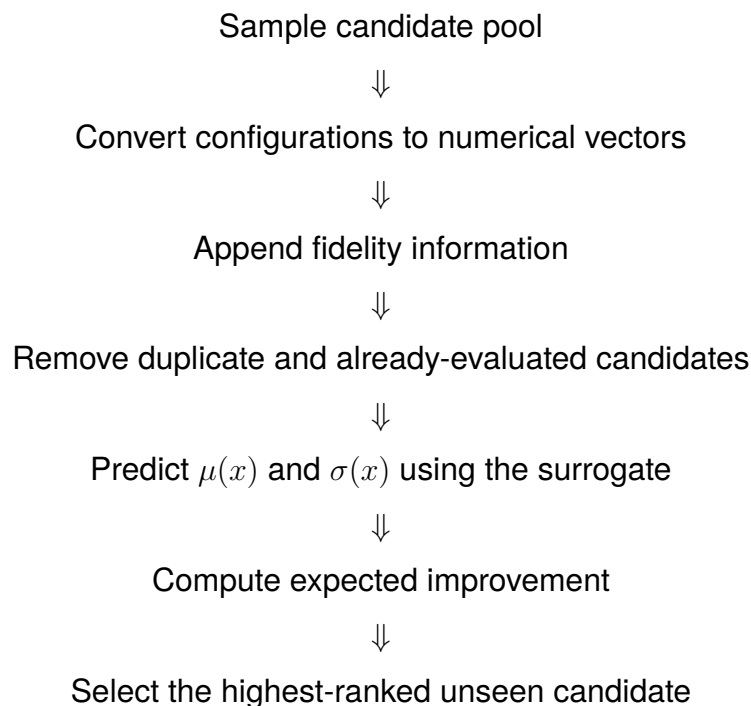


Figure 4.13: Surrogate-guided candidate-selection workflow

This procedure avoids exhaustive enumeration of the mixed conditional search space. Instead of evaluating every possible configuration, the optimizer uses the surrogate

model to focus the search on configurations that are either predicted to perform well or have enough uncertainty to be worth exploring.

4.6.3.6 Warm-Start Selection

Warm-start selection gives the optimizer an informed starting point by reusing configurations that performed well on related historical datasets. For a new dataset D , the engine first computes a task-specific metafeature vector m_D . Historical datasets are represented by stored metafeature vectors m_i , together with their previously successful configurations.

The similarity between the new dataset and each historical dataset is measured using a normalized distance. Let r be a scaling vector for the metafeature dimensions. The distance between the current dataset D and a historical dataset i is

$$d(D, i) = \left\| \frac{m_D - m_i}{r} \right\|_2.$$

The normalization prevents large-scale metafeatures, such as the number of samples or features, from dominating smaller-scale metafeatures. Historical datasets are then ranked by increasing distance, so datasets with smaller distances are considered more similar to the current task.

After ranking the historical datasets, the engine gathers candidate configurations from the nearest datasets. These configurations are not inserted blindly into the search. First, configurations whose model names are not present in the active model set are removed. This ensures that warm starts respect the current user constraints and dataset compatibility checks. Next, configurations are deduplicated using their model name and parameter signature, so repeated historical suggestions do not occupy multiple positions in the initial queue.

Each remaining historical configuration is then converted into a valid ConfigSpace configuration for the current search space. During this conversion, invalid or unsupported parameters are skipped safely. This can happen, for example, when the current version of a component does not contain a parameter that existed in a stored historical configuration, or when the current active space has stricter bounds. A failed conversion does not stop the search; it only removes that warm-start candidate.

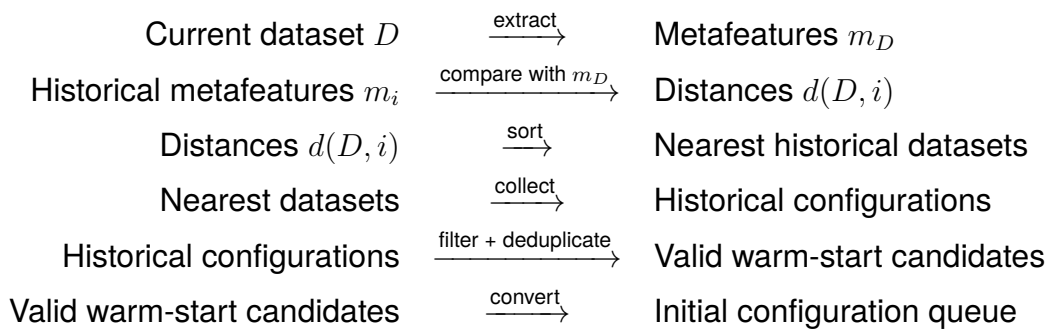


Figure 4.14: Warm-start transfer from similar historical datasets

The number of historical datasets used, the number of configurations taken from each dataset, and the total number of warm-start configurations are bounded by user parameters. These bounds keep the warm-start stage controlled and prevent historical suggestions from dominating the search.

Warm starts are inserted into the initial configuration queue before uninformed random samples. However, they are still evaluated on the current dataset using the same validation and cost-conversion procedure as all other configurations. Therefore, warm starts only guide the early search; they do not bypass evaluation and cannot directly force the final selected model.

4.6.3.7 Parallel Execution and Timeout Control

The optimizer may suggest a batch of trials in one scheduling step. A batch can contain promoted configurations from earlier fidelity stages, warm-start configurations from historical datasets, and newly proposed configurations selected either by the surrogate model or by random sampling. This batching allows the search to use multiple CPU cores while keeping the optimizer logic centralized in the parent process.

For parallel execution, each trial is executed in a separate worker process. The worker receives the configuration, builds the corresponding estimator, evaluates it, and returns an `EvaluationResult` through a process-safe queue. The parent process does not wait passively for all workers to finish. Instead, it periodically checks the worker queues, collects completed results, and monitors the elapsed time of each running process.

If a worker process exceeds the configured `per_run_time_limit`, the parent process terminates it and records the trial with a timeout status. The timeout result is stored in the run history in the same way as other failed evaluations, together with a failure cost. This ensures that one slow or non-terminating estimator does not block the complete AutoML search.

The timeout mechanism can be summarized as follows:

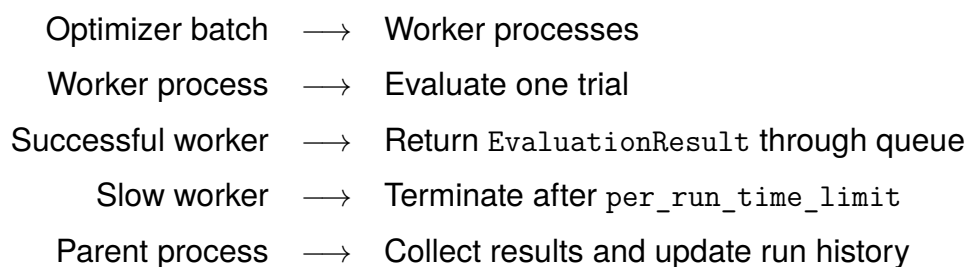


Figure 4.15: Process-level parallel execution and timeout handling

Process-level execution is used instead of thread-level execution because some estimators cannot always be interrupted reliably once they enter a long-running fit procedure. Terminating a separate process is more robust than attempting to stop a thread inside the same Python process. This is especially useful when an estimator

hangs, becomes extremely slow, or calls compiled code that does not regularly return control to Python.

Overall, this design improves robustness and resource control. The optimizer can continue receiving valid results even when some trials fail or time out, and the caller can control the amount of parallel work performed by the search.

4.6.3.8 Final Model and Ensemble Selection

After the search phase finishes, the engine selects the strongest configuration evaluated at full fidelity. This configuration is rebuilt from its stored model name, preprocessing choice, and hyperparameters, and then fitted on all available training data. This final refitting step is necessary because the search phase estimates performance using validation folds, while the final model should use the complete training set before being returned to the user.

If ensemble construction is disabled, the refitted best individual model is returned directly. If ensembles are enabled, the engine considers successful candidates from the run history as possible ensemble members. Failed trials, timed-out trials, and configurations without valid fitted estimators are ignored. This ensures that the ensemble is built only from models that were evaluated successfully during the search. The ensemble construction stage supports two strategies: voting and stacking. For voting, the engine selects a bounded set of competitive estimators from the run history. These estimators are combined using a task-specific voting model. For classification, the voting ensemble combines class predictions or class probabilities. For regression, it combines numeric predictions. Voting is simple and robust because it does not train an additional meta-model.

Stacking uses a more expressive structure. Instead of directly averaging the base-model outputs, it trains a meta-model that learns how to combine them. The engine first considers candidate diversity before selecting base estimators, so the stack is not formed only from many near-identical models. Each selected base estimator is wrapped in bagging to improve stability. The base estimators then produce out-of-fold predictions, which are used as training features for the meta-model.

Forward selection is used to choose the final set of base estimators for the stacking ensemble. The procedure starts with the strongest single candidate. Additional candidates are tested one at a time. A candidate is kept if adding it improves the validation score of the meta-model. The process stops when no remaining candidate improves the score, or when the configured ensemble size is reached. However, the minimum base-model requirement is also respected, so the engine can include enough base estimators to form a valid stack even if early improvements are small.

For classification, each base estimator may output a probability vector over the class labels. Before these outputs are passed to the meta-model, the probability columns are aligned according to the global class order. This avoids a common problem where different classifiers store class labels in different orders. If the columns were not

aligned, the meta-model could interpret the probability of one class as if it belonged to another class. After alignment, the probability blocks from all base estimators are concatenated and passed to logistic regression as the classification meta-model.

For regression, the stacking representation is simpler. Each base estimator contributes one numeric prediction column. These columns are concatenated to form the meta-feature matrix, and ridge regression is used as the regression meta-model. Ridge regression is suitable here because it can combine correlated base predictions while controlling the size of the learned coefficients.

The best individual model is always available as a safe fallback. The ensemble is only returned when it performs at least as well as this individual model under the same validation metric. The final decision can be written as

$$\text{Final model} = \begin{cases} \text{ensemble,} & \text{if } S_{\text{ens}} \geq S_{\text{best}}, \\ \text{best individual model,} & \text{otherwise,} \end{cases}$$

where S_{ens} is the validation score of the ensemble and S_{best} is the validation score of the strongest individual model. This rule prevents the system from returning a more complex ensemble unless it provides at least the same predictive quality as the best single model. As a result, ensemble construction can improve performance when the base models complement each other, while still preserving the simpler individual model when the ensemble does not provide a measurable benefit.

4.6.4 Modular Decomposition

The AutoML module is divided into cooperating components so that search, evaluation, model definitions, warm starts, and final model construction can be extended separately.

Table 4.18: AutoML implementation units

File or directory	Responsibility
<code>base.py</code>	Shared estimator lifecycle, validation, fitted state, leaderboard, and warning handling.
<code>classification.py</code>	Classification defaults and task-specific engine construction.
<code>regression.py</code>	Regression defaults and task-specific engine construction.
<code>core/automl.py</code>	Search orchestration, compatibility filtering, final model fitting, and ensemble selection.
<code>components/</code>	Model registries, hyperparameter definitions, conditional rules, and estimator factories.
<code>configspace_search_space.py</code>	Task-specific conditional configuration-space assembly and configuration conversion.
<code>evaluation.py</code>	Fidelity-aware estimator construction, preprocessing comparison, cross-validation, metric conversion, and failure handling.
<code>optimization/smac_optimizer.py</code>	Trial scheduling, surrogate modelling, expected improvement, fidelity promotion, parallel workers, and timeouts.
<code>meta_learning/</code>	Metafeature extraction, historical dataset similarity ranking, and warm-start retrieval.
<code>meta_learning_data/</code>	Packaged historical classification and regression metadata used for warm starts.
<code>stacked_ensemble.py</code>	Classification adapters, bagging, forward selection, and stacked classification.
<code>stacked_ensemble_regression.py</code>	Regression stacking and forward selection.

4.6.5 Design Constraints

- The current module targets supervised tabular classification and regression.
- Inputs must be compatible with Scikit-learn validation utilities.
- The evaluator assumes independent and identically distributed samples. Classification uses stratified splitting where possible, while regression uses ordinary K-fold or shuffle splitting.

4.6.6 Module Summary

The AutoML module provides an Accelera-integrated model-search layer for supervised tabular classification and regression. It combines conditional configuration-space construction, cross-validated evaluation, staged fidelity, surrogate-guided candidate selection, meta-learning warm starts, process-level timeout control, and final ensemble selection. The module keeps a familiar Scikit-learn-style interface while exposing useful inspection outputs such as the leaderboard and run history. This makes it usable as both a practical model selection tool and an extensible research component inside Accelera.

4.7 Deployment Module

4.7.1 Module Responsibility

The Deployment module is the part of Accelera that turns a trained machine-learning result into a usable software service. Without deployment, the output of a machine-learning workflow remains a local artifact, such as a `.pkl` file or a saved preprocessing object. These artifacts may be technically valuable, but they cannot serve real users until they are loaded by a backend application, validated against expected input formats, exposed through an API, and executed in an environment that can be reached by clients.

The module therefore completes Accelera's end-to-end workflow. It receives deployment inputs from the workspace, including `config.json` and model pickle files under `models/`. It can snapshot these files using a lightweight model VCS, prepare a FastAPI serving application, package the service inside Docker, and execute deployment workflows for local Docker, Heroku, and AWS EC2. At runtime, the generated service validates incoming requests using Great Expectations, executes model inference through `ModelService`, and records prediction telemetry through the request tracker.

Figure 4.16 summarizes this responsibility. The figure is important because it shows the Deployment module as a complete bridge between saved workspace artifacts and running deployment targets. The upper part of the flow represents local inputs and version control, the middle part represents service generation and serving behavior, and the lower part represents the targets on which the service can run.

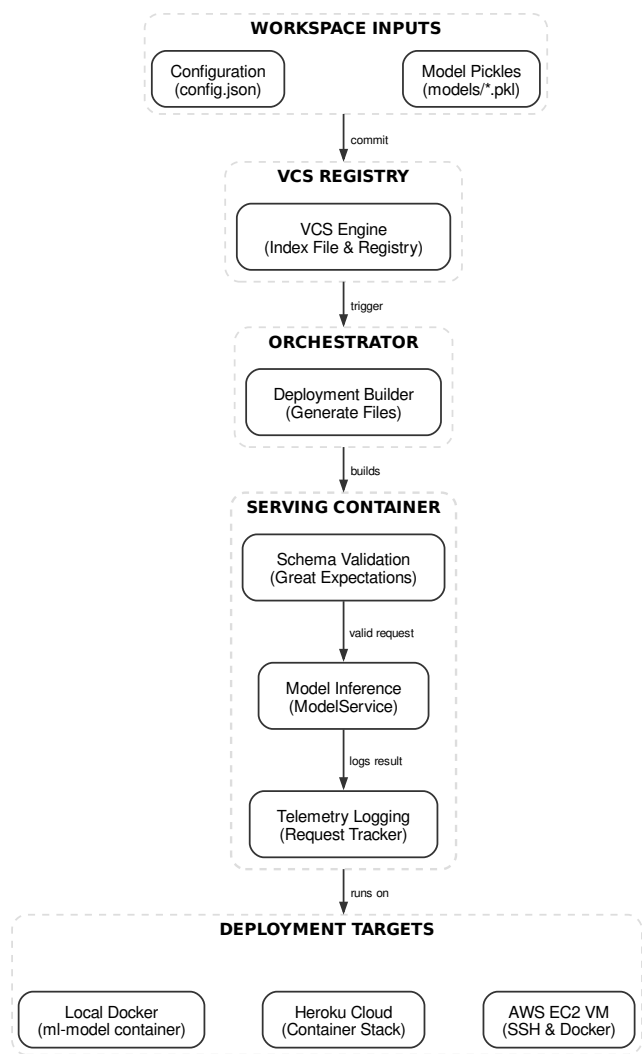


Figure 4.16: Deployment module overview from workspace artifacts to serving targets.

The core files under `accelera/src/deployment/` are listed in Table 4.19. This table connects the conceptual architecture with the actual source files in the repository and shows how each file owns a clear part of the deployment workflow.

Table 4.19: Deployment module implementation units.

File	Responsibility
<code>server.py</code>	Defines the FastAPI application, prediction endpoints, CSV inference endpoint, health endpoint, tracking summary endpoint, and interactive GUI route.
<code>modelservice.py</code>	Loads <code>config.json</code> , deserializes model and preprocessor artifacts, runs preprocessing transforms, and calls the final estimator's prediction method.
<code>schema_validation.py</code>	Validates request columns, required values, allowed values, numeric bounds, and type coercion using Great Expectations and local validation logic.
<code>tracking.py</code>	Records prediction activity, request status, row counts, latencies, and tracking summaries in line-separated JSON records.
<code>deployment.py</code>	Prepares deployment files, writes requirements, generates the Dockerfile, builds and runs local containers, and orchestrates Heroku and EC2 deployments.
<code>vcs.py</code>	Creates deployment commits, stores model/configuration snapshots, records the active HEAD, and restores historical snapshots for rollback.

4.7.2 Service Architecture

The serving subsystem is built around FastAPI, Uvicorn, `ModelService`, schema validation, and telemetry tracking. FastAPI receives HTTP requests, `ModelService` loads the configured model artifacts, the validation layer checks input quality before inference, and the tracker records each request outcome. This design gives the deployed model a clear runtime boundary: clients interact with API endpoints, while the model itself remains isolated behind validation and service logic.

Before presenting the internal stages, Figure 4.17 shows the serving architecture used inside the container. The diagram explains how requests flow from the HTTP layer to validation, model inference, and tracking. This separation is important because it prevents the model from being called directly with untrusted or malformed data.

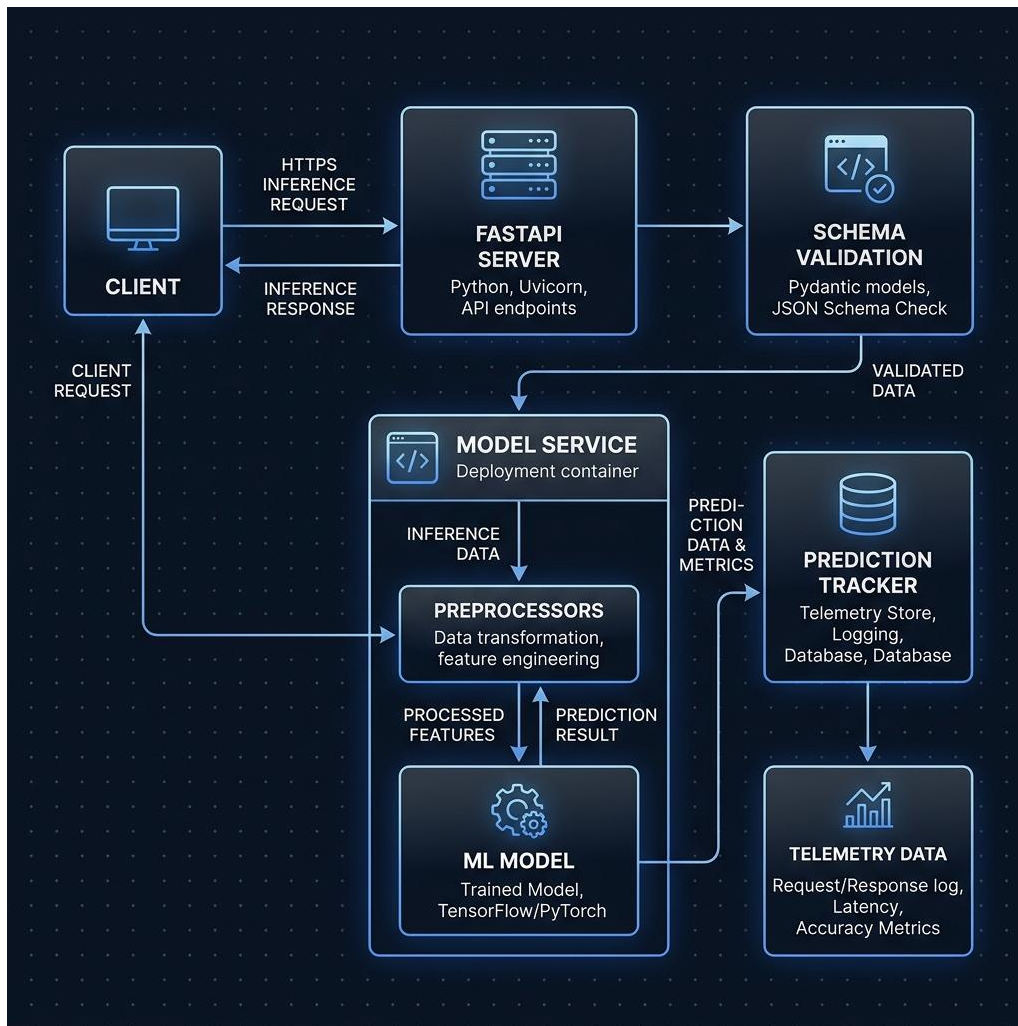


Figure 4.17: Model serving architecture and execution flow.

The service exposes several routes. The `/predict` endpoint accepts JSON payloads, while `/predict/csv` accepts uploaded CSV files for batch prediction. The `/health` endpoint reports whether the service is loaded, which is useful for local checks and remote deployment verification. The `/tracking/summary` endpoint exposes basic request-tracking information, and `/gui` renders an interactive form generated from the configured schema.

4.7.3 Configuration and Artifact Preparation

The deployment process begins from two concrete workspace inputs: `config.json` and model artifacts. The configuration file describes the models to load, the validation schema, and the tracking configuration. The model files are serialized Python objects, typically Scikit-learn estimators, preprocessors, or feature selectors stored as `.pkl` files. The module does not assume one fixed estimator type; instead, `ModelService` loads configured artifacts and treats the object with a `predict` method as the final model, while objects with `transform` methods are treated as preprocessors.

Algorithm 4.14 describes the preparation workflow. This algorithm matters because

deployment should not begin until the module confirms that every configured model path exists and that all runtime files are available inside the deployment workspace.

Algorithm 4.14: Preparing deployment artifacts

Input: workspace root, config.json, configured model paths

Output: deployment workspace with runtime files, models, requirements, and Dockerfile

```

1: Read config.json from the deployment workspace
2: Extract the models mapping from the configuration
3: For each configured model path do
4:     Resolve the path relative to the configuration file
5:     If the file does not exist, stop with a descriptive error
6:     Copy the artifact into the deployment models directory
7:     Rewrite the model path as a deployment-relative path
8: End for
9: Copy server.py, modelservice.py, schema_validation.py, tracking.py, and deployment
10: Synchronize the accelera package into the serving workspace
11: Write runtime requirements for FastAPI, Uvicorn, Great Expectations, Pandas, NumP
12: Generate a Dockerfile that copies the service, configuration, and model files
13: Return the prepared deployment workspace

```

After this algorithm completes, the local workspace has a repeatable service package. The important result is that the deployment engine no longer depends on scattered project files; it has a prepared runtime directory, a Dockerfile, dependency information, the active configuration, and the model artifacts needed by the service.

4.7.4 Runtime Validation, Inference, and Tracking

Runtime validation is a central part of the Deployment module because machine-learning models are sensitive to input shape, feature order, data type, and value ranges. A request may be syntactically valid JSON but still invalid for the trained model. The validation layer handles this by checking expected columns, missing values, unexpected fields, allowed values, and numeric bounds. It also coerces values into the expected data types where possible.

Algorithm 4.15 summarizes the prediction path followed by the serving application. The algorithm is written at the service level rather than as raw code because the key contribution is the sequence of validation, inference, and telemetry responsibilities.

Algorithm 4.15: Validating a request, running inference, and tracking the result

Input: API payload or uploaded CSV, active deployment configuration

Output: prediction response or structured API error

```

1: Receive the request at /predict or /predict/csv

```

- 2: Record the request start time
 - 3: Convert the payload into a tabular representation
 - 4: Validate required feature names and detect extra or missing features
 - 5: Coerce values to the configured types when possible
 - 6: Check numeric ranges and allowed categorical values
 - 7: If validation fails, record a `validation_error` event and return HTTP 422
 - 8: Convert validated rows into a NumPy-compatible array
 - 9: Apply loaded preprocessors in the configured order
 - 10: Call `predict` on the final model artifact
 - 11: Compute request latency in milliseconds
 - 12: Record endpoint, status, row count, latency, and predictions in telemetry
 - 13: Return row count and predictions to the client
-

The result is a deployed model service that is safer and more observable than a minimal prediction script. Invalid inputs are rejected with structured errors, successful predictions are returned through a consistent API response, and request activity is recorded for later inspection.

4.7.5 Model Version Control

The Deployment module includes a lightweight VCS dedicated to model deployment artifacts. This system is different from Git because it does not track all source-code changes. Instead, it snapshots the active `config.json` file and model files into an `experiments/` directory, assigns a short SHA-1 commit identifier, and records metadata in `experiments.json`. This allows a user to return to a previous deployed model configuration without manually copying files.

Figure 4.18 gives the high-level VCS structure. It shows the relationship between the active deployment workspace, the index file, the committed experiment directories, and the restored deployment state.

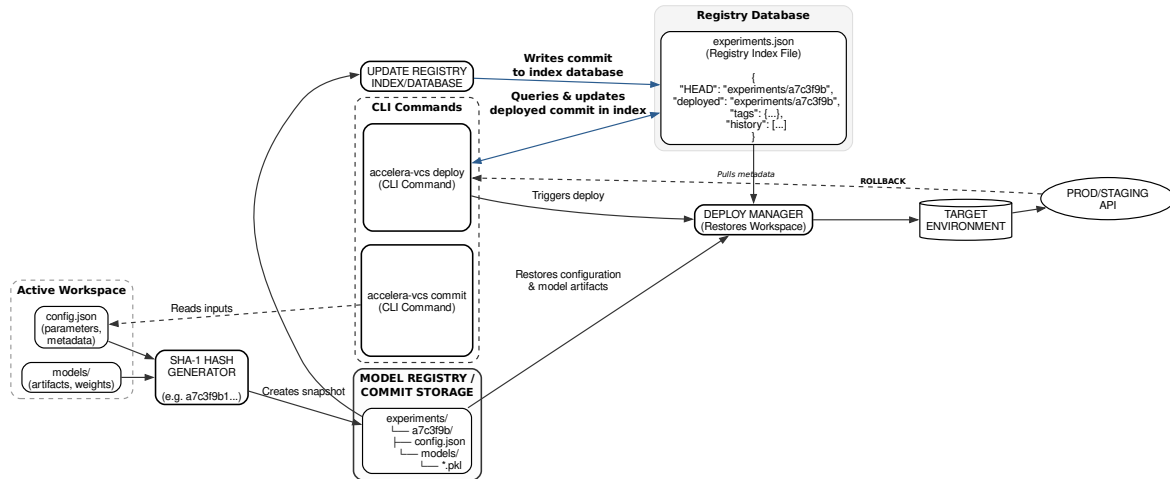


Figure 4.18: Deployment VCS registry structure.

The commit workflow is shown in Figure 4.19. Before this figure, it is useful to note that a deployment commit is not only a text label. It is a reproducibility unit containing both configuration and model binaries.

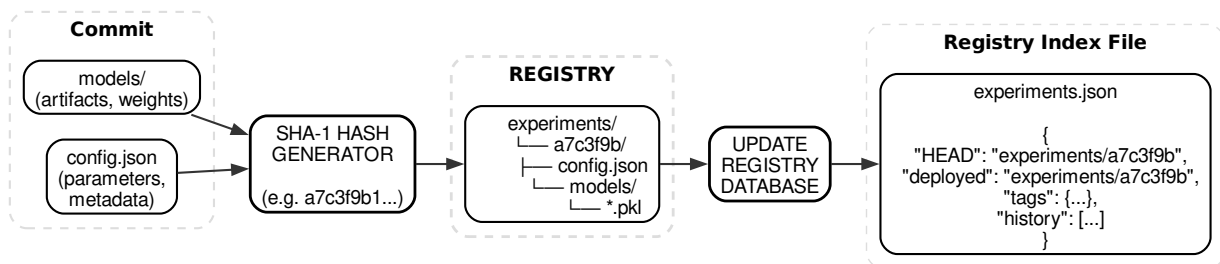


Figure 4.19: VCS commit workflow for deployment artifacts.

Algorithm 4.16 describes the commit process in pseudocode. This workflow supports reproducibility by ensuring that the model files referenced by a configuration are copied into the same commit directory as the configuration itself.

Algorithm 4.16: Creating a deployment VCS commit

Input: commit message, active config.json, active model files

Output: new deployment commit and updated experiments index

- 1: Require a non-empty commit message
- 2: Load and parse config.json
- 3: Resolve every model path listed in the models mapping
- 4: If any configured artifact is missing, stop the commit

- 5: Load `experiments.json`
- 6: Generate a short SHA-1 commit identifier from timestamp and message
- 7: Create `experiments/<commit-hash>/models/`
- 8: Copy model artifacts into the commit models directory
- 9: Rewrite model paths in the committed configuration as relative paths
- 10: Write `config.json` and `metadata.json` inside the commit directory
- 11: Append the commit record to `experiments.json`
- 12: Move HEAD to the new commit hash

After committing, the deployment workspace has an auditable model history. The history can be listed, inspected, and used to restore an earlier model snapshot. Figure 4.20 shows the deploy/rollback workflow that copies a selected snapshot back into the active runtime workspace.

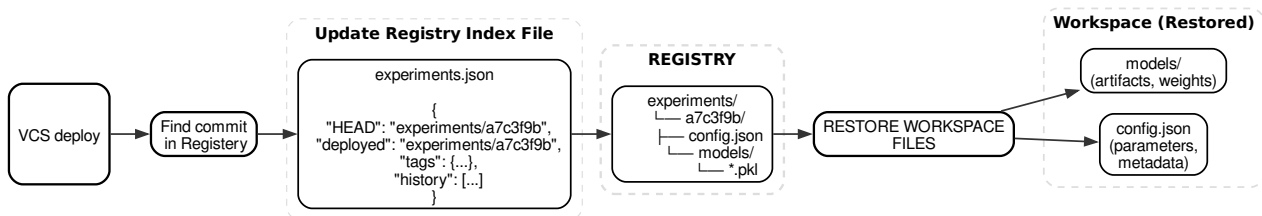


Figure 4.20: VCS deploy and rollback workflow.

4.7.6 Deployment Orchestration

The deployment orchestrator is responsible for converting the prepared service package into a running service. Its local path uses Docker to build and run the container on the developer machine. Its Heroku path uses the Heroku CLI and Container Registry. Its EC2 path uses SSH and `rsync` to synchronize files to a remote virtual machine, build the Docker image on the server, start the container, and verify the service through a health request.

Figure 4.21 presents the supported deployment targets. The figure is placed before the detailed algorithms because it helps the reader understand that the same prepared model service can be sent to different execution environments.

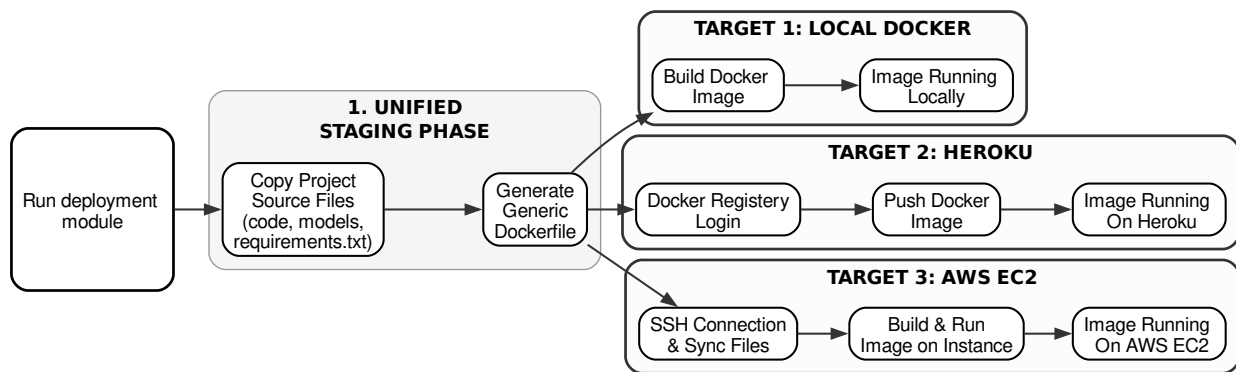


Figure 4.21: Multi-target deployment orchestration flow.

Algorithm 4.17 describes the local Docker workflow. This path is useful for testing the deployment service before moving it to a cloud or server environment.

Algorithm 4.17: Building and running the local deployment container

Input: prepared deployment workspace and target port

Output: running local Docker container

- 1: Prepare deployment files and Dockerfile
- 2: Build the Docker image named ml-model
- 3: Read the service port from the PORT environment variable or use 8000
- 4: Stop any currently running container that publishes the same port
- 5: Start the ml-model container with the selected port mapping
- 6: Print the local API URL and GUI URL
- 7: Keep the container process attached for local inspection

Once this workflow completes, the user can test the model through the local /predict, /health, or /gui routes on the selected port. This local path is especially valuable because it validates the complete serving package before a remote deployment is attempted.

Algorithm 4.18 describes the EC2 workflow. EC2 deployment is more complex because it involves a remote machine, SSH authentication, network ports, remote Docker execution, and health verification.

Algorithm 4.18: Deploying the model service to AWS EC2

Input: EC2 host, SSH user, optional private key, remote directory, port, image name,
 Output: running EC2-hosted deployment service or a diagnostic error

- 1: Prepare local deployment files and Dockerfile
 - 2: Build the SSH command using host, user, key, and host-key settings
 - 3: Create the remote deployment directory over SSH
 - 4: Synchronize Dockerfile, config.json, accelera_deployment, and models using rsync
 - 5: If requested, install and start Docker on the EC2 host
 - 6: Build the Docker image on the remote host
 - 7: Remove any old container with the target container name
 - 8: Start the new container with restart policy and port mapping
 - 9: Probe the container health endpoint from inside the host
 - 10: Print docker ps status, public application URL, and GUI URL
 - 11: Probe the public /health URL from the local machine
 - 12: If the public check fails, report that the TCP port may need to be opened
-

The result of the EC2 workflow is a public service URL when the instance, firewall rules, and Docker runtime are configured correctly. If the service is running internally but the public health check fails, the module gives a useful diagnostic message so the user can inspect security-group or port settings.

4.7.7 Command-Line Interface

The Deployment module exposes its behavior through command-line interfaces. This is appropriate for deployment because users often need to run the same operational steps repeatedly during testing, rebuilding, release, and rollback. Table 4.20 summarizes the orchestrator commands, while Table 4.21 summarizes the model VCS commands.

Table 4.20: Deployment orchestrator commands.

Command	Purpose
<code>prepare</code>	Copy serving files, synchronize the package, write requirements, validate model paths, and generate the Dockerfile.
<code>build</code>	Build the local Docker image, optionally without cache.
<code>run-local</code>	Start the local Docker container and publish the selected port.
<code>local</code>	Run <code>prepare</code> , <code>build</code> , and <code>run-local</code> as one local deployment workflow.
<code>heroku-create</code>	Create a Heroku app configured for the container stack.
<code>heroku-push</code> and <code>heroku-release</code>	Push the web container to Heroku and release it for traffic.
<code>heroku-deploy</code>	Execute the combined Heroku deployment sequence.
<code>ec2-deploy</code>	Synchronize files to EC2, build the container remotely, start the service, and verify health.
<code>ec2-stop</code> and <code>ec2-logs</code>	Stop the remote container or stream its logs for inspection.

Table 4.21: Deployment VCS commands.

Command	Purpose
<code>init</code>	Initialize the deployment experiment directory and empty <code>experiments.json</code> index.
<code>commit</code>	Snapshot the active <code>config.json</code> and model files under a short hash and commit message.
<code>status</code>	Display registry status, including total commits, HEAD, and deployed commit when available.
<code>log</code>	Print recorded commits in reverse chronological order.
<code>show</code>	Display a specific commit's metadata, configuration, and model files.
<code>deploy</code>	Restore a selected commit's configuration and model files into the active deployment workspace.

These commands make the module usable from scripts, demos, and repeatable terminal workflows. They also make the deployment process more transparent: users can run preparation, local testing, remote deployment, logging, stopping, committing,

and rollback as separate steps instead of treating deployment as a single hidden operation.

4.7.8 Technical Value of the Module

The Deployment module is valuable because it pushes Accelera beyond model construction. It gives the project a practical final stage where trained models become reachable services with validation, observability, versioning, and target-specific deployment workflows. This makes Accelera stronger than a normal training-only tool: it supports the path from data preparation and model selection to an API that can be tested through a browser, called by another application, monitored through tracking summaries, and moved between local and remote environments.

The design is intentionally lightweight. It does not claim to replace large enterprise MLOps platforms, Kubernetes serving systems, or managed cloud AI services. Instead, it provides a focused deployment layer that fits the scale of the project and connects directly to Accelera's generated artifacts. For a graduation project, this is a significant engineering contribution because it requires combining machine-learning artifact management, web serving, validation, telemetry, Docker packaging, cloud/server operations, and testing into one coherent module.

4.8 Benchmark Platform Module

The Benchmark Platform module is designed to provide a shared learning environment where users can create machine learning benchmarks, submit predicted results, compare submissions, and improve their techniques by studying other solutions. The module includes a backend API, database schemas, Python validation and scoring scripts, and a frontend interface for users and admins.

The module separates benchmark creation from submission evaluation. This makes the workflow clearer: first, the benchmark creator defines the task and evaluation rules; then, users submit predictions under the same conditions. This separation helps ensure that all users are evaluated using the same ground-truth file and the same metric configuration. The leaderboard and repository links also allow users to see which approaches performed better, learn from them, improve their own model, and submit again. Another important design point is the use of Python scripts for file validation and metric scoring. The backend is implemented using Node.js and Express, while Python handles the machine learning evaluation part through Scikit-learn metrics and dataset loading utilities.

4.8.1 Functional Description

The main function of this module is to support fair comparison and learning between different machine learning solutions. A user can create a benchmark by providing the benchmark title, description, problem type, target column, dataset link, test dataset link,

ground-truth target link, and evaluation metric. The system validates the provided links and checks that the test set does not contain the target column while the target file contains the expected target column.

After a benchmark is created, other users can submit their prediction file and repository link. The system validates the submitted prediction file, calculates the score using the selected Scikit-learn metric, stores the submission, and displays submissions on a leaderboard. The leaderboard is ordered based on the metric configuration, where some metrics are better when their value is higher and others are better when their value is lower. By checking the ranking and the repository links, users can learn from other solutions, modify their own approach, and submit improved predictions.

4.8.2 Modular Decomposition

The Benchmark Platform is decomposed into several smaller components. Each component has a clear responsibility, so the benchmark workflow can be easier to build, test, and maintain.

- **Backend Server and API Layer:** The backend creates the Express application, enables CORS and JSON parsing, connects to MongoDB, and mounts the main API routes. The backend exposes separate routes for users, metrics, benchmarks, and submissions. This makes the server organized because each group of operations is placed in its own route.
- **Database Models:** The module uses Mongoose schemas to store the main data. The user schema stores the user name, email, hashed password, and role. The benchmark schema stores the title, description, target column, dataset links, problem type, selected metric, metric parameters, creation date, and creator. The metric schema stores the display name, Scikit-learn metric function name, problem type, needed parameters, and whether higher or lower values are better. The submission schema stores the benchmark id, submitting user, repository link, prediction file link, submission date, and calculated score.
- **Benchmark Management:** This component handles creating, listing, filtering, viewing, and deleting benchmarks. Users can list all benchmarks, filter benchmarks by problem type, or show only benchmarks created by a specific user. When a benchmark is created, the backend checks the problem type, checks that the title is unique, validates the dataset links, and calls the Python validation script before saving the benchmark in MongoDB. Deletion is protected, so only the benchmark creator or an admin can delete a benchmark, and a benchmark cannot be deleted if it already has submissions.
- **Metric Management:** This component allows metrics to be listed, filtered by problem type, displayed, created, and deleted. Creating and deleting metrics is limited to admin users because a wrong metric can affect many benchmarks.

Each metric contains a display name, the matching Scikit-learn metric function name, the problem type, needed parameters, and the value direction using `higher` or `lower`. The delete route also checks if the metric is already used by a benchmark, so the system does not remove a metric that is still needed.

- **Submission Management:** This component allows users to submit prediction files for a specific benchmark and attach a repository link that shows their solution. Before saving a submission, the backend checks that the user did not already submit to the same benchmark, validates the links, loads the benchmark metric, and calls the Python scoring script. Users can also update their prediction file, which recalculates the score, or delete their submission if they are the owner or an admin.
- **Validation and Scoring Scripts:** The backend calls Python scripts for dataset validation and scoring. During benchmark creation, the validation script checks that the test file does not include the target column, that the target file includes the required target column, and that the test and target files have the same number of rows. During submission, the scoring script loads the hidden target file and the submitted prediction file, checks their length and target column, then computes the selected metric from `sklearn.metrics`. A helper function is also used to convert Google Drive file links into direct download links.
- **Authentication and Authorization:** The platform uses user accounts so benchmark and submission actions can be connected to their owners. The user routes handle signup and login, hash passwords, and return a JWT token. The authentication middleware reads the token and attaches the logged-in user to the request. The admin middleware is used for admin-only actions such as creating metrics and deleting metrics. These checks are also used to protect benchmark and submission deletion.
- **Leaderboard Interface:** The frontend leaderboard displays the submissions of one benchmark with the submitter name, repository link, score, and submission date. The backend sorts the submissions based on the selected metric direction, so the best score appears correctly whether the metric should be maximized or minimized. The same screen also lets a logged-in user add a new submission, update an existing prediction file, or delete a submission if the user has permission.
- **Frontend Benchmark and Metric Screens:** The React frontend provides screens for the main user actions. The benchmarks screen lists benchmarks and supports filtering by problem type or by the logged-in user. The create benchmark screen lets a user enter the title, description, target column, dataset links, problem type, metric, and metric parameters. The display benchmark screen shows benchmark details and links to the leaderboard. The metrics

screens allow users to view available metrics, while admins can create new metrics with their Scikit-learn function name, needed parameters, problem type, and score direction.

Algorithm 4.19: Benchmark creation flow

Input: benchmark form data

Output: saved benchmark or validation error

- 1: Receive benchmark creation request
 - 2: Validate problem type
 - 3: Check title uniqueness
 - 4: Validate dataset, test set, and target label links
 - 5: Run benchmark file validation
 - 6: If the test file contains the target column, reject benchmark
 - 7: If the target file does not contain the target column, reject benchmark
 - 8: Save benchmark document in MongoDB
-

Algorithm 4.20: Submission scoring flow

Input: benchmark target file, submitted prediction file, metric name

Output: score or validation error

- 1: Retrieve true target file
 - 2: Retrieve submitted prediction file
 - 3: Check that both files have the same number of rows
 - 4: Check that submitted file contains the target column
 - 5: Load metric function from sklearn.metrics
 - 6: Compute metric(true_target, predicted_target, metric_parameters)
 - 7: Return score to the backend as JSON
 - 8: Save submission with score
-

4.8.3 Design Constraints

Several constraints affected the design of the Benchmark Platform:

- **Task and data constraint:** The platform is limited to classification and regression benchmarks that use tabular files. This includes the training data, test data, prediction file, and target file.
- **Storage and access constraint:** Dataset and prediction files are handled through accessible links to reduce server storage usage.

- **Scoring constraint:** The platform uses Scikit-learn metrics for scoring. Therefore, the selected metric must be supported by Scikit-learn and must match the benchmark problem type.

4.8.4 Module Summary

The Benchmark Platform module provides a controlled environment for benchmark creation, prediction submission, validation, scoring, and leaderboard ranking. It also supports a learning cycle where users compare their results with others, review submitted repository links, improve their techniques, and submit updated predictions. It keeps the web management logic in the backend and frontend, while the metric calculation logic is handled by Python scoring scripts.

Chapter 5: System Testing and Verification

This chapter explains how the project was tested to make sure that the main parts work correctly. The testing was not done in one way only, because each part of the project has a different output. The main purpose of testing was to make sure that each part can receive the input, apply the required steps, save the needed files, and return the expected output without errors.

5.1 Testing Setup

The testing was done mainly in a Python development environment. The main tools used during testing were Python, Pytest, Pandas, NumPy, Scikit-learn, PyTorch, Matplotlib, and the other libraries used in the implementation. These tools were used to test data preprocessing, model training, prediction, evaluation, and report generation.

For the benchmark platform, the testing also included the web and backend tools. Node.js, Express, React, MongoDB, and Python scoring scripts were checked to make sure that submissions could be uploaded, processed, scored, and displayed correctly. This helped verify that the platform works as a complete web-based system, not only as separate frontend and backend files.

Some tests were written and executed automatically using Pytest. These tests were useful for checking repeated cases quickly and making sure that changes in the code did not break existing functionality. Other parts were tested manually, especially visual outputs such as generated reports, graphs, benchmark pages, and deployment responses. Manual testing was also used when the expected output needed to be inspected by the user.

5.2 Testing Plan and Strategy

The testing plan was based on testing the project step by step. First, each module was tested separately to make sure that its main function worked correctly. For example, the preprocessing module was tested by checking the generated cleaned data and reports, while the machine learning modules were tested by checking predictions, accuracy values, and saved models.

After testing individual modules, integration testing was performed. In this stage, the output of one module was passed to another module to check that the modules could work together. For example, preprocessing output was passed to machine learning models, generated reports were opened and inspected, and benchmark submissions were checked after scoring.

The testing strategy also included comparing actual outputs with expected outputs.

For machine learning parts, this included checking that the model produced valid predictions and reasonable metrics. For the benchmark platform, this included checking that submitted files were scored correctly and that the leaderboard was updated. For saved files and reports, the test checked that the files were created successfully and could be opened again.

Overall, the testing strategy focused on three main goals: checking that each module works alone, checking that connected modules work together, and checking that the final outputs are correct and usable.

5.2.1 Module Testing

5.2.1.1 Auto Preprocessing Module

Most of the preprocessing functions were tested using unit tests. These tests were used to check validation rules, artifact saving, and feature transformation. Real datasets were also used to test the system in a more practical way. These datasets were collected from Kaggle for tabular, text, and image data.

How the Auto Preprocessing testing works The Auto Preprocessing testing was done as a practical full-flow test. The idea was to run preprocessing on real datasets, then pass the processed output to a model. This means the test does not only check that preprocessing runs without errors, but also checks that the returned data can actually be used for training, validation, and prediction.

For tabular datasets, the test starts by loading each dataset with its target column and problem type. The preprocessing flow splits the data, fits the needed encoders and scalers, transforms the features and target, applies feature selection, and saves the preprocessing artifacts. After that, a machine learning model is trained on the processed training data and evaluated on the processed validation data. Classification datasets are evaluated using Macro F1-score, and regression datasets are evaluated using the R-squared score. The same datasets are also processed by a baseline automated cleaning tool, so the final scores can be compared and the Accelera output can be checked against an existing preprocessing approach.

For text datasets, the test checks that raw text can be converted into features that a classifier can understand. The preprocessing flow receives the text column and target column, cleans the text, tokenizes it, applies TF-IDF, encodes the labels, and returns processed training and validation data. A classifier is then trained on the transformed text features and evaluated using Macro F1-score. This confirms that the text preprocessing output is numeric, consistent, and usable for a normal machine learning model.

For image classification, the test checks the full image preparation flow. The preprocessing reads images from class folders, validates image files, skips corrupted images, creates class-to-label mappings, resizes the images, applies augmentation when enabled, and returns PyTorch DataLoaders. A neural network model is then

trained using these loaders. The inference path is also checked by loading the saved model and applying the testing preprocessing to new images. This verifies that the saved image size and class mapping can be reused during testing, not only during training.

The main concept in all three tests is to check preprocessing through the next step in the machine learning pipeline. If the tabular arrays, TF-IDF matrices, labels, image tensors, or saved artifacts were wrong, the model training or evaluation step would fail or give invalid results.

Table 5.1: Datasets Used in Module Testing

Dataset Name	Shape	Dataset Type	Problem Type
startup_founder_burnout_2026	(50000, 29)	Tabular	Classification
healthy_diet_calorie_intake	(6000, 15)	Tabular	Classification
main gaming_addiction	(250, 49)	Tabular	Classification
student_exam_performance_dataset	(10000, 17)	Tabular	Classification
heart	(1025, 14)	Tabular	Classification
Dataset_spine	(310, 7)	Tabular	Classification
diabetes_dataset	(100000, 31)	Tabular	Classification
student_placement_synthetic	(100000, 18)	Tabular	Classification
Titanic-Dataset	(891, 12)	Tabular	Classification
customer_purchase_data	(1500, 9)	Tabular	Classification
global_ai_jobs	(90000, 35)	Tabular	Regression
CarPrice_Assignment	(205, 26)	Tabular	Regression
Housing	(545, 13)	Tabular	Regression
PetImages	(24998,)	Image	Classification
DailyDialog	(11327, 2)	Text	Classification
TestReviews	(4321, 2)	Text	Classification

Table 5.2: Models and Evaluation Metrics Used in Module Testing

Dataset Name	Model Used	Evaluation Metric	Accelera Result
startup_founder_burnout_2026	RandomForest	Macro F1-score	0.9999
healthy_diet_calorie_intake	RandomForest	Macro F1-score	1.0000
gaming_addiction	RandomForest	Macro F1-score	1.0000
student_exam_performance_dataset	RandomForest	Macro F1-score	0.8573
heart	RandomForest	Macro F1-score	0.7860
Dataset_spine	RandomForest	Macro F1-score	0.7840
diabetes_dataset	RandomForest	Macro F1-score	0.9985
student_placement_synthetic	RandomForest	Macro F1-score	0.9986
Titanic-Dataset	RandomForest	Macro F1-score	0.7906
customer_purchase_data	RandomForest	Macro F1-score	0.9385
global_ai_jobs	RandomForest	R-squared Score	0.9227
CarPrice_Assignment	RandomForest	R-squared Score	0.9562
Housing	RandomForest	R-squared Score	0.6037
PetImages	ResNet model	Accuracy	0.9700
DailyDialog	XGBClassifier	Macro F1-score	0.9000
TestReviews	XGBClassifier	Macro F1-score	0.6080

These results show that the preprocessing output can be used by different models after transformation. The tabular results were strong in many datasets, especially the larger classification datasets and the car price regression dataset. The lower scores on datasets such as TestReviews show that preprocessing alone does not solve every modeling problem. Some datasets still need better features, model tuning, or more suitable models.

5.2.1.2 Accelera Pipe Module

Accelera Pipe was tested at node, graph, and persistence levels. The automated tests construct preprocessing, model, prediction, metric, merge, and branch nodes, then execute the resulting graphs in both serial and parallel modes. The tests compare transformed data, predictions, probabilities, and metric values with the corresponding Scikit-learn operations. They also exercise supervised and unsupervised workflows, mixed node types, and the `all`, `max`, `min`, and custom path-selection strategies.

Table 5.3 organizes these tests according to the graph behavior being verified. Each

test builds a small graph with a known expected result, executes it through the same public pipeline interface used by applications, and compares the returned values or graph state with that expectation.

Table 5.3: Accelera Pipe module-testing coverage.

Test area	Test procedure	Verified result
Node chains	Execute single and multiple preprocessors followed by model, prediction, and metric nodes.	Transformed values, predictions, probabilities, and metrics agree with the reference operations.
Branches and merges	Build preprocessing and model branches, merge predictions, and exercise hard voting and single-model merges.	Every branch receives the correct input and the merge returns the expected combined output.
Path selection	Execute the same candidate graph with <code>all</code> , <code>max</code> , <code>min</code> , and custom selectors.	Returned and retained paths follow the requested metric-selection rule.
Data lifetime	Share duplicate first branch nodes and execute multiple consumers of one transformed array.	Shared work is not repeated, and intermediate data is released only after its final consumer completes.
Persistence and reuse	Save and reload unexecuted pipelines and fitted executed graphs, including custom and lambda preprocessors.	Training graphs remain trainable, while loaded executed graphs reproduce inference results.
Failure handling	Supply invalid metrics, incompatible inputs, and unsupported node behavior.	The pipeline raises an explicit error associated with the node that failed instead of returning a partial result.

Branch-specific tests verify that equivalent first preprocessing nodes are shared without incorrectly merging later nodes whose inputs differ. Consumer-count tests ensure that an intermediate array remains available until its final dependent node finishes and is then released. Save-and-load tests train a graph, serialize the fitted executed graph, reload it, and confirm that inference produces the same result. Invalid model, metric, and graph inputs are also tested to confirm that failures are reported at the responsible node. The tests passed for the supported graph structures, showing that graph execution, branch sharing, intermediate lifetime management, and fitted-state persistence behave consistently.

Testing both serial and parallel graph execution is important because the mathematical result should not depend on scheduling. The same prediction tests are therefore executed with parallel execution enabled and disabled. Unsupervised tests additionally

verify that clustering models can execute without a supervised target, while mixed-type tests confirm that feature selection and preprocessing objects can be chained before fitting. Together, these tests cover the graph from construction to repeated inference rather than checking only individual Python wrapper methods.

5.2.1.3 Code Parallelizer Module

The Code Parallelizer was tested through its complete source-to-execution path. Test inputs include C/C++ files, C/C++ source strings, Python functions, and custom transformer methods. The tests first verify input dispatch and loop extraction, then check feature collection, classifier or rule-based decisions, OpenMP pragma generation, native compilation, and callable loading. Temporary output paths are supplied explicitly so file generation is isolated and can be verified on Linux and Windows.

Table 5.4 summarizes the principal testing layers. The test suite separates source transformation from native execution so that a failure can be located in input handling, loop analysis, pragma generation, conversion, compilation, or runtime behavior.

Table 5.4: Code Parallelizer module-testing coverage.

Test area	Test procedure	Verified result
Input modes	Provide C/C++ files, source strings, Python functions, and custom transformer methods.	Each input reaches the correct file-based or in-memory processing path.
Loop extraction	Analyze files containing one loop, multiple loops, nested loops, or no loops and serialize extracted features.	Loop boundaries and feature records match the source, including the empty-input case.
Decision safety	Test classifier responses, obvious independent writes, reductions, and unavailable classifier service.	Deterministic rules validate or replace predictions and provide a rule-based fallback.
Pragma generation	Generate directives for independent loops, reductions, and nested structures.	Valid directives are inserted once; unsafe reductions and redundant inner-loop directives are rejected.

Continued on the next page

Table 5.4: Code Parallelizer module-testing coverage (continued).

Test area	Test procedure	Verified result
Python conversion	Convert assignments, augmented assignments, powers, subscripts, ranges, function definitions, and supported helper calls.	The generated C++ preserves the supported Python operation and parameter names.
Native callable path	Compile supported NumPy loops, nested helpers, referenced helpers, and custom-transformer <code>transform</code> methods.	The loaded PyBind11 callable matches the original Python output and remains pickleable.
Fallback and cache	Pass unsupported syntax and repeat a supported compilation.	Unsupported functions retain their Python implementation, while repeated supported functions reuse cached artifacts.

The generated code is inspected for supported directives such as `parallel for` and reductions. Nested-loop tests confirm that selecting an outer loop does not also insert an unsafe directive into its inner loop. Callable tests compare the compiled result with the original Python result, while unsupported syntax is expected to return the original callable through the conservative fallback path. Repeated compilation tests also verify cache reuse. These checks showed that supported loops are transformed and executed correctly, whereas unsupported or unsafe cases remain sequential instead of silently changing program behavior.

Correctness is checked before performance. For compiled Python callables, the test invokes both the original function and the generated native callable with equivalent arrays and compares their outputs. File-based tests also confirm that the requested output file is created at the supplied path, which prevents tests and concurrent runs from writing to an implicit shared location. The fallback tests are equally important: a construct outside the converter subset must remain executable in Python rather than producing invalid C++ or an incorrect OpenMP transformation.

5.2.1.4 AutoML Module

The AutoML experiments compare Accelera AutoML with Auto-sklearn, FLAML, and TPOT on supervised tabular classification and regression tasks. The evaluation asks two practical questions: whether the systems attain comparable predictive quality, and how much wall-clock time they require to do so. Classification is evaluated primarily by accuracy, while regression is evaluated by the coefficient of determination (R^2), root mean squared error (RMSE), and mean absolute error (MAE). Runtime is reported in seconds for both task types.

The classification experiments were executed through the open-source AutoMLBenchmark (AMLB) runner, available at <https://github.com/openml/automlbenchmark.git>. They

use the `flaml_paper_39` classification suite and the `flaml_paper_1h` constraint. The reported classification evaluation covers 12 datasets. The regression benchmark was built from the Penn Machine Learning Benchmarks (PMLB) repository, available at <https://github.com/EpistasisLab/pmlb.git>, and was also executed through AMLB. The reported regression evaluation covers 10 datasets and uses the `pmlb_regression` suite with the `pmlb_30m` constraint.

All systems were run through the same benchmark interface. The recorded setup used one fold and one allocated core per task. The regression time budget was 1,800 seconds per dataset; the classification constraint allowed up to one hour.

5.2.1.5 Deployment Module

Experimental Setup and Test Suite

The Deployment module was verified using a focused `pytest` test suite that exercises the module as a serving, packaging, versioning, and orchestration component. The goal of this verification is not only to check individual functions, but also to confirm that the deployment workflow can move from saved artifacts and configuration files to a prepared API service in a repeatable manner.

The test environment is intentionally isolated from external cloud services. Temporary workspaces created with the `tmp_path` fixture are used to hold generated configuration files, model artifacts, VCS registries, Dockerfile outputs, and telemetry logs. The `monkeypatch` fixture replaces external dependencies such as subprocess calls, SSH execution, Heroku commands, and EC2 health checks. This allows the test suite to verify the correctness of the deployment logic without requiring a live Heroku account, a running EC2 instance, or active Docker containers during every test run.

The command used to execute the deployment test suite is shown below. The explicit `PYTHONPATH` value is important because the deployment tests load modules both through the package path and through the isolated serving runtime path.

Algorithm 5.1: Deployment test-suite execution command

```
PYTHONPATH=accelera/src/deployment:accelera/src \
  pytest accelera/src/deployment/tests
```

This setup verifies the Deployment module in a way that is suitable for headless execution and continuous integration. Instead of depending on the availability of external platforms, the tests check that Accelera generates the correct files, calls the correct commands, validates inputs correctly, records telemetry correctly, and preserves model snapshots correctly.

Verification Objectives

The Deployment module contains several responsibilities that must work together. Table 5.5 summarizes the main verification objectives used in the test suite. The table is included before the detailed results to show how each group of tests supports a concrete quality requirement of the module.

Table 5.5: Deployment module verification objectives.

Objective	Reason	Evidence in tests
Artifact reproducibility	The deployed service must be based on known model files and a known configuration.	VCS tests check commit creation, parent links, HEAD, deployed markers, and rollback restoration.
Serving correctness	The service must load models and preprocessors in the expected order before prediction.	Model service tests load serialized artifacts, verify idempotent loading, and check preprocessing before prediction.
Input reliability	Invalid payloads must be rejected before reaching the model.	Schema validation tests cover missing columns, unexpected columns, type coercion, booleans, integers, numeric ranges, and allowed values.
API behavior	The deployed service must expose predictable routes for prediction, health, GUI, CSV upload, and tracking.	Server tests verify successful responses, validation errors, internal errors, row counting, GUI rendering, and tracking summary behavior.
Operational repeatability	Deployment commands must produce correct Docker, Heroku, and EC2 command flows.	Orchestration tests inspect Docker build/run calls, Heroku sequencing, SSH transport, remote scripts, <code>rsync</code> , <code>stop</code> , and <code>log</code> commands.

Subsystem Verification Details

The test suite targets the major components of the Deployment module: model VCS, model loading, runtime validation, API serving, telemetry, and deployment orchestration. These components were tested separately because each one represents a failure point in a real deployment workflow.

- Model Version Control verification:** The VCS tests verify the stability of short SHA-1 commit identifiers, prefix resolution, index save/load behavior, workspace initialization, commit creation, parent tracking, log formatting, status reporting, and deployment rollback. The most important result is that committed model artifacts and configuration files can be restored into the active deployment workspace, which supports repeatable deployment and rollback.
- Model Service verification:** The `ModelService` tests verify that the service reads

`config.json`, loads serialized preprocessors and models, identifies the predict-capable artifact, and applies preprocessing before prediction. They also verify that loading is idempotent and that a clear runtime error is raised when no loaded artifact supports prediction.

3. **Runtime Schema Validation verification:** The schema tests verify both disabled and enabled validation modes. They confirm that missing columns and unexpected columns are detected, that numeric and integer values are coerced safely, that boolean representations are normalized, and that invalid values produce structured validation errors. These checks are important because the deployed API must protect the model from malformed inputs.
4. **Web Serving Endpoints verification:** The server tests verify the behavior of the FastAPI routes without requiring a live server process. They check that the root path redirects to the GUI, that `/health` and `/tracking/summary` report service state, that missing prediction input is rejected, that successful predictions record telemetry events, and that validation or internal errors map to the expected HTTP error behavior.
5. **Prediction Tracking verification:** The telemetry tests verify that a disabled tracker performs no writes, while an enabled tracker writes valid JSON lines and can summarize request counts, total rows, status counts, and the last recorded event. This confirms that the deployed service has a lightweight observability layer rather than only a prediction endpoint.
6. **Deployment Orchestration verification:** The orchestration tests verify the command-building and file-generation logic of `deployment.py`. They check valid and invalid ports, model path validation, service-source synchronization, generated `requirements.txt`, generated `Dockerfile`, Docker build and run commands, Heroku deployment order, SSH configuration, remote EC2 script generation, `rsync` synchronization, EC2 deployment flow, and remote stop/log operations.

Command-Flow Verification

In addition to unit-level behavior, the tests verify important command flows that a user would execute during deployment. This is significant because deployment failures often happen not inside model inference, but in the surrounding operational steps: building containers, selecting ports, synchronizing files, and starting services on remote hosts.

Table 5.6 summarizes the command flows covered by the orchestration tests. The table shows that the Deployment module was tested as an operational tool, not only as a collection of Python helper functions.

Table 5.6: Verified deployment command flows.

Flow	Verified behavior	Result
Local Docker build	The build command includes Docker build arguments, image naming, and optional cache bypass.	Command construction passed.
Local Docker run	The module checks for containers already publishing the selected port, stops conflicting containers, and starts a new mapped container.	Port-handling flow passed.
Heroku deployment	The complete deployment sequence runs login, optional app creation, container login, push, release, and open in the expected order.	Heroku orchestration passed.
EC2 remote deployment	The module prepares files, creates the remote directory, synchronizes build inputs, runs the remote deployment script, and checks the public URL.	EC2 orchestration passed.
EC2 operations	The stop and logs commands generate the expected remote Docker operations.	Operational commands passed.

Deployment Test Results

When the deployment test suite is executed in an environment with the required Python dependencies available, all 48 test cases pass. The result is a strong indication that the module's core responsibilities are implemented consistently: artifacts are versioned, models are loaded, inputs are validated, API behavior is predictable, telemetry is recorded, and deployment commands are generated correctly.

Table 5.7 presents the final test distribution. The table is organized by test file so that each subsystem can be traced back to its verification source in the repository.

Table 5.7: Deployment Module Test Suite Verification Summary

Test File	Cases	Passed	Failed	Verified Feature
vcs_test.py	16	16	0	Registry snapshots, HEAD, status, and rollback restoration
modelservice_test.py	5	5	0	Artifact loading, idempotent loading, and preprocessor sequencing
schema_validation_test.py	3	3	0	Schema description, missing-column checks, and unexpected-column reporting
server_test.py	8	8	0	FastAPI routes, prediction responses, GUI rendering, and telemetry events
tracking_test.py	3	3	0	JSONL telemetry writing and summary aggregation
deployment_test.py	13	13	0	Docker, Heroku, SSH, EC2, model-path, and generated-file workflows
Total	48	48	0	All deployment checks passed

The successful result confirms that the Deployment module operates as a reliable final stage for Accelera. It does not merely save model files; it validates the artifacts needed for serving, builds a deployable runtime package, protects inference through schema validation, exposes API behavior, records prediction activity, and verifies the command paths needed for local and remote deployment. This strengthens Accelera’s position as an end-to-end machine-learning system that continues beyond training and evaluation into practical service delivery.

5.2.1.6 Accelera Pipeline Reporting Module

The pipeline reporting part was tested by generating reports from executed pipeline results. The test checks that the report can read the serialized graph file, create the graph image, detect branches, show the selected path, and display metric results in the correct format.

Different metric result types were checked, such as single numbers, arrays, dictionaries, strings, tuples, and plotted figures. The expected output was a `report.html` file with readable tables and figures. The graph image was also checked manually to make sure that selected nodes and normal nodes were shown using different colors.

5.2.1.7 Benchmark Platform Module

The Benchmark Platform module was evaluated using manual functional testing to check its main features from the user side. The testing focused on creating benchmark tasks, validating dataset links, submitting prediction files, calculating scores, and displaying the results in the leaderboard.

The leaderboard was tested by submitting different results and checking whether they were ordered correctly based on the selected evaluation metric. This included metrics where higher values are better and metrics where lower values are better.

The scoring process was also tested manually by using different target and prediction files. The platform was checked to make sure it rejects invalid submissions, such as files with different row counts or missing target columns. These tests showed that the benchmark platform can handle result submission, calculate scores, validate files, and display leaderboard results correctly.

5.2.2 Integration Testing

Integration testing was done by connecting the output of one part with another part. For Auto Preprocessing, the transformed data was passed to machine learning models to check that the output was usable. For pipeline reporting, the report was generated after graph serialization and metric calculation. For benchmarking, the backend called the Python scoring script and then saved the returned score in MongoDB.

These checks were important because a function can pass a unit test but still fail when used with the rest of the system.

5.3 Testing Schedule

Testing was performed continuously during implementation and then repeated at module, integration, and project-release milestones. Table 5.8 summarizes the schedule followed across the complete project. A stage was not treated as complete until its automated checks passed and its generated outputs had been inspected where manual verification was required.

Table 5.8: Project-wide testing schedule.

Stage	Modules emphasized	Testing activity
During feature implementation	All modules	Run focused unit tests after each function, class, graph node, API route, or compiler rule is introduced or modified.
Module stabilization	Auto Preprocessing and AutoML	Validate cleaning, transformation, artifact persistence, model search, metrics, time limits, and classification and regression behavior on representative datasets.
Native-system stabilization	Accelera Pipe and Code Parallelizer	Test serial and parallel graphs, branch behavior, memory lifetime, persistence, source conversion, loop analysis, OpenMP generation, compilation, and fallback.
Service stabilization	Benchmark Platform and Deployment	Verify frontend and backend workflows, database operations, submission scoring, API validation, telemetry, model versioning, and local or remote deployment command generation.
Integration milestone	Connected project workflow	Pass preprocessing outputs to training and AutoML, execute and save graphs, compile custom preprocessing, generate reports, score submissions, and load trained artifacts for serving.

Continued on the next page

Table 5.8: Project-wide testing schedule (continued).

Stage	Modules emphasized	Testing activity
Regression and portability	Complete repository	Run the full Pytest and pre-commit suites after merges, rebuild native bindings, and check relevant Linux and Windows paths.
Final experimental evaluation	All evaluated modules	Freeze datasets and configurations, repeat measurements, verify output equivalence, produce tables and figures, and compare with the selected previous systems.

Manual verification followed the automated schedule for outputs that cannot be judged fully by assertions. This included preprocessing and pipeline reports, graph diagrams, benchmark pages, deployment responses, and generated figures. Comparative experiments were performed only after the corresponding module and integration tests were stable, which reduced the risk of reporting performance measurements for an incorrect workflow.

5.4 Comparative Results to Previous Work

5.4.1 Accelera Pipe Module

The Accelera Pipe experiments compare three implementations under the same train, validation, and test split. The first implementation is a manually written scikit-learn loop over the candidate pipelines. The second uses `sklearn.pipeline.Pipeline` to represent the same candidates. The third uses Accelera Pipe with graph branches. Each candidate is selected using validation accuracy, and the final selected path is evaluated on the test set. This protocol checks whether the graph execution improves latency without changing the selected model or the measured predictive accuracy.

Table 5.9: Accelera Pipe latency scaling in seconds

Samples	sklearn	sklearn Pipeline	Accelera Pipe
1,000	0.06	0.05	0.34
5,000	2.47	2.12	2.02
10,000	2.48	2.48	1.79
50,000	23.96	23.25	14.81
100,000	77.79	77.10	67.04
250,000	803.56	803.01	494.64

Table 5.9 shows the main performance trend without compressing the font. At very small sizes, Accelera Pipe is slower because the native graph setup, branch creation, and

Python/C++ boundary cost dominate the actual model computation. Once the dataset grows, those fixed costs become small compared with fitting multiple candidate branches. At 50,000 samples, Accelera Pipe reduces runtime from 23.96 seconds to 14.81 seconds. At 250,000 samples, it reduces runtime from about 803 seconds to about 495 seconds, saving more than five minutes while selecting the same candidate.

Table 5.10: Accelera Pipe RSS memory scaling in MB

Samples	sklearn	sklearn Pipeline	Accelera Pipe
1,000	1.62	0.16	19.32
5,000	3.62	1.09	21.62
10,000	11.47	2.19	7.25
50,000	126.95	9.75	209.25
100,000	139.70	47.65	300.93
250,000	303.95	28.52	594.86

Table 5.10 shows the current cost of graph-level parallelism. Accelera Pipe uses more RSS memory because multiple branch states and intermediate arrays can exist at the same time. The memory optimizations in the methodology section, especially consumer-count based release, duplicate first-node sharing, and guarded preprocessing copies, target exactly this issue.

Table 5.11: Selected candidate and accuracy across sample sizes

Samples	Selected candidate	Validation accuracy	Test accuracy
1,000	MinMaxScaler → SVC	0.7900	0.8250
5,000	MinMaxScaler → SVC	0.9160	0.9100
10,000	MinMaxScaler → SVC	0.9385	0.9260
50,000	MinMaxScaler → SVC	0.9565	0.9598
100,000	StandardScaler → SVC	0.9675	0.9709
250,000	StandardScaler → SVC	0.9774	0.9768

Table 5.11 confirms that the speedup does not come from changing the search space. Accelera Pipe evaluates the same candidate pipelines and reports the same validation and test behavior. The selected preprocessing strategy changes from MinMax scaling to standard scaling only when the larger dataset makes StandardScaler with SVC the best validation candidate. This is expected because the candidate set and selection rule are fixed while the data distribution becomes better represented at larger sizes.

Table 5.12: Largest Accelera Pipe comparison

Implementation	Time (s)	RSS MB	VMS MB	Test accuracy
sklearn manual loop	803.56	303.95	315.41	0.9768
sklearn Pipeline	803.01	28.52	28.61	0.9768
Accelera Pipe	494.64	594.86	787.47	0.9768

Table 5.12 isolates the largest run: 150,000 training samples, 50,000 validation samples, and 50,000 test samples. Accelera Pipe keeps exactly the same selected path, StandardScaler followed by SVC, and the same test accuracy. The runtime drop is 308.91 seconds relative to the manual scikit-learn loop and 308.36 seconds relative to scikit-learn Pipeline. This result is important because it demonstrates the value of graph scheduling on realistic branch-heavy workloads. The memory columns also show the next engineering target: scikit-learn Pipeline is extremely memory efficient, while Accelera Pipe trades memory for parallel branch execution and native graph state.

5.4.2 Code Parallelizer Module

The OpenMP classifier was evaluated as a three-class loop classifier. The labels are `none`, `parallel_for`, and `reduction`. The classification report shows that the model is balanced enough to guide the rule layer, while the final pragma decision remains protected by dependency checks.

Table 5.13: OpenMP classifier report

Class	Precision	Recall	F1-score	Support
none	0.82	0.85	0.84	3048
parallel_for	0.86	0.81	0.83	2696
reduction	0.79	0.82	0.80	775
accuracy			0.83	6519
macro avg	0.82	0.83	0.83	6519
weighted avg	0.83	0.83	0.83	6519

Table 5.13 should be read as a guidance-quality result, not as a proof that every predicted loop is safe. The classifier is responsible for identifying likely parallelization opportunities. The rule layer is responsible for rejecting unsafe or unsupported cases. This two-stage design is useful because source code parallelization has correctness constraints that pure statistical classification should not decide alone.

Table 5.14: Hard parallelizer evaluation

File	Baseline ms	Parallel ms	Speedup	Output match
hard_eval_000.cpp	1786.28	195.47	9.138×	true
hard_eval_001.cpp	63.98	31.47	2.033×	true
hard_eval_002.py	42.66	7.67	5.563×	true

Table 5.14 evaluates harder programs from `data/hard_test_files`. The benchmark compiles and runs both the baseline and the generated parallelized code, then compares their outputs. The first file obtains the largest improvement because its loop body has enough work to amortize OpenMP thread overhead. The second file still improves, but the smaller absolute runtime limits the maximum speedup. The Python-origin file demonstrates the full converter path: Python subset to C++, then loop extraction, then pragma insertion. Across the three files, the evaluation extracted 11 loops, generated 8 pragmas, matched all 8 expected pragmas, reached average similarity 1.0, and obtained an average speedup of 5.578×

Table 5.15: Linux parallelizer and Accelera Pipe integration benchmark

Mode	Samples	Time (s)
Python custom preprocessing function	500,000	6.83
End-to-end optimized run, first measured process	500,000	5.88
End-to-end optimized run, warmed method and module cache	500,000	0.08

Table 5.15 reports the Linux direct `examples/parallel_accpipe.py` run after normalizing the example execution directory to the repository root. The Python implementation takes 6.83 seconds on 500,000 samples. The first optimized process in the measurement sequence takes 5.88 seconds because it still pays process-local setup and cache lookup costs. The immediate repeated run takes 0.08 seconds after the Python-method cache and compiled-module cache are both warm. The validation in the example confirmed that the optimized output matched the Python output. This result shows why the integration needs two cache layers: one layer avoids repeating Python-to-C++ conversion and OpenMP insertion, while the lower compiled-module cache avoids recompiling the pybind extension.

Table 5.16: Linux direct example-script verification runs

Example	Baseline time (s)	Optimized time (s)	Validation
accpipe_demo	2.48	1.79	same accuracy, 0.9260
parallel_accpipe, run 1	6.83	5.88	outputs match
parallel_accpipe, run 2	6.83	0.08	outputs match
code_parallelizer, run 1	5.81	0.014	outputs match
code_parallelizer, run 2	6.05	0.015	outputs match
save_load, run 1	1.89	2.69 / 2.65	same accuracy, 0.268
save_load, run 2	2.53	2.20 / 2.14	same accuracy, 0.268

Table 5.16 summarizes the Linux examples invoked by `examples/Makefile`, but each file was run directly rather than through Make. Cache-sensitive files were repeated. The standalone `code_parallelizer_demo.py` result measures Python script execution against the generated OpenMP executable for `hard_eval_002.py`; the C path also reported about 0.26–0.27 seconds of compilation time, outside the listed executable runtime. The save/load example verifies persistence rather than pure acceleration: both sklearn and Accelera load persisted state and produce the same accuracy, while the second Accelera run is slightly faster than the sklearn baseline.

Table 5.17: Windows direct example correctness and path timing

Example	Run/cache state	Baseline (s)	Accelera/C path (s)	Validation
accpipe_demo	normal	1.75 / 1.78	1.03	same test accuracy, 0.9260
parallel_accpipe	isolated cache, run 1	6.66	6.00	outputs match
parallel_accpipe	isolated cache, run 2	6.58	0.09	outputs match
code_parallelizer	isolated cache, run 1	5.59	1.04	outputs match
code_parallelizer	isolated cache, run 2	5.61	1.05	outputs match
save_load	normal	0.57	0.51 / 0.53	same accuracy, 0.268

Table 5.18: Windows compilation and process overhead

Example	Run/cache state	Compile/link (s)	Wall time (s)	Interpretation
accpipe_demo	normal	–	6.53	graph example startup
parallel_accpipe	isolated cache, run 1	included	14.37	cold compile path
parallel_accpipe	isolated cache, run 2	cached	8.33	warm cache path
code_parallelizer	isolated cache, run 1	0.87	8.74	temporary executable
code_parallelizer	isolated cache, run 2	1.04	9.00	temporary executable
save_load	normal	–	2.71	persistence example

Tables 5.17 and 5.18 report the same Makefile example set on Windows. The first table keeps the user-visible comparison: baseline runtime, optimized path runtime, and validation result. The second table separates compilation and process-level overhead so the timing table

remains readable. The examples were run one by one in Makefile order. The cache-sensitive examples were then repeated under an isolated Accelera cache so that the first run represents cold cache behavior and the second run represents warm cache behavior. The strongest cache effect appears in `parallel_accpipe.py`: the cold end-to-end Accelera path takes 6.00 seconds because it must convert the function, select loops, compile a pybind extension, link it, and load it into Python. The immediate warm-cache run takes 0.09 seconds because the processed-code cache, method cache, and compiled-module cache avoid repeating those setup stages.

The Windows `code_parallelizer_demo.py` runs show a different limitation. The generated C/OpenMP executable is compiled into a temporary directory on every run, so the second run does not reuse the final linked program. The measured C executable subprocess takes about 1.04 seconds, while the instrumented main-body timer inside the generated program reports only about 0.024–0.025 seconds. This gap is Windows process startup, OpenMP runtime initialization, dynamic library loading, and executable loading overhead. The compile/link step itself adds about 0.87–1.04 seconds. On Linux, the same benchmark has lower launch and linking overhead in the reported measurements, so executable runtime is closer to the timed main-body work. For this reason, Windows results should be interpreted as end-to-end toolchain latency results, not only as loop-kernel speed results.

5.4.3 Auto Preprocessing

Auto Preprocessing was compared with AutoCleanML to check the performance of the proposed preprocessing approach. The comparison was done using several tabular datasets collected from Kaggle. The same datasets and the same machine learning models were used for both tools. The comparison was applied only on tabular datasets because AutoCleanML mainly works with tabular data. For classification datasets, the Macro F1-score was used as the evaluation metric. For regression datasets, the R-squared score was used to measure how well the model predicts the target values. After preprocessing the datasets, the output data was tested using four machine learning models. For classification problems, Logistic Regression, Random Forest, Decision Tree, and KNN were used. For regression problems, Linear Regression, Random Forest, Decision Tree, and KNN were used. This helped test the preprocessing output with different model types, not only one model. The results were saved and plotted as comparison graphs between Accelera and AutoCleanML. These graphs show the score of each tool on the selected datasets. The purpose of this comparison was to check whether Accelera can produce preprocessing results that are close to, or better than, an existing preprocessing tool.

Table 5.19: Classification comparison using Logistic Regression and Random Forest

Dataset	Logistic Regression		Random Forest	
	AutoCleanML	Accelera	AutoCleanML	Accelera
Startup founder burnout	1.0000	1.0000	0.9999	0.9999
Healthy diet calorie intake	0.9917	0.9890	1.0000	1.0000
Gaming addiction	0.8937	0.8937	0.9646	1.0000
Student exam performance	0.8667	0.8657	0.8579	0.8573
Heart	0.8019	0.8007	0.7689	0.7860
Spine dataset	0.8691	0.7943	0.8360	0.7840
Diabetes dataset	0.9995	0.9985	0.9996	0.9985
Student placement synthetic	0.5657	0.5879	0.6241	0.9986
Titanic dataset	0.7734	0.7864	0.7906	0.7906
Customer purchase data	0.8484	0.8375	0.9386	0.9385

Table 5.20: Classification comparison using Decision Tree and KNN

Dataset	Decision Tree		KNN	
	AutoCleanML	Accelera	AutoCleanML	Accelera
Startup founder burnout	1.0000	1.0000	0.9824	0.9881
Healthy diet calorie intake	1.0000	1.0000	0.9379	0.9184
Gaming addiction	1.0000	1.0000	0.6905	0.8512
Student exam performance	0.7987	0.8063	0.8423	0.8338
Heart	0.7213	0.7704	0.7689	0.7860
Spine dataset	0.7686	0.7632	0.8239	0.8042
Diabetes dataset	0.9995	0.9972	0.9925	0.9980
Student placement synthetic	0.5827	0.9968	0.5995	0.8348
Titanic dataset	0.6979	0.7877	0.7203	0.7562
Customer purchase data	0.8486	0.8449	0.8544	0.8806

Table 5.21: Regression comparison using Linear Regression and Random Forest

Dataset	Linear Regression		Random Forest	
	AutoCleanML	Accelera	AutoCleanML	Accelera
Global AI jobs	0.9022	0.7498	0.9288	0.9227
Car price assignment	0.8886	0.8943	0.9535	0.9562
Housing	0.6506	0.6482	0.6087	0.6037

Table 5.22: Regression comparison using Decision Tree and KNN

Dataset	Decision Tree		KNN	
	AutoCleanML	Accelera	AutoCleanML	Accelera
Global AI jobs	0.8640	0.8412	0.7586	0.7510
Car price assignment	0.9269	0.8825	0.7905	0.7478
Housing	0.3819	0.3634	0.5312	0.5336

From the classification and regression results, Accelera preprocessing achieved competitive performance compared to AutoCleanML. The comparison was tested using different types of models, including linear models, tree based models, Random Forest, and KNN. In many cases, the results of Accelera were very close to AutoCleanML, and in some cases Accelera achieved better scores.

These results show that the implemented preprocessing steps in Accelera are able to prepare data in a valid way for different machine learning techniques. The performance was not limited to one specific model type, but appeared across several models and datasets. This means that Accelera successfully reached results close to an existing automated preprocessing tool, while still using its own preprocessing approach.

However, the results also show that no preprocessing tool gives the best score in all cases. The final performance can still depend on the dataset, the problem type, and the model used after preprocessing.

5.4.4 AutoML Module

Comparative Results: Classification Table 5.23 summarizes the classification results stored with the project. Auto-sklearn obtained the highest mean accuracy at 0.8842. Accelera reached a mean accuracy of 0.8791, a difference of about 0.0051, while recording the lowest mean duration at 1,201.7 seconds. Its mean runtime was approximately one third of the runtimes recorded for the other three systems. Accelera also had the largest number of recorded speed wins.

Table 5.23: Classification benchmark summary.

System	Mean accuracy	Mean time (s)	Accuracy wins	Speed wins
Auto-sklearn	0.884178	3615.71	3	3
FLAML	0.881427	3655.63	6	0
Accelera	0.879076	1201.72	6	15
TPOT	0.877654	3953.78	8	1

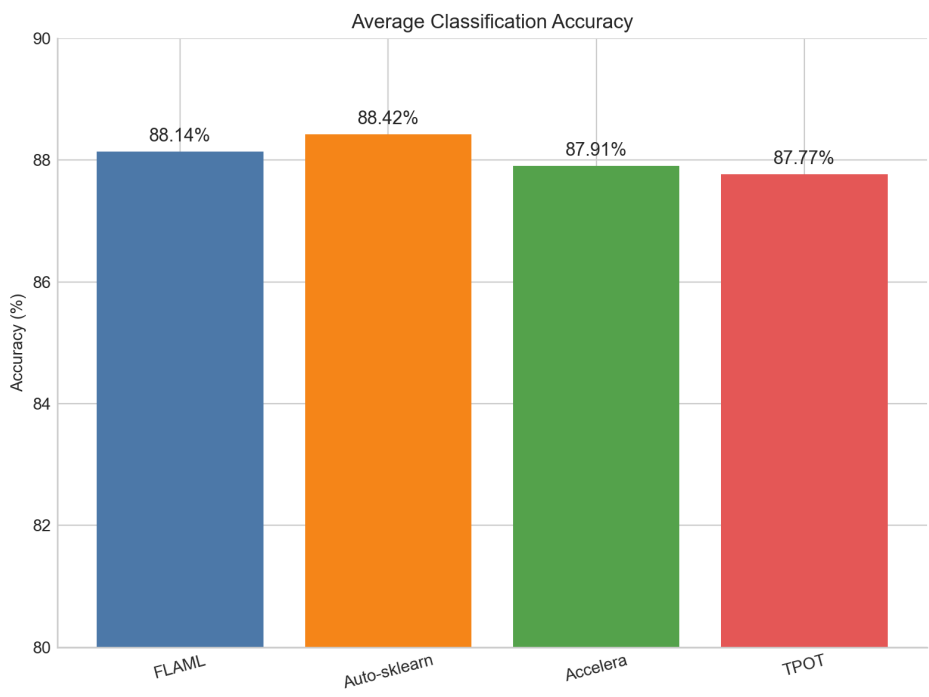


Figure 5.1: Mean classification accuracy for the evaluated AutoML systems.

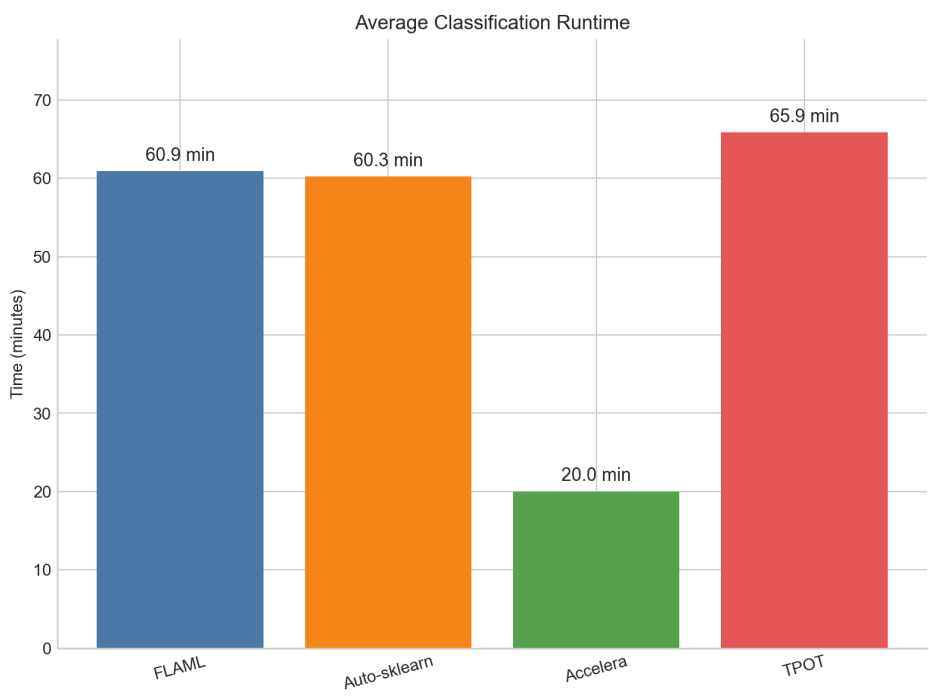


Figure 5.2: Mean classification runtime for the evaluated AutoML systems.

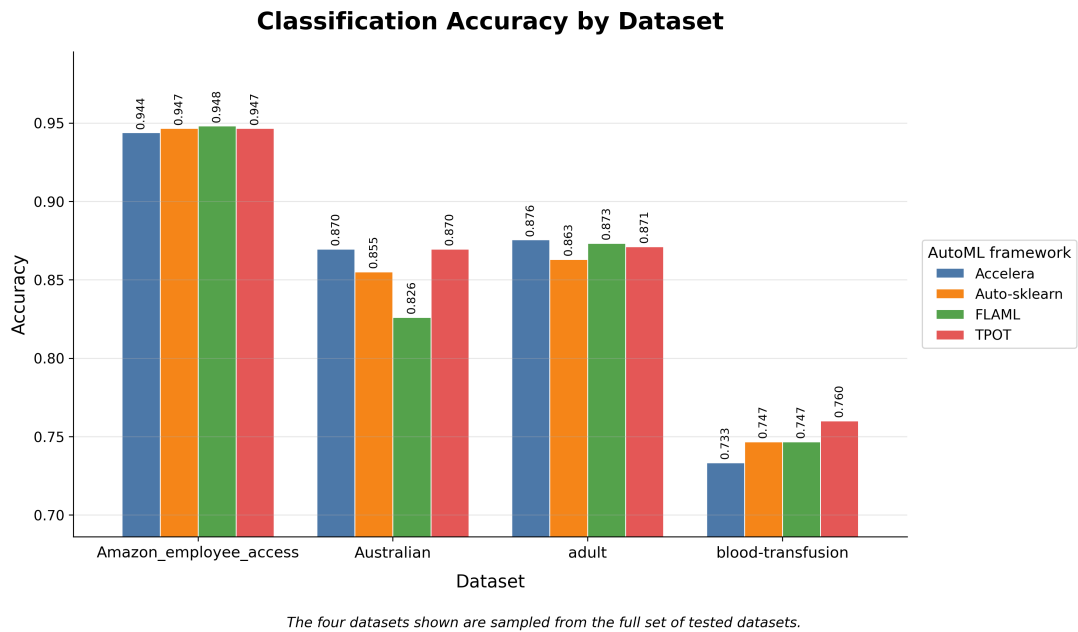


Figure 5.3: Per-dataset classification accuracy on common datasets.

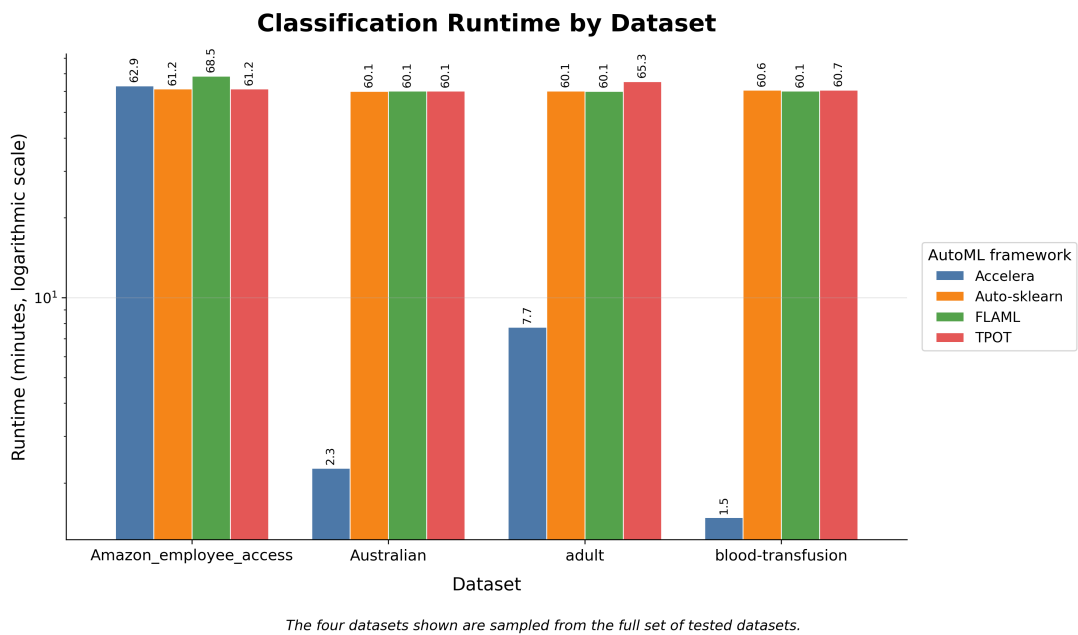


Figure 5.4: Per-dataset classification duration on common datasets.

Comparative Results: Regression Table 5.24 reports the updated regression summary over ten datasets. Auto-sklearn obtained the highest mean R^2 at 0.5609, followed closely by TPOT at 0.5592. Accelera reached 0.5294 and recorded the lowest mean runtime at 892.64 seconds, compared with 1,820–1,864 seconds for the competing systems. Accelera also achieved six speed wins, the most of any evaluated system.

Table 5.24: Updated regression benchmark summary over ten datasets.

System	Mean R^2	Median R^2	Mean time (s)	R^2 wins	RMSE wins	MAE wins	Speed wins
Auto-sklearn	0.560947	0.494362	1820.18	2	2	2	2
TPOT	0.559238	0.493144	1849.70	2	2	2	0
Accelera	0.529389	0.432577	892.64	2	2	2	6
FLAML	0.522743	0.497530	1863.85	4	4	4	2

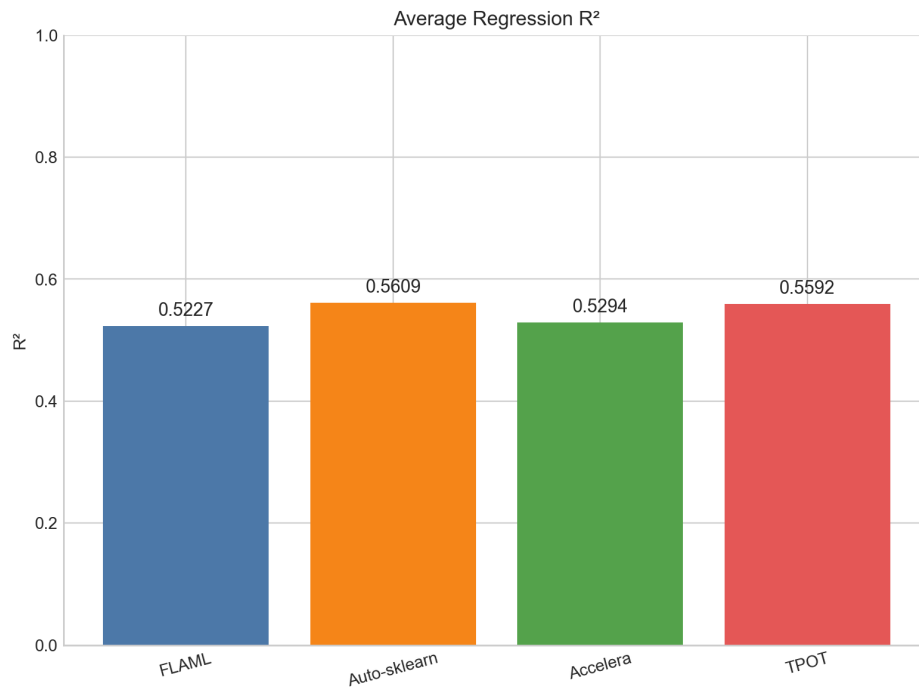


Figure 5.5: Mean regression R^2 for the evaluated AutoML systems.

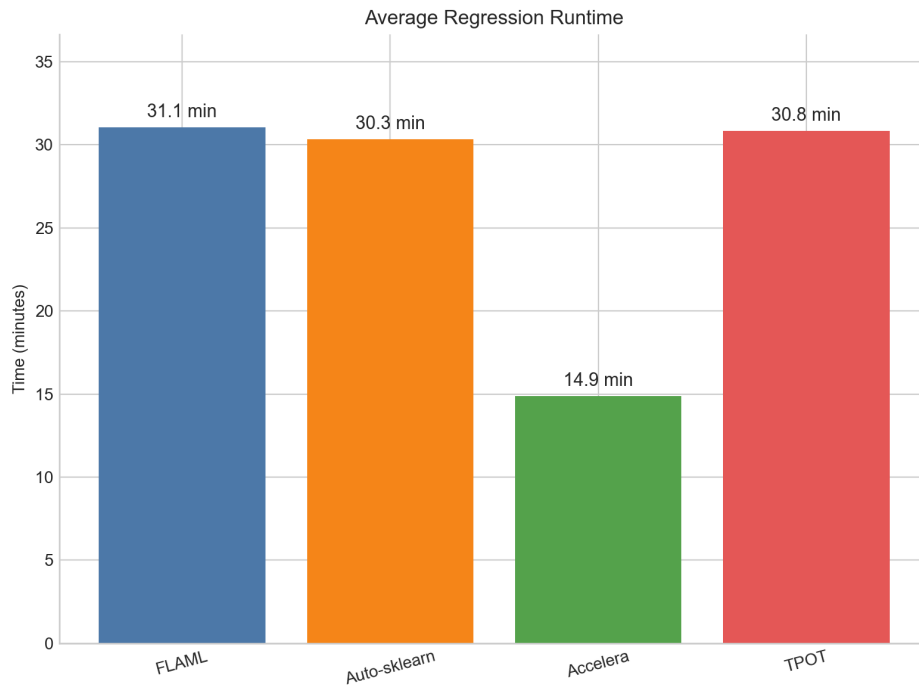
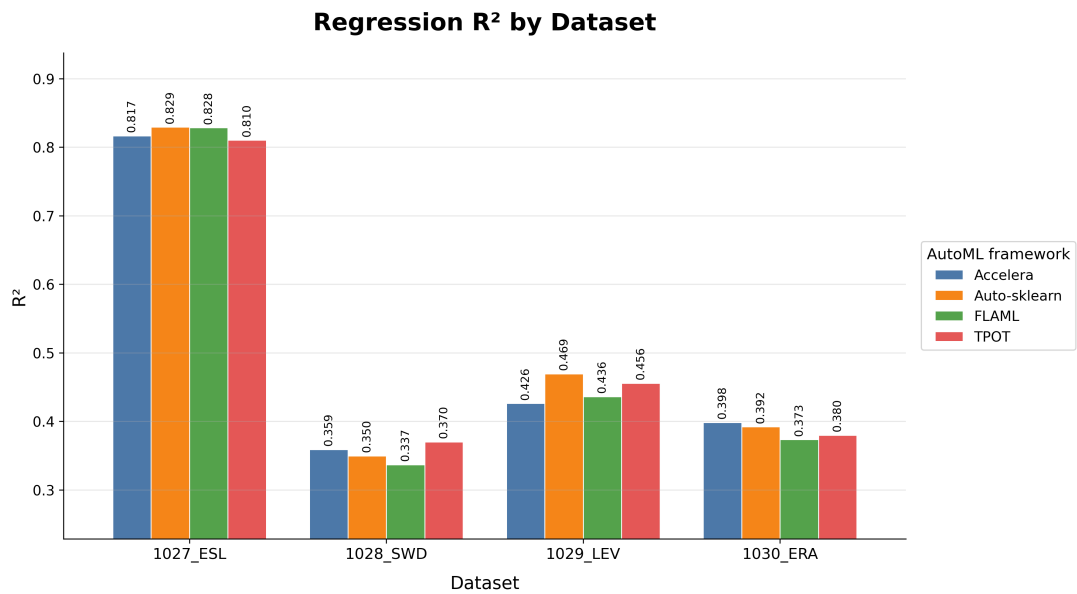


Figure 5.6: Mean regression runtime for the evaluated AutoML systems.



The four datasets shown are sampled from the full set of tested datasets.

Figure 5.7: Per-dataset regression R^2 on common datasets.

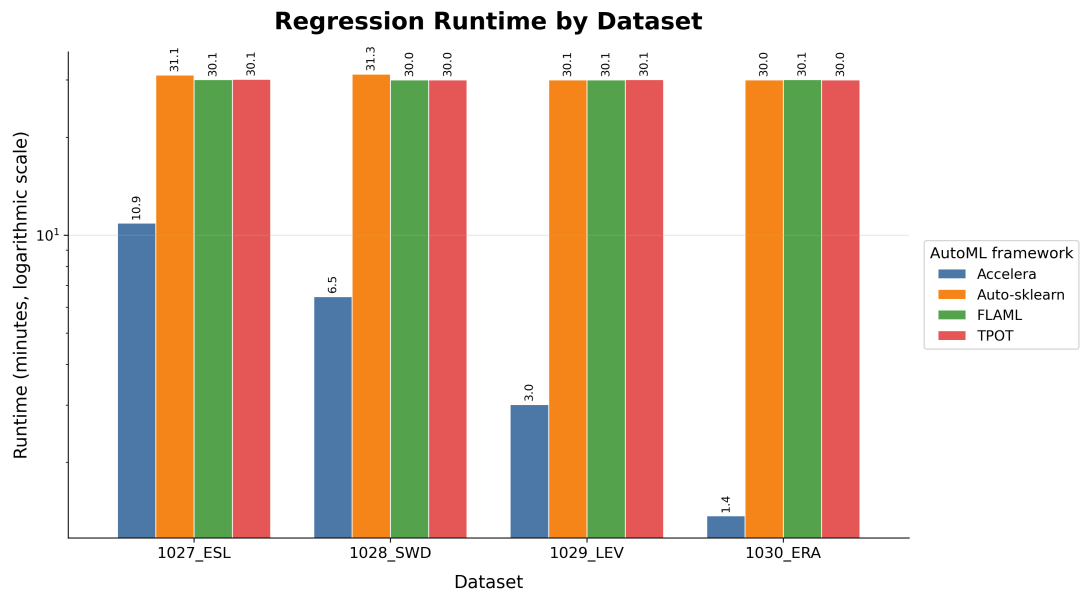


Figure 5.8: Per-dataset regression duration on common datasets.

Result Reproduction The benchmark experiments can be reproduced using the instructions provided in the project repository. These instructions describe the required environment setup and the commands used to rerun the classification and regression benchmarks. The generated figures, CSV summaries, and raw benchmark outputs are stored with the project results, allowing the reported values to be checked against future reruns. The reproduction guide is available at `setup/reproduce_results.md`.

AutoML Result Summary Across the stored experiments, Accelera achieved predictive results close to the strongest competing framework while requiring markedly less mean runtime. The classification results show a small accuracy difference relative to Auto-sklearn, and the regression results show a modest R^2 difference relative to Auto-sklearn, accompanied in both cases by substantial runtime reductions. The complete stored artifacts and reproduction instructions are retained with the implementation so that the reported measurements can be inspected and rerun.

Chapter 6: Conclusions and Future Work

6.1 Faced Challenges

The main implementation challenges were preserving correctness while sharing branch computations, releasing intermediate arrays only after their final consumer, and limiting generated OpenMP code to loops that pass dependency and conversion checks. The experiments also identify memory usage during branch-level parallel execution as the main remaining graph-backend challenge.

6.2 Gained Experience

Developing Accelerera provided practical experience in designing a modular machine-learning system whose components exchange data, models, reports, and deployment artifacts through explicit interfaces. The team gained experience with tabular, text, and image preprocessing; constrained AutoML search; reproducible model evaluation; and the distinction between training state and the fitted state required for inference.

The project also connected Python development with systems and compiler engineering. Implementing the graph backend, Python–C++ bindings, and Code Parallelizer required work with C++, PyBind11, CMake, Clang ASTs, OpenMP, native-module compilation, and cross-platform build behavior. This work demonstrated how data ownership, dependency analysis, caching, conservative fallback, and serialization directly affect correctness, memory use, and latency.

Finally, integrating the React and Express benchmark platform, MongoDB storage, FastAPI serving, model versioning, telemetry, and container deployment strengthened experience in end-to-end software delivery. Automated tests, continuous integration, controlled comparisons, and output-equivalence checks showed the importance of validating complete workflows rather than only individual functions. The project therefore developed both machine-learning knowledge and the engineering discipline needed to make experimental software repeatable, testable, and usable across environments.

6.3 Conclusions

Accelerera provides a connected framework for preparing data, selecting models, building graph-based machine learning workflows, improving supported custom code, deploying trained models, and comparing results through a benchmark platform. The work shows that the project is not only a collection of separate tools, but a workflow where each module can support a different stage of the machine learning process.

The Auto Preprocessing module reduces repeated manual preparation work for tabular, text, and image datasets. It saves fitted artifacts so that testing data can be transformed in the same way as training data. The AutoML module adds automated model search under time and trial limits, which helps compare candidate models more systematically. Accelerera Pipe

represents workflows as graphs, so shared preprocessing, branches, predictions, and metrics can be organized more clearly. The Code Parallelizer adds a source-level optimization path for supported loop-heavy code while keeping conservative safety checks.

The Deployment module and Benchmark Platform complete the workflow by supporting model serving, version tracking, submission validation, scoring, and leaderboard comparison. The testing and experimental results show that the implemented modules can produce usable outputs, generate reports, evaluate models, and compare results. Overall, Accelera gives a practical base for a more organized and reusable machine learning workflow.

6.4 Future Work

Future work should continue reducing graph memory consumption while preserving parallel branch execution, and should extend safe converter , loop-analysis coverage without weakening correctness checks, The preprocessing pipeline can be improved by adapting the preprocessing steps based on the selected model type and by adding techniques to handle imbalanced datasets. In the future, the benchmark platform can be improved by saving uploaded datasets on the server instead of using Google Drive links. It can also allow users to upload code files directly, instead of submitting only links.

Reference

- [1] Matthieu Komorowski, Dominic C. Marshall, Justin D. Saliccioli, and Yves Crutain. “Exploratory data analysis.” In *Secondary Analysis of Electronic Health Records*, pages 185–203. Springer, 2016.
- [2] Mohammad Hossin and Md Nasir Sulaiman. “A review on evaluation metrics for data classification evaluations.” *International Journal of Data Mining and Knowledge Management Process*, 5(2):1, 2015.
- [3] Navin Sabharwal and Amit Agrawal. “Introduction to Natural Language Processing.” In *Hands-on Question Answering Systems with BERT: Applications in Neural Networks and Natural Language Processing*, pages 1–14. Springer, 2021.
- [4] Koukaras Paraskevas and Tjortjis Christos. “Data preprocessing and feature engineering for data mining: Techniques, tools, and best practices.” *AI*, 6(10):257, 2025. MDPI AG.
- [5] Chris Thornton, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. “Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms.” In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2013.
- [6] Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Springenberg, Manuel Blum, and Frank Hutter. “Efficient and robust automated machine learning.” In *Advances in Neural Information Processing Systems*, 2015.
- [7] Randal S. Olson, Nathan Bartley, Ryan J. Urbanowicz, and Jason H. Moore. “Evaluation of a tree-based pipeline optimization tool for automating data science.” In *Applications of Evolutionary Computation*, 2016.
- [8] Chi Wang, Qingyun Wu, Markus Weimer, and Erkang Zhu. “FLAML: A fast and lightweight AutoML library.” In *Proceedings of Machine Learning and Systems*, 2021.
- [9] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, P. Prettenhofer, R. Weiss, V. Dubourg, and others. “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [10] T. Kadosh, N. Hasabnis, P. Soundararajan, V. A. Vo, M. Capota, N. Ahmed, Y. Pinter, and G. Oren, “OMPar: Automatic Parallelization with AI-Driven Source-to-Source Compilation,” arXiv preprint arXiv:2409.14771, 2024.
- [11] Nick Erickson et al. “AutoGluon-Tabular: Robust and accurate AutoML for structured data.” *arXiv preprint arXiv:2003.06505*, 2020.
- [12] Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. “Sequential model-based optimization for general algorithm configuration.” In *Learning and Intelligent Optimization*, 2011.

- [13] James Bergstra and Yoshua Bengio. "Random search for hyper-parameter optimization." *Journal of Machine Learning Research*, 13:281–305, 2012.
- [14] David H. Wolpert. "Stacked generalization." *Neural Networks*, 5(2):241–259, 1992.

Appendix A: Development Platforms and Tools

A.1 Software Platform

The implemented modules use Python, C++, PyBind11, CMake, Clang-based AST analysis, OpenMP, NumPy, Pandas, Matplotlib, PyTorch, Scikit-learn, Node.js, React, and MongoDB. The Auto Preprocessing module depends mainly on Python data-science libraries. The Pipeline Reporting module uses Python, HTML report generation, Matplotlib, Pandas tables, and Graphviz for graph visualization. The Benchmark Platform uses a Node.js and Express backend, MongoDB for storing benchmarks and submissions, React for the frontend, and Python scripts for dataset validation and metric scoring. The available experiments include Linux and Windows execution paths.

Appendix B: Use Cases

B.1 Auto Preprocessing Use Cases

The user can use Auto Preprocessing when they have a dataset and want to prepare it before training a model. For tabular data, the module can clean missing values, encode categorical columns, scale numerical columns, select features, save artifacts, and generate a report. For text data, it can clean sentences, tokenize them, apply TF-IDF, encode labels, and generate a report. For image data, it can validate images, map class folders to labels, resize images, apply augmentation, and create DataLoaders.

B.2 AutoML Use Cases

The user can use Accelera AutoML when they need a competitive supervised tabular model without manually testing many algorithms and hyperparameter settings. The module supports classification and regression workflows through `AutoMLClassifier` and `AutoMLRegressor`.

The main use cases are fitting a classifier and obtaining predicted labels or class probabilities, fitting a regressor and obtaining numeric predictions, restricting the search to an allowed subset of model families, selecting the evaluation metric and cross-validation folds, limiting the total search budget or per-trial timeout, enabling or disabling meta-learning warm starts and ensemble construction, and inspecting the best score, leaderboard, and complete run history.

B.3 Pipeline Reporting Use Cases

The user can use the pipeline report after running an Accelera pipeline. The report helps show the graph structure, branches, selected path, and metric results. This is useful when the pipeline contains more than one branch and the user wants to understand which path was selected.

B.4 Benchmark Platform Use Cases

The benchmark platform can be used when different users or models need to be compared on the same task. A benchmark creator defines the dataset, test file, target column, and metric. Users then submit prediction files, and the platform calculates the score and displays the leaderboard.

Appendix C: User Guide

C.1 Using Tabular Auto Preprocessing

To use tabular preprocessing, the user first loads the dataset as a Pandas DataFrame. Then the user gives the target column, problem type, and output folder. The output folder is important because the module saves the fitted preprocessing objects and the HTML report inside it.

Code Example C.1: Calling tabular training preprocessing

```
import pandas as pd
from accelera.src.auto_preprocessing.core.classical_training_preprocessing import (
    ClassicalTrainingPreprocessing,
)

df = pd.read_csv("dataset.csv")

preprocessor = ClassicalTrainingPreprocessing(
    df=df,
    target_col="target",
    problem_type="classification",
    folder_path="outputs/tabular_run",
    val_size=0.2,
    random_state=42,
    is_report=True,
)

X_train, y_train, X_val, y_val = preprocessor.common_preprocessing()
```

For regression, the user only changes the problem type:

Code Example C.2: Changing tabular preprocessing to regression

```
preprocessor = ClassicalTrainingPreprocessing(
    df=df,
    target_col="price",
    problem_type="regression",
    folder_path="outputs/regression_run",
)
```

After training preprocessing, the same folder should be used for test data. This makes the test data use the saved training transformers instead of fitting new ones.

Code Example C.3: Calling tabular testing preprocessing

```
import pandas as pd
from accelera.src.auto_preprocessing.core.classical_testing_preprocessing import (
    ClassicalTestingPreprocessing,
)
```

```
test_df = pd.read_csv("test.csv")

test_preprocessor = ClassicalTestingPreprocessing(
    df=test_df,
    folder_path="outputs/tabular_run",
)

X_test, y_test = test_preprocessor.common_preprocessing()
```

C.2 Using Text Auto Preprocessing

For text preprocessing, the user provides the text column and the target column. The text column must contain text values, and the target is used for classification.

Code Example C.4: Calling text training preprocessing

```
import pandas as pd
from accelera.src.auto_preprocessing.core.text_training_preprocessing import (
    TextTrainingPreprocessing,
)

df = pd.read_csv("reviews.csv")

text_preprocessor = TextTrainingPreprocessing(
    df=df,
    target_col="label",
    text_col="review",
    folder_path="outputs/text_run",
    tfidf_max_features=500,
    is_report=True,
)

X_train, y_train, X_val, y_val = text_preprocessor.common_preprocessing()
```

For testing text data, the user again passes the same folder path. If the test data does not contain the target column, the class returns the transformed text features and `None` for the labels.

Code Example C.5: Calling text testing preprocessing

```
from accelera.src.auto_preprocessing.core.text_testing_preprocessing import (
    TextTestingPreprocessing,
)

test_text = pd.read_csv("reviews_test.csv")

text_test_preprocessor = TextTestingPreprocessing(
    df=test_text,
    folder_path="outputs/text_run",
)
```

```
X_test, y_test = text_test_preprocessor.common_preprocessing()
```

C.3 Using Image Auto Preprocessing

For image preprocessing, the dataset should be arranged in class folders. For example:

```
Training/
  Cats/
  Dogs/
```

The user passes the training folder and the output folder. If a validation folder already exists, it can be passed also. If not, the module can split the training folder.

Code Example C.6: Calling image training preprocessing

```
from accelera.src.auto_preprocessing.core.classification_image_training_preprocessing import (
    ClassificationImageTrainingPreprocessing,
)

image_preprocessor = ClassificationImageTrainingPreprocessing(
    training_folder_images="data/PetImages/Training",
    folder_path="outputs/image_run",
    split_training=True,
    val_size=0.2,
    images_size=(224, 224),
    batch_size=16,
    augment=True,
)

train_loader, val_loader = image_preprocessor.common_preprocessing()
```

For image testing, the user provides a list of image paths. If class names are available, they can be passed too. If not, the loader can still be used for prediction.

Code Example C.7: Calling image testing preprocessing

```
from accelera.src.auto_preprocessing.core.classification_image_testing_preprocessing import (
    ClassificationImageTestingPreprocessing,
)

image_paths = ["test/cat1.jpg", "test/dog1.jpg"]
class_names = ["Cats", "Dogs"]

image_test_preprocessor = ClassificationImageTestingPreprocessing(
    image_paths=image_paths,
    image_class_names=class_names,
    folder_path="outputs/image_run",
    batch_size=4,
)

test_loader, invalid_images = image_test_preprocessor.common_preprocessing()
```

C.4 Reading Auto Preprocessing Reports

After running preprocessing with `is_report=True`, the user opens the generated `report.html` file from the output folder. The report shows the dataset overview, missing values, duplicate rows, dropped columns, graphs, preprocessing decisions, selected features, and processed data samples.

C.5 Using Accelera AutoML

Accelera AutoML follows a Scikit-learn-style workflow. The user imports either `AutoMLClassifier` or `AutoMLRegressor`, creates an estimator with the required search settings, fits it on the training data, and then uses the fitted model for prediction, scoring, or leaderboard inspection.

Code Example C.8: Basic AutoML classification workflow

```
from accelera.src.accelera_automl import AutoMLClassifier

model = AutoMLClassifier(
    time_budget=1800,
    cv=5,
    scoring="accuracy",
    use_ensemble=True,
    random_state=42,
)

model.fit(X_train, y_train)

predictions = model.predict(X_test)
score = model.score(X_test, y_test)
leaderboard = model.return_leaderboard()
```

In this example, the search is limited to 1,800 seconds, five-fold cross-validation is used, and accuracy is selected as the optimization metric. After fitting, the model can generate predictions, evaluate performance on test data, and return the stored leaderboard for inspection.

Code Example C.9: Basic AutoML regression workflow

```
from accelera.src.accelera_automl import AutoMLRegressor

model = AutoMLRegressor(
    time_budget=1800,
    cv=5,
    scoring="r2",
    n_trials=50,
    use_meta_learning=True,
    random_state=42,
)

model.fit(X_train, y_train)

predictions = model.predict(X_test)
```



```
r2_score = model.score(X_test, y_test)
leaderboard = model.return_leaderboard(top_n=5)
```

The same public workflow is used for regression. The main difference is that the selected score is a regression metric, such as R^2 , and the predictions are numeric values instead of class labels.

C.6 Using Pipeline Reports

To generate a pipeline report, the pipeline graph should be serialized first. Then the user passes the XML graph path, output folder, and metric results to `GraphReport`.

Code Example C.10: Calling the pipeline graph report

```
from accelera.src.accelera_pipe.wrappers.graph_report import GraphReport

report = GraphReport(
    folderpath="outputs/pipeline_report",
    xmlpath="outputs/pipeline_graph.xml",
    results=metric_results,
)

report.execute()
```

The output is a `report.html` file. It contains the graph image, branch tables, selected branch, and metric summary.

C.7 Using the Benchmark Platform

The benchmark platform is used from the web interface. The benchmark creator enters the title, problem type, target column, dataset link, test-set link, target-label link, and evaluation metric. A user submits a prediction file link and repository link. The backend calls the Python scoring script, saves the score, and displays the result in the leaderboard.

The prediction file must contain the target column name expected by the benchmark. Also, the number of rows in the prediction file must match the number of rows in the target-label file.

Appendix D: Feasibility Study

D.1 Purpose and Evaluation Criteria

This chapter checks whether Accelerera is practical to build, run, and improve. The study covers the full project: Auto Preprocessing, AutoML, Accelerera Pipe, the Code Parallelizer, the Benchmark Platform, and Deployment. Since this is a student project and some parts are still prototypes, the chapter separates what already works from what still needs more production-level work.

The evaluation is based on seven simple points: technical feasibility, operational feasibility, economic feasibility, schedule feasibility, legal and ethical feasibility, security feasibility, and scalability. In simple words, the study asks whether the tools exist, whether users can run the system, whether the cost is reasonable, whether the work can be delivered in stages, whether the project respects data and license rules, whether it is safe to use, and whether it can grow later.

In this chapter, a module is called **feasible** when its main workflow can be implemented and tested with available tools. It is called **feasible with constraints** when it can work, but still depends on extra requirements such as compilers, cloud accounts, hardware limits, or stronger production controls. A **prototype-feasible** result means that the idea and core workflow are shown, but the module still needs more security, testing, or scaling before public production use.

D.2 Technical Feasibility

D.2.1 Architecture and Technology Availability

Accelerera is technically feasible because it is built from common and available technologies. Python is used for the public APIs and machine-learning libraries such as NumPy, Pandas, and Scikit-learn. C++20 and PyBind11 are used for the native graph backend and Python bindings. CMake handles native builds, while LLVM/Clang and OpenMP support loop analysis and parallel code generation. The web platform uses React, Express, and MongoDB. The Deployment module uses FastAPI and Docker, with optional commands for Heroku and EC2. These tools are well known and do not require special hardware.

Table D.1: Module-level technical feasibility assessment.

Module	Implementation evidence and resources	Constraints and assessment
Auto Preprocessing	Uses Pandas, NumPy, Scikit-learn, text tools, image tools, saved transformers, and HTML reports. Training and testing can reuse saved preprocessing artifacts.	Feasible. It needs valid input columns and enough storage for reports and artifacts. Very unusual data types may need more rules.
AutoML	Supports classification and regression search, trial and time budgets, meta-learning support, and ensembles using existing estimator libraries.	Feasible with constraints. Search time grows with dataset size, number of trials, and number of model families. Fixed seeds and versions are needed for repeatable results.
Accelera Pipe	Uses a Python graph API, a C++ execution backend, PyBind11 bindings, branch scheduling, shared nodes, memory release, metrics, path selection, and pickle saving/loading.	Feasible with constraints. Native compilation is required. Parallel branches can improve speed, but branch width and dataset size must be controlled to avoid high memory use.
Code Parallelizer	Uses Python-to-C++ conversion, Clang AST loop extraction, safety rules, classifier support, OpenMP insertion, PyBind11 compilation, and a compiled-module cache.	Feasible with constraints. It supports only a documented subset of Python and analyzable loops. LLVM, Clang, a C++ compiler, and OpenMP must be available. Unsupported functions fall back to Python.
Benchmark Platform	Uses a React frontend, an Express backend, MongoDB, authentication helpers, submission records, metrics, and scoring scripts.	Prototype-feasible. The main web technologies are mature, but public use needs stronger upload isolation, authorization tests, rate limiting, monitoring, backups, and more backend tests.
Deployment	Uses FastAPI serving, input validation, telemetry, model artifact loading, Docker packaging, and commands for local Docker, Heroku, and EC2.	Feasible with constraints. Local Docker deployment is practical. Remote deployment needs accounts, credentials, network access, supported CLIs, health checks, and proper secret management.

D.2.2 Integration and Portability

Integration is feasible because the modules exchange normal artifacts such as arrays, data frames, fitted models, preprocessing objects, graph objects, metric results, configuration files, and saved models. Auto Preprocessing can prepare data for AutoML or Accelera Pipe. Accelera Pipe can call the Parallelizer for supported custom preprocessing functions. Selected models or executed graphs can then be used by the Deployment module. The Benchmark Platform is a separate path for comparing prediction files and showing results.

The package supports Python 3.9 or newer and uses CMake with C++20. Linux and Windows are realistic targets because the build logic includes platform-specific LLVM discovery and installation paths. However, this does not mean everything is zero-configuration. Compiler versions, OpenMP runtimes, Node.js, MongoDB, Docker, and cloud CLIs still need clear documentation and testing. Users can still run pure-Python parts even if the native graph or AST tools are not installed.

D.3 Operational Feasibility

Accelera is operationally feasible for students, researchers, and data-science developers who already know basic Python and machine-learning workflows. The APIs use familiar words such as `fit`, `transform`, and `predict`. Reports, graph visualizations, metrics, examples, and saved artifacts make the output easier to understand. The project is modular, so users can run one module without running the full web platform.

Table D.2: Operational workflows and requirements.

Workflow	Main user and output	What is needed
Prepare a dataset	Data scientist; processed data, saved artifacts, and a report.	Correct target and feature settings, writable output folders, and consistent use of training transformations during testing.
Search for a model	Data scientist; ranked trials, selected model, and optional ensemble.	Clear time/trial budget, suitable metric, fixed seed, and enough memory for candidate evaluation.
Build a graph pipeline	Python developer; validation results and an executed graph.	Native graph binding, compatible nodes, validation data for selecting a path, and memory-aware branch design.
Parallelize code	Developer who needs speed; annotated C++ or compiled Python callable.	Supported source syntax, compiler toolchain, correctness comparison, and timing comparison with the original code.

Workflow	Main user and output	What is needed
Run a benchmark	Platform admin and participants; scored submissions and leaderboard.	Running frontend/backend, database connection, available dataset, metric definition, user controls, and submission validation.
Deploy a model	ML or DevOps user; local or remote prediction service.	Valid model/config files, Docker or provider tools, credentials, network access, monitoring, and rollback plan.

The main operational challenge is installation. A full setup may include Python, C++ tools, Node.js, MongoDB, Docker, and optional cloud tools. This is acceptable for a modular research project, but the user experience can be improved with version checks, clearer error messages, prebuilt wheels, containerized services, and smoke tests. The Parallelizer fallback also helps because unsupported code continues to run in Python instead of breaking the full workflow.

D.4 Performance, Scalability, and Maintenance Feasibility

Accelera can scale on one machine and can also grow at service boundaries, but each module has a different limit. Auto Preprocessing is limited mainly by data size and transformation cost. AutoML is limited by the number and cost of trials. Accelera Pipe is limited by branch width, model training time, and memory. The Parallelizer is limited by the supported source patterns and whether the target loop actually benefits from parallelism. The Benchmark Platform is limited by database traffic, storage, and submissions. Deployment is limited by request rate, model latency, container resources, and telemetry growth.

Some controls already support moderate workloads. AutoML has time and trial budgets. Accelera Pipe releases intermediate arrays after their last consumer and keeps node caching optional. The Parallelizer caches generated and compiled code. Docker helps make deployment repeatable, and MongoDB stores benchmark records. Further scaling needs load tests, queues, artifact cleanup rules, indexes, isolation, and monitoring. Parallel execution should be enabled based on timing measurements, not just assumed to help every workload.

Maintenance is feasible because the modules are separated, but the dependency surface is large. The project should clearly define supported Python, Node.js, compiler, LLVM, database, and cloud-tool versions. CI should build native modules on Linux and Windows, run Python tests, lint frontend/backend code, build Docker images, and run a small end-to-end test. Generated caches and models should store version metadata so old artifacts can be rejected or rebuilt safely.

D.5 Overall Feasibility Decision

Table D.3: Overall project feasibility decision.

Dimension	Decision	Condition
Technical	Feasible constraints	with Keep native toolchain support and document module limits.
Operational	Feasible	Keep modular workflows, examples, diagnostics, and clear artifacts.
Security	Prototype-feasible	Add isolation, authentication, secret management, artifact trust, and public-service hardening.
Scalability	Feasible for moderate workloads	Measure memory and latency, limit concurrency, and scale services based on demand.

The final decision is that Accelera is feasible as an academic, research, and controlled prototype platform. The main module workflows use available technologies and can be tested in the repository. The project is also practical for local and small-team use when dependencies and resource limits are respected. Public production use is not fully complete yet. The Benchmark Platform and remote Deployment paths still need stronger security, load testing, monitoring, backups, and clear ownership. These missing parts do not make the design wrong; they show the next engineering step needed to move from a working prototype to a reliable production service.